L7- Data Synchronization & Replication

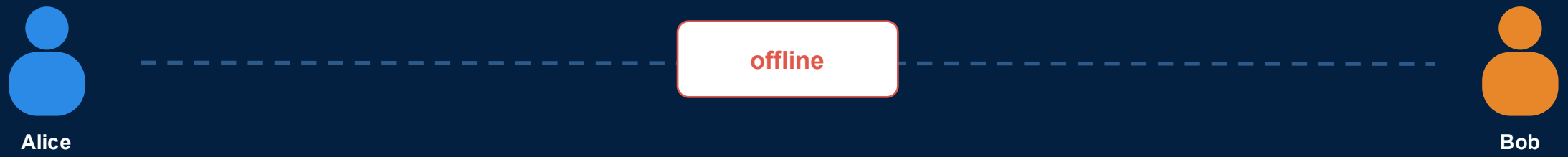
D7024E Mobile and Distributed Computing Systems · Johan Kristiansson · Autumn 2026



LULEÅ UNIVERSITY OF TECHNOLOGY

Two students. One planner. No connection.

Alice and Bob edit their shared study planner while offline. When they reconnect, three things can collide.



Task title

Alice: "Exam plan"

Bob: "Revision plan"

Which title survives?

Shared note

Alice edits the first sentence

Bob edits the last sentence

Can we keep both edits?

Last study-room slot

Alice books Tue 14:00

Bob books Tue 14:00

Can both be confirmed?

The lecture in one map

1 Write

Who may accept a write, and what has ”write" promised?

· Replication vs sharding; single leader, multi-master, quorum

· Received · persisted · applied

· Atomic across replicas: 2PC

· Idempotent retries; STORE in the lab

2 Read

What may a client observe before the replicas agree?

· Read anomalies

· Consistency spectrum, CAP

· Linearizability, sequential, causal

· Monotonic, eventual (Kademlia), best-effort

3 Delete

How does a deletion reach a replica that was offline?

· Resurrection, delete ‖ edit, stale reads

· Tombstones, blocking delete, GC; lifecycle policy

4 Synchronize

Which changes are missing, and how do concurrent edits combine?

· Reconnect: checkpoints, gaps, Merkle repair

· Clocks detect concurrency; CRDTs merge

· Text: OT (Jupiter), sequence CRDTs (LSEQ)

Replication keeps copies; synchronization reconciles them



Replication

Store copies of the same logical data on several devices or servers, for availability and read locality.

Synchronization

Exchange updates between copies and reconcile the differences — this is where the policy lives.

Sharding divides the data; replication copies it

Sharding

Each node holds a different part. Scales storage and throughput.



Replication

Each node holds the same data. Survives node loss; reads can be local.



Sharding divides the data; replication copies it

Sharding

Each node holds a different part. Scales storage and throughput.



Replication

Each node holds the same data. Survives node loss; reads can be local.



Erasure coding

Split the data into k fragments, add m parity fragments; any k of the k + m rebuild everything. Here k = 4, m = 2.



XOR: the one-bit arithmetic behind parity

a	b	a ⊕ b
0	0	0
0	1	1
1	0	1
1	1	0

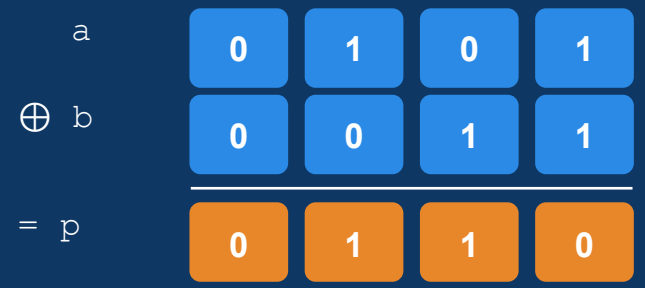
Why XOR undoes itself

$x \oplus x = 0$ and $x \oplus 0 = x$

If $p = a \oplus b$ and a is missing:

$$a = p \oplus b = (a \oplus b) \oplus b = a \oplus (b \oplus b) = a$$

Bit by bit



each column on its own: 0⊕0=0, 1⊕0=1, 0⊕1=1, 1⊕1=0

XOR creates the parity bit that makes each column even

CREATE PARITY
P = 1 ⊕ 0 ⊕ 0 = 1
Stored column: 1, 0, 0, 1
Two ones means even parity

RECOVER LOST D1
D1 = D2 ⊕ D3 ⊕ P
= 0 ⊕ 0 ⊕ 1 = 1
XOR the survivors and the parity bit

XOR calculates the parity bit and later reconstructs one missing bit.

Erasure coding: parity lets any k of n fragments rebuild the rest

Parity by hand: XOR of the data fragments



`P = D1 ⊕ D2 ⊕ D3 = 0101 ⊕ 0011 ⊕ 1001 = 1111`

Node 2 dies. Rebuild D2 from the other three:

`D2 = D1 ⊕ D3 ⊕ P = 0101 ⊕ 1001 ⊕ 1111 = 0011` ✓

One parity equation recovers one unknown: k data + 1 parity survives one loss (RAID-5). A parity fragment is not a copy — it is a computed combination that any single missing fragment can be solved from.

	Storage for 1 GB	Tolerates	Read	Repair one lost node	Update one record
3× replication	3 GB	2 node losses	1 node, no compute	Copy 1 GB from any replica	Write 3 copies
RS(4, 2)	1.5 GB	2 node losses	4 fragments + decode	Read 4 fragments: 4× the lost data	Re-encode: rewrite parity on all 6

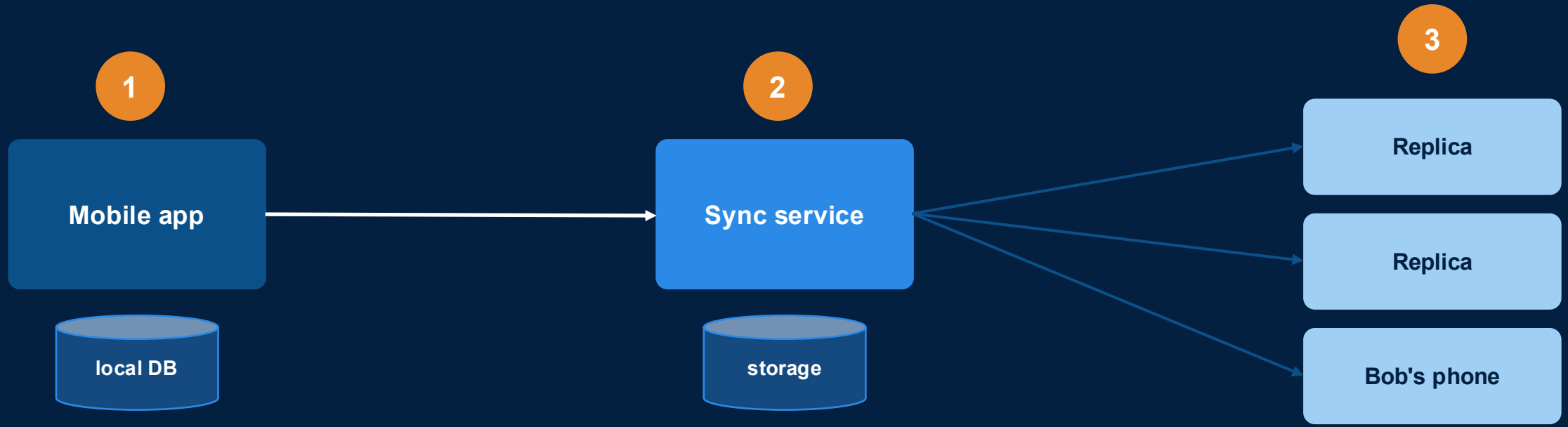
Erasure coding buys storage with read and repair bandwidth. Right for large, cold, immutable objects; wrong for the small, hot, often-updated records this lecture replicates.

Reed–Solomon (1960): https://doi.org/10.1137/0108018

Write

Where does the data live, and who may write to it?

"Write" means three different things



1 Saved on this device

The edit and its pending update are in durable local storage. Survives an app crash.

2 Acknowledged by the service

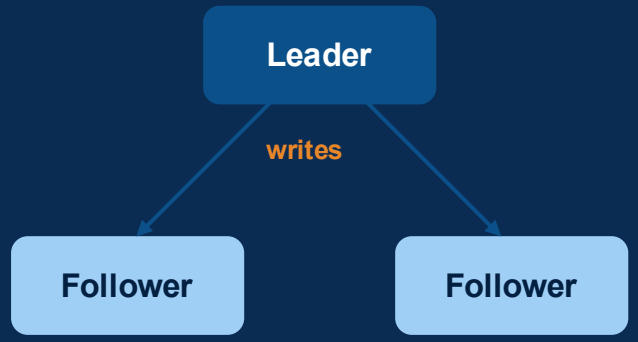
The service has durably stored it. Survives the phone being lost.

3 Visible to others

Other replicas have applied it. Until then, Bob may still see the old state.

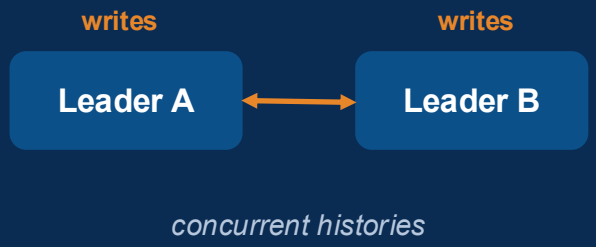
Who is allowed to accept a write?

Single leader



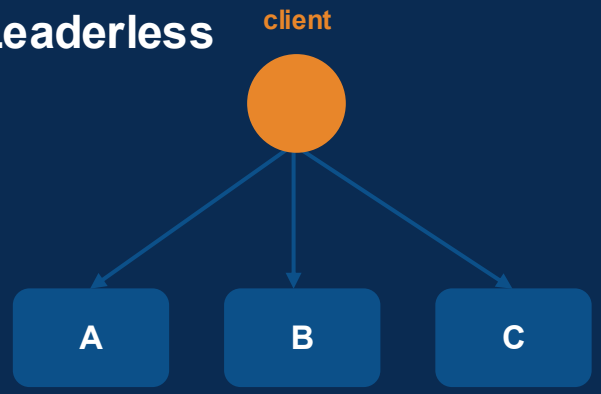
All writes go to one leader, which orders them. Followers receive the ordered updates. Simple to reason about; the leader is a bottleneck and a failover risk.

Multiple leaders



Several sites accept writes — e.g. one per region, or every offline device. Concurrent writes are unavoidable, so a conflict policy is mandatory.

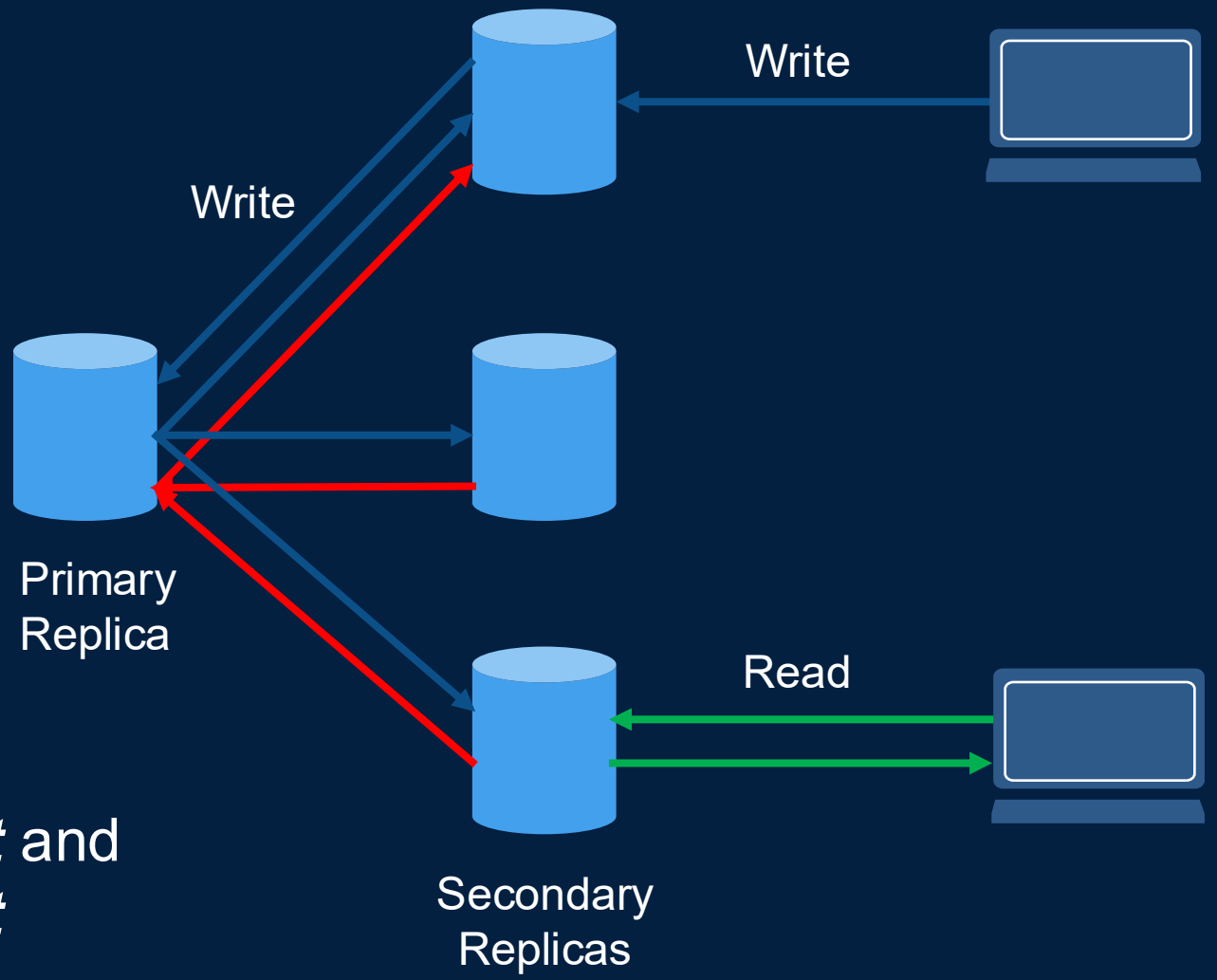
Leaderless



A client writes to and reads from several replicas and waits for a quorum. Replicas hold versions; the read reconciles them.

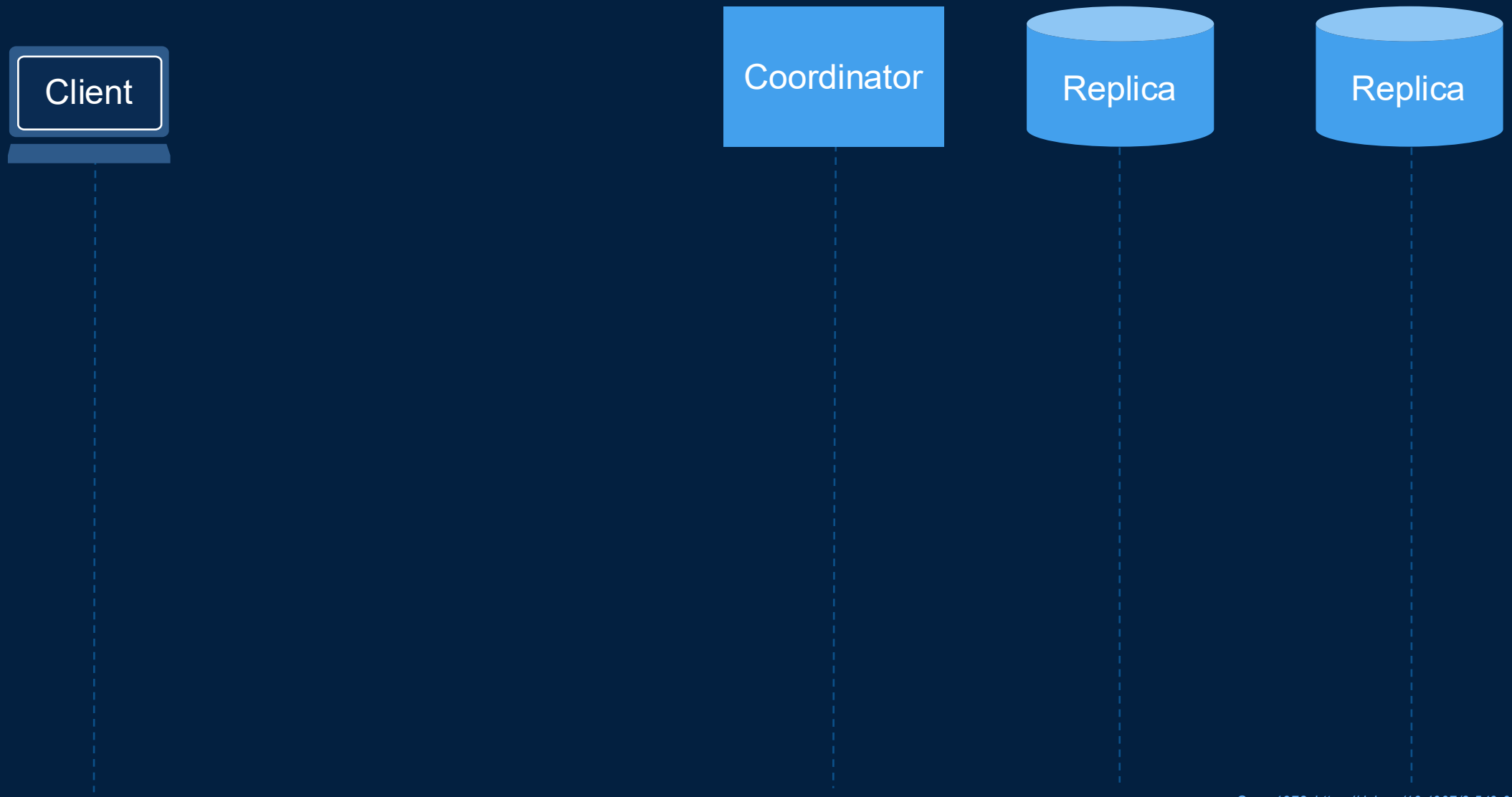
Topology, acknowledgement rule and conflict policy are three separate choices. An offline phone is a leader for its own edits.

Atomic writes across replicas: primary–backup and commit protocols



2-Phase Commit and
3-Phase Commit

Two-phase commit (Gray, 1978)

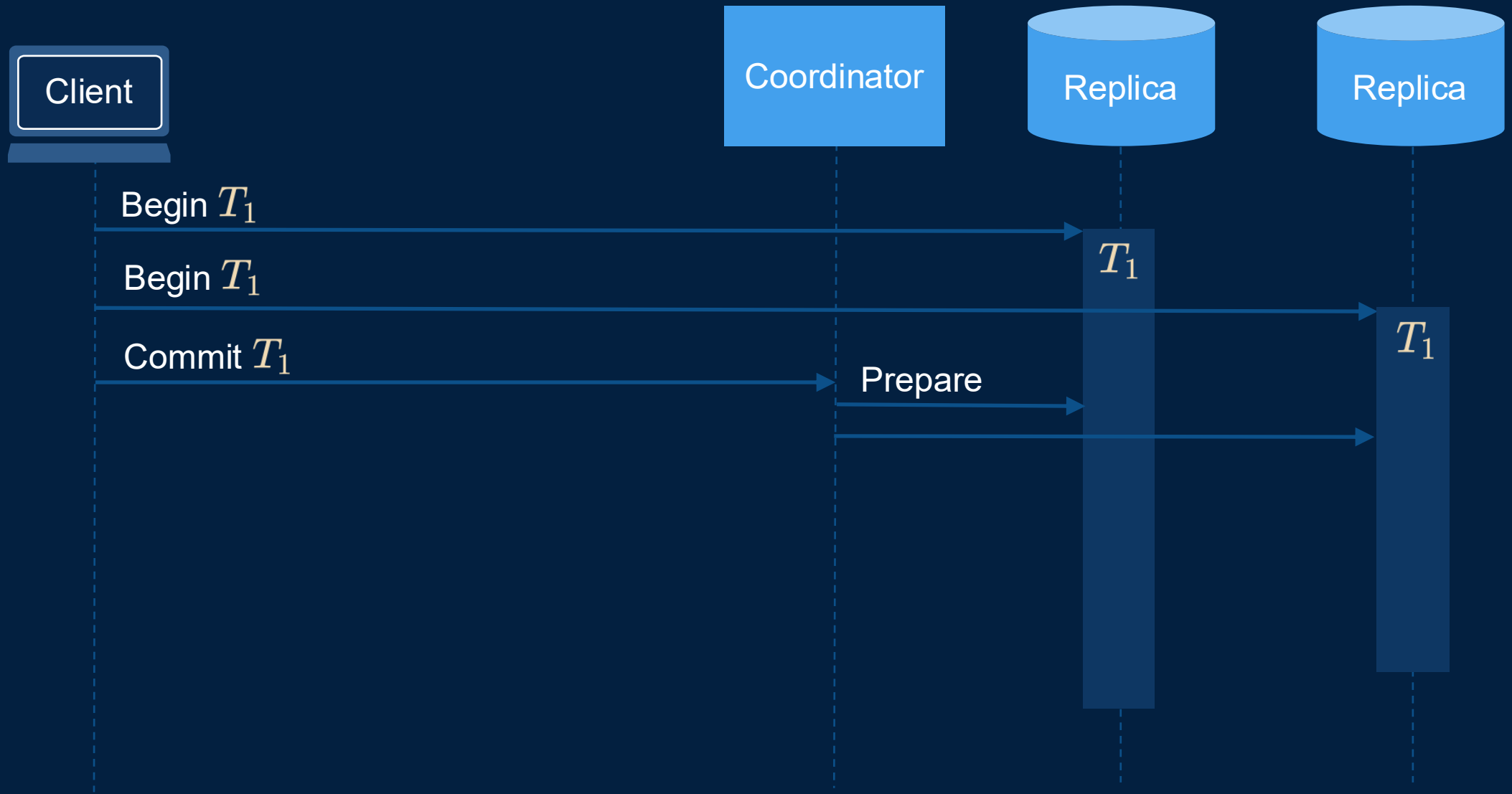


Gray, 1978: https://doi.org/10.1007/3-540-08755-9_9

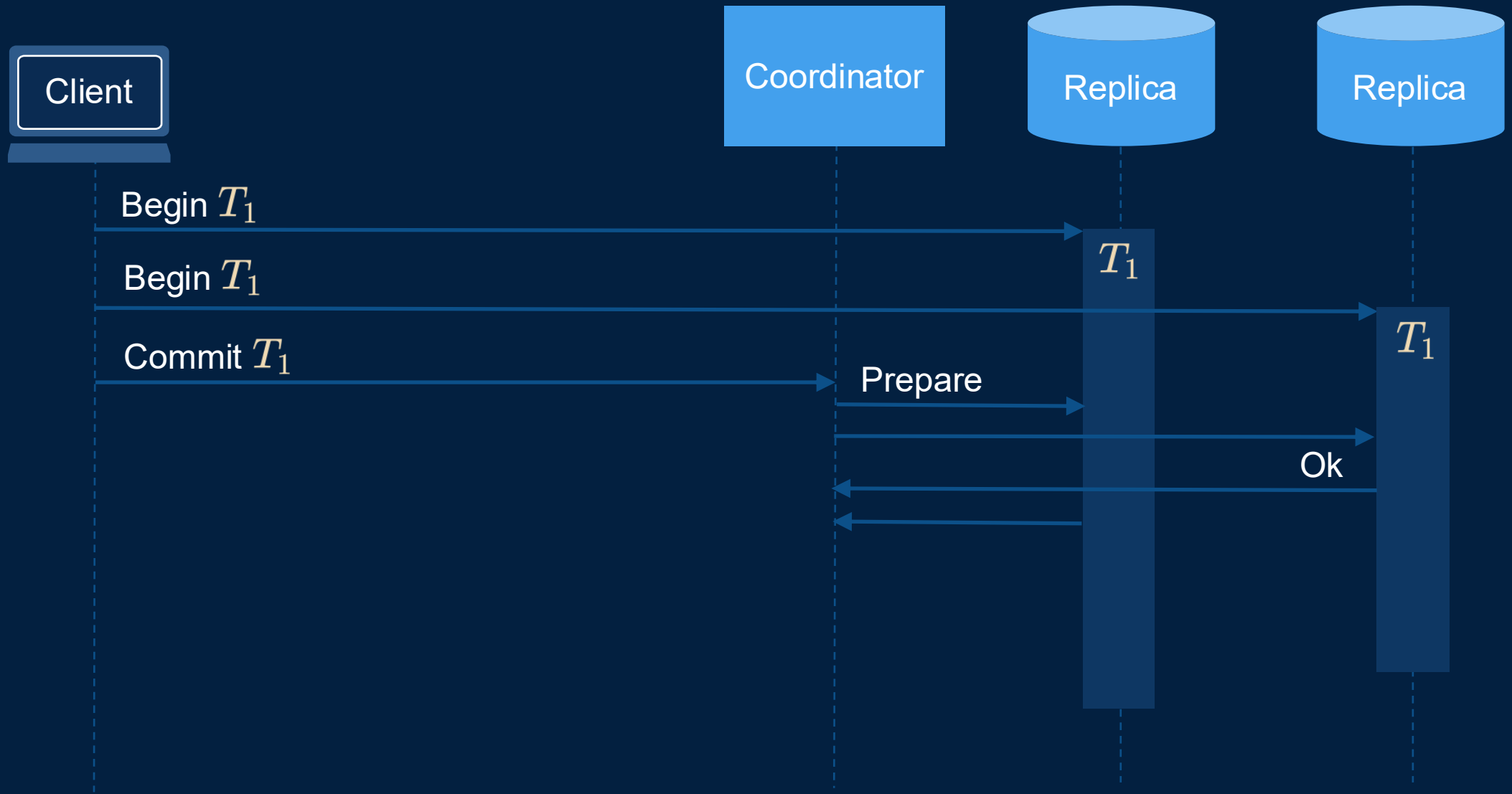
Two-phase commit: begin the transaction



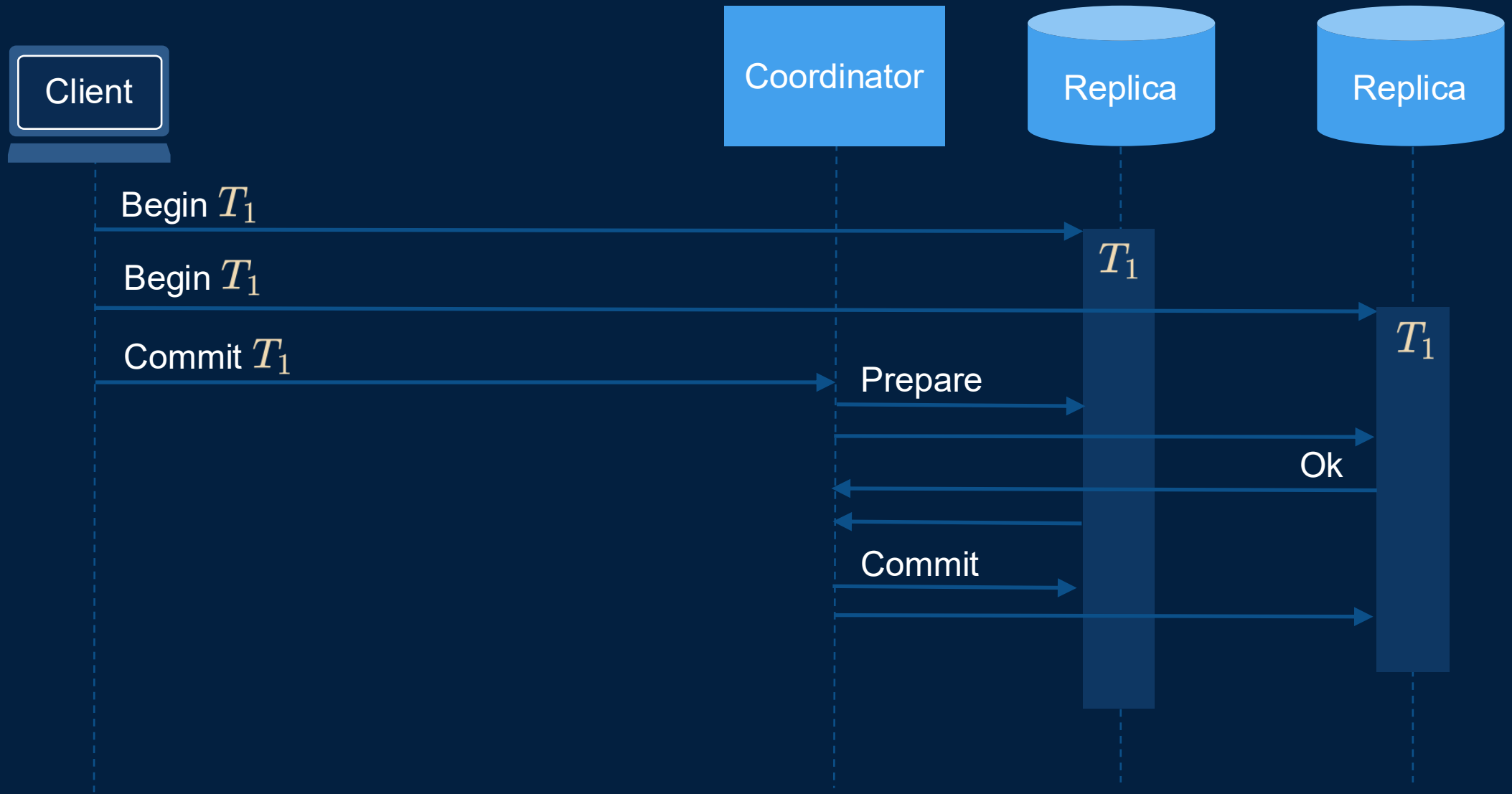
Two-phase commit: phase 1, prepare



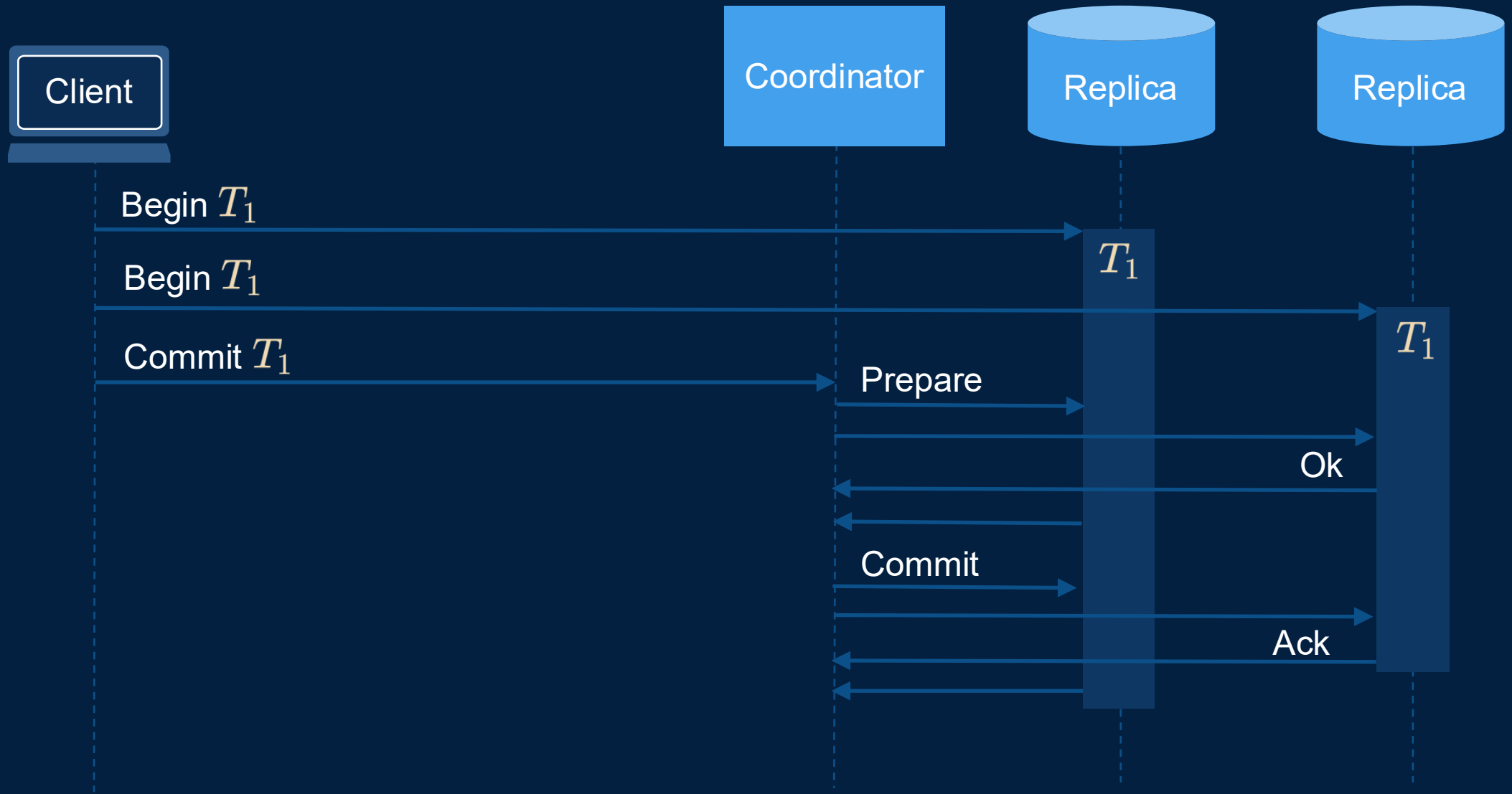
Two-phase commit: participants vote



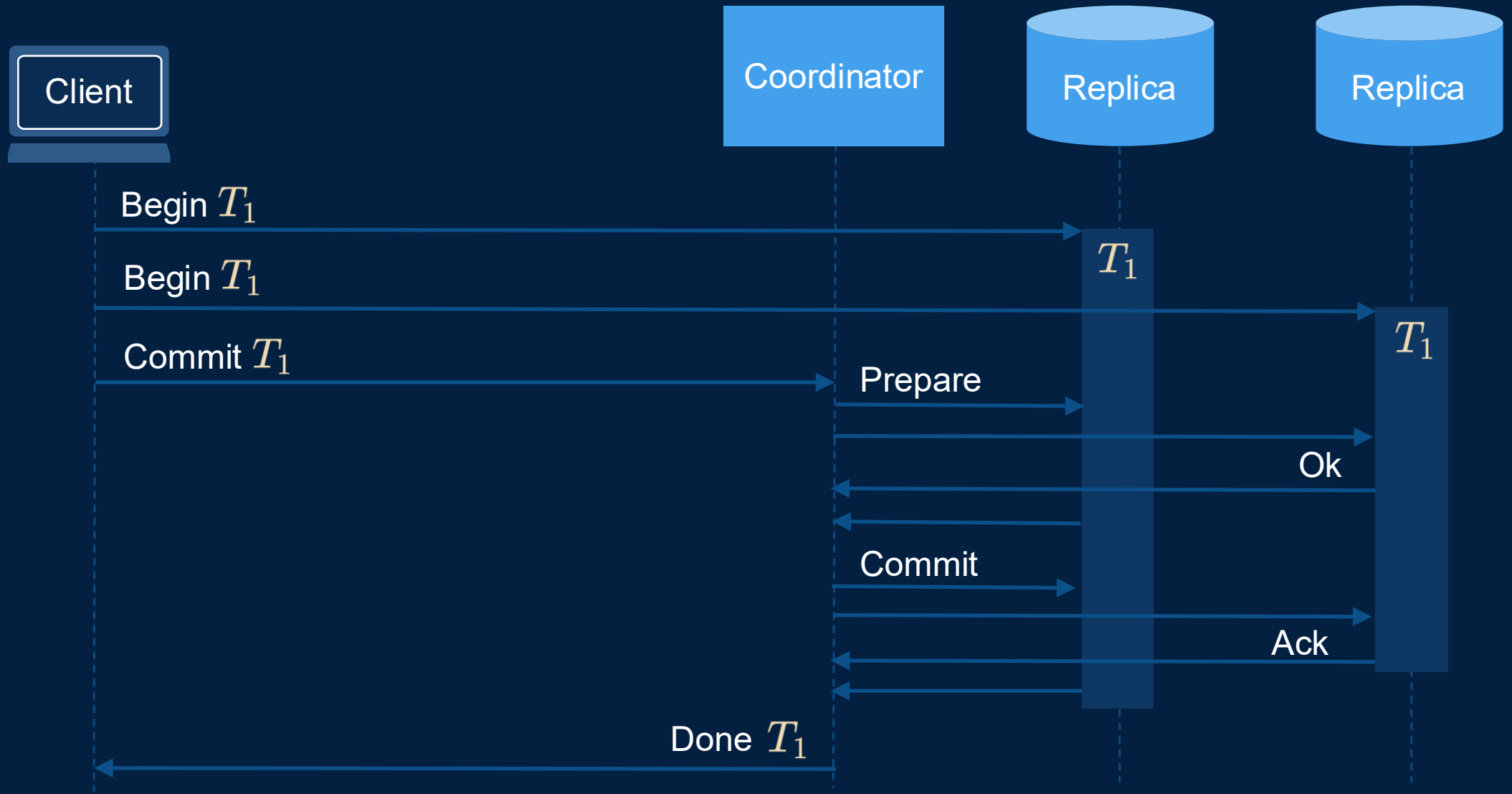
Two-phase commit: phase 2, commit



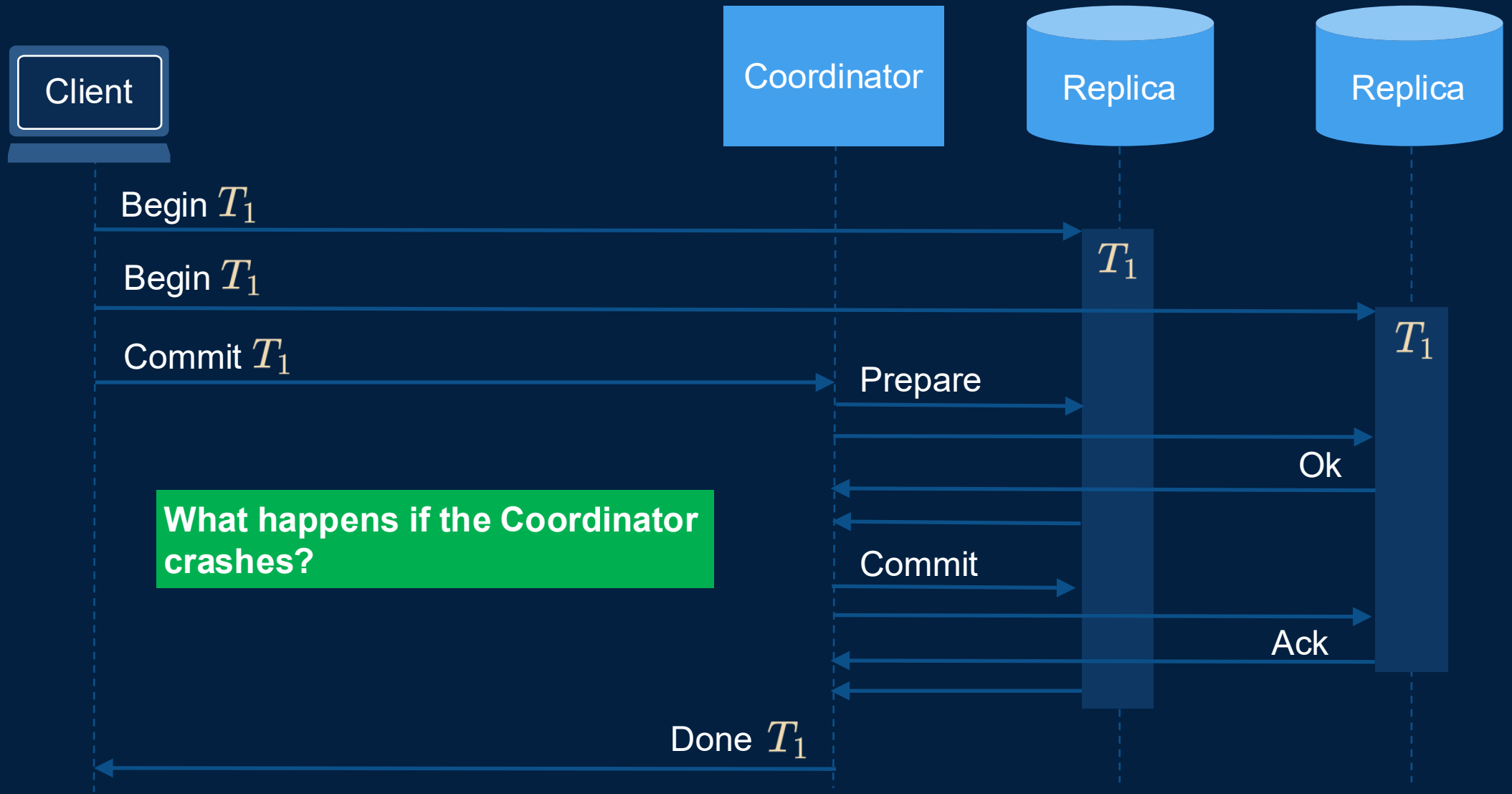
Two-phase commit: acknowledge



Two-phase commit: done



Two-phase commit: what if the coordinator crashes?



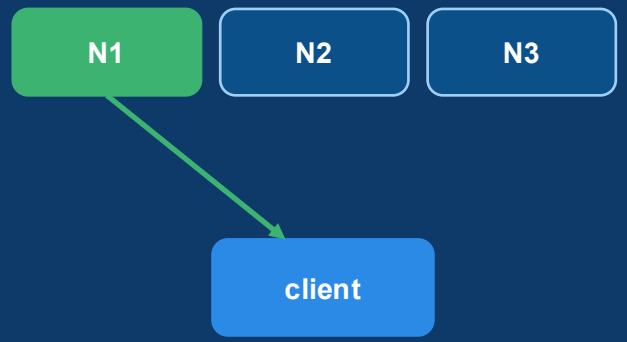
Lab: how many acknowledgements should STORE wait for?

60 s

SCENARIO

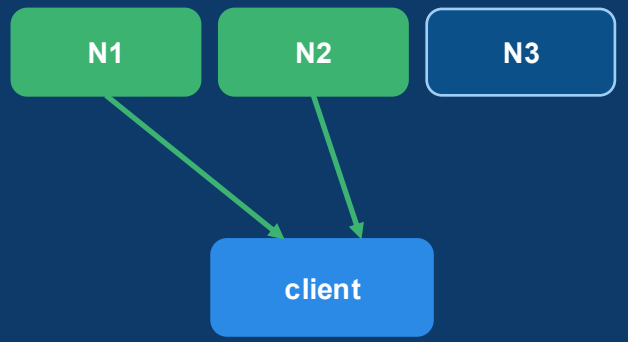
System: Your Kademlia DHT with k = 3. Nodes join and leave continuously (churn).
Situation: STORE(key, value) sends the value to the k closest nodes; some may be slow or already gone.
Requirement: A later FIND_VALUE must find the value with high probability, and STORE must not hang on one dead node.

A After 1 ack



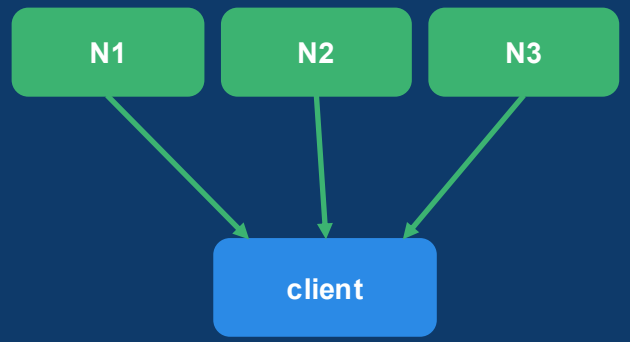
fast · one copy may be the only one

B After a majority (2 of 3)



overlaps a majority read

C After all k



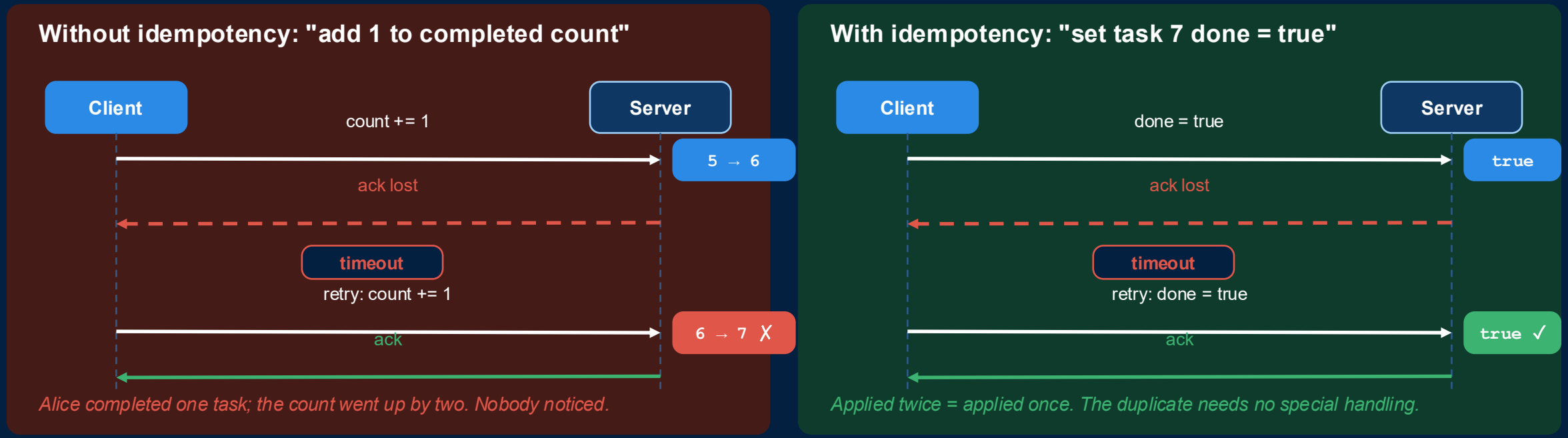
one slow or dead node blocks the write

A 1 ack — fast, weakest guarantee

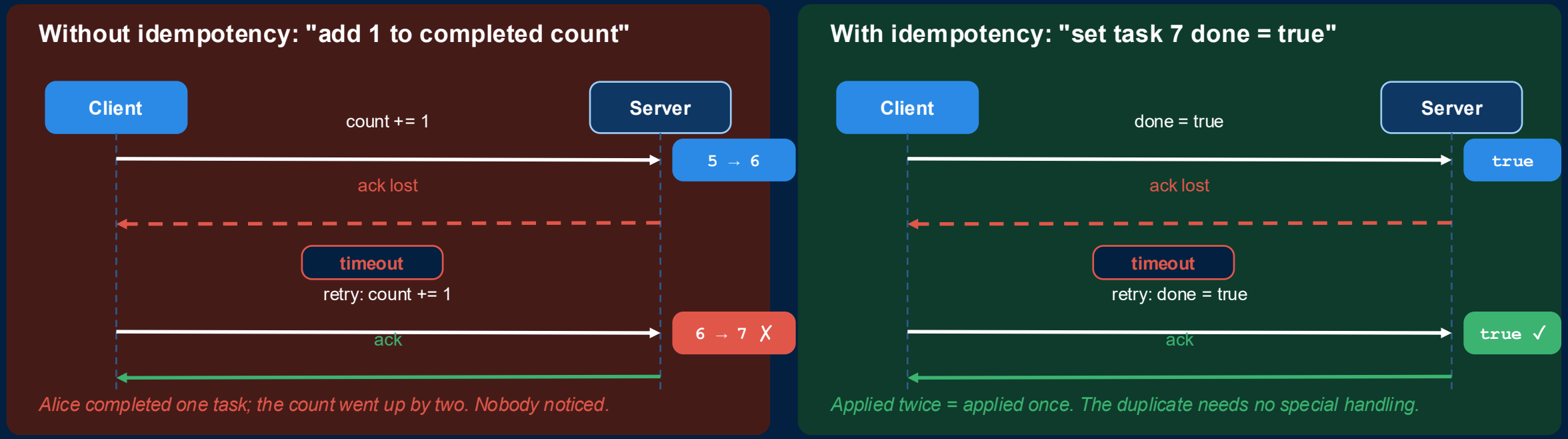
B Majority — durable enough, still available

C All k — strongest, blocked by one node

Idempotency: why a duplicate message is harmless



Idempotency: why a duplicate message is harmless



Making any operation idempotent: give it an identity

`{id: 42, count += 1}` → apply + record 42 (one txn) → `retry {id: 42, …}` → 42 seen → stored result

Every retry protocol creates duplicates. Either the operation is naturally idempotent, or you make it so with an ID and a dedup record — never neither.

Read

What may a client observe before the replicas agree?

Alice writes x = 1 and immediately reads x. What does the read return?

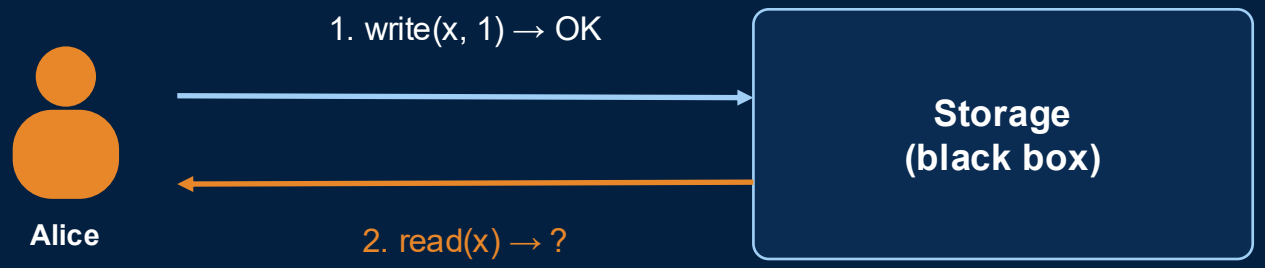
60 s

SCENARIO

System: A storage service that Alice uses through one API.
Internally it may be a single node or a replicated system — *she cannot tell.*

Situation: Alice issues write(x, 1), receives "OK", then issues read(x).x

Task: Predict the value, and say what assumption about the storage your prediction relies on.

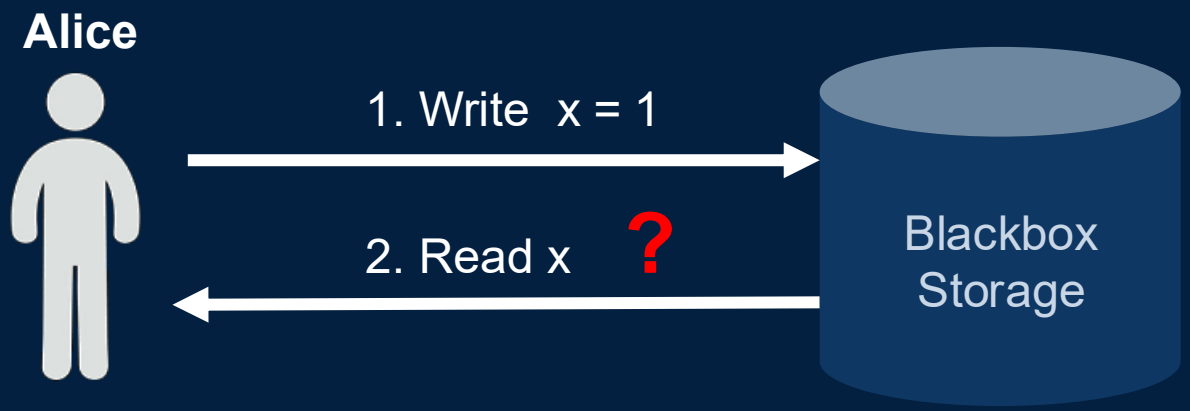


A 1 — the write was acknowledged before the read

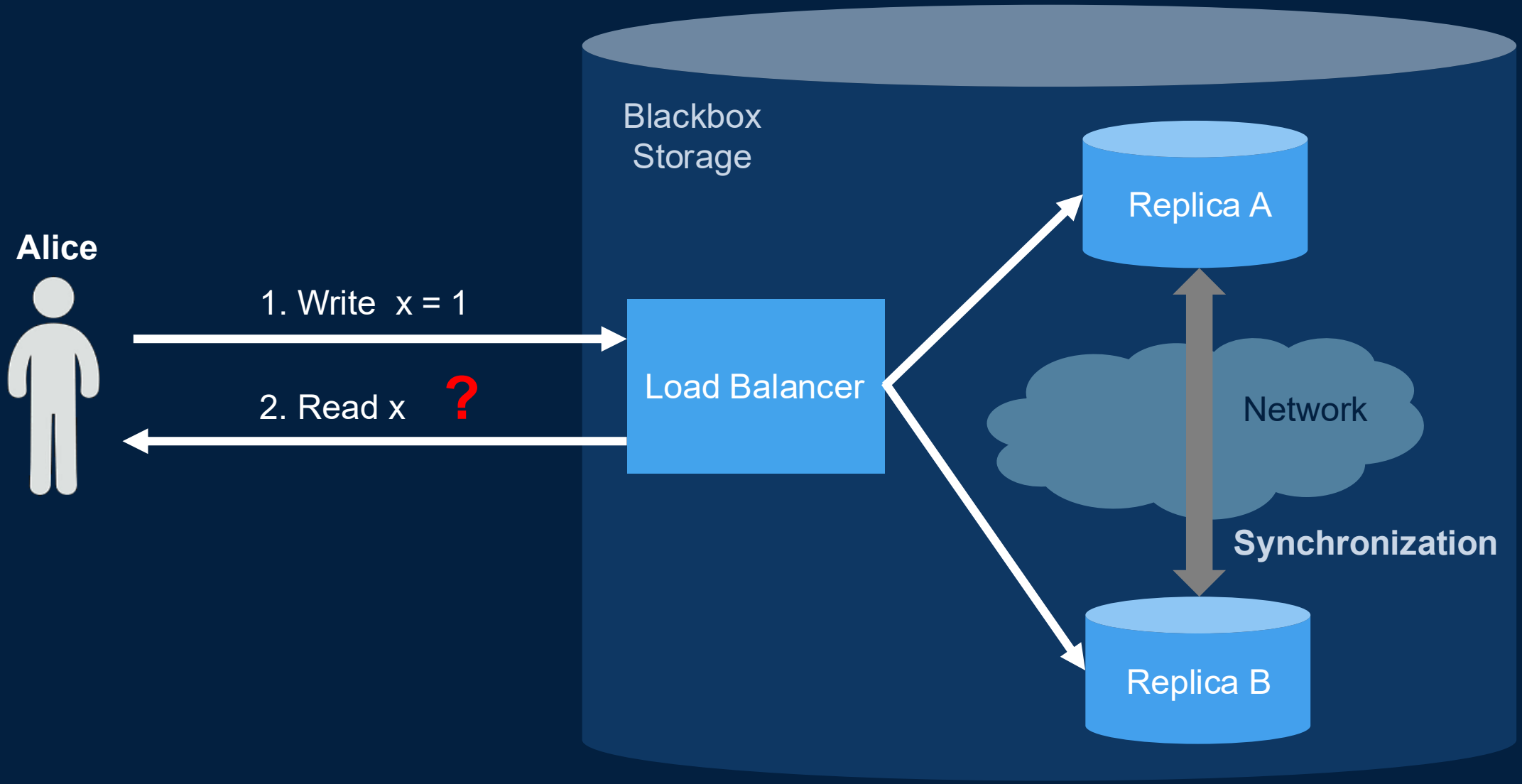
B 1 or nothing — it depends on which replica answers

C Cannot say without knowing the consistency model

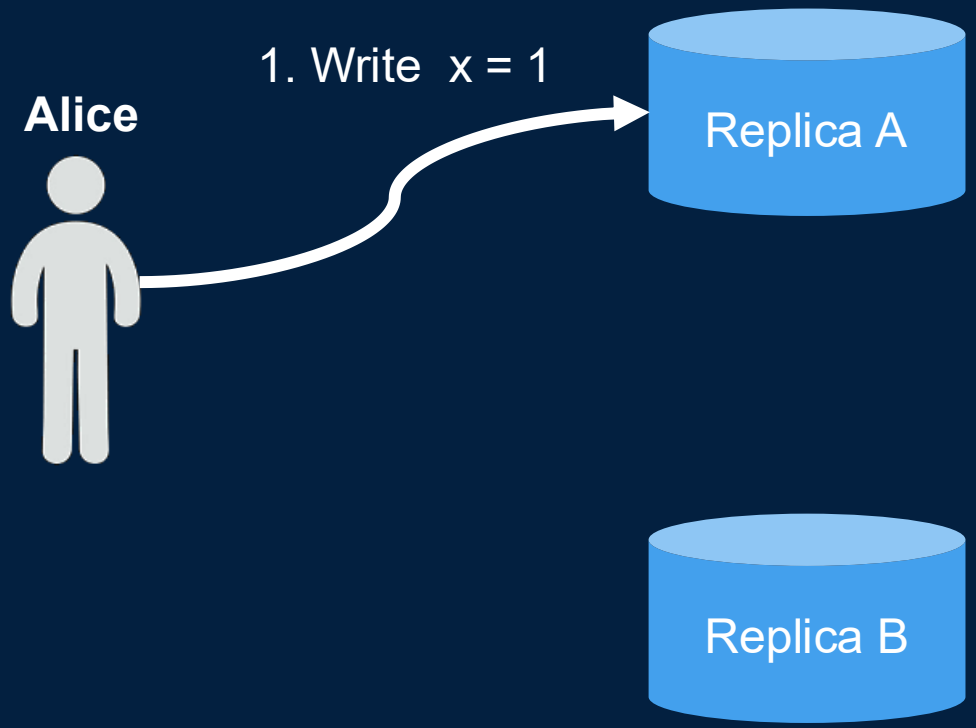
What will the read return?



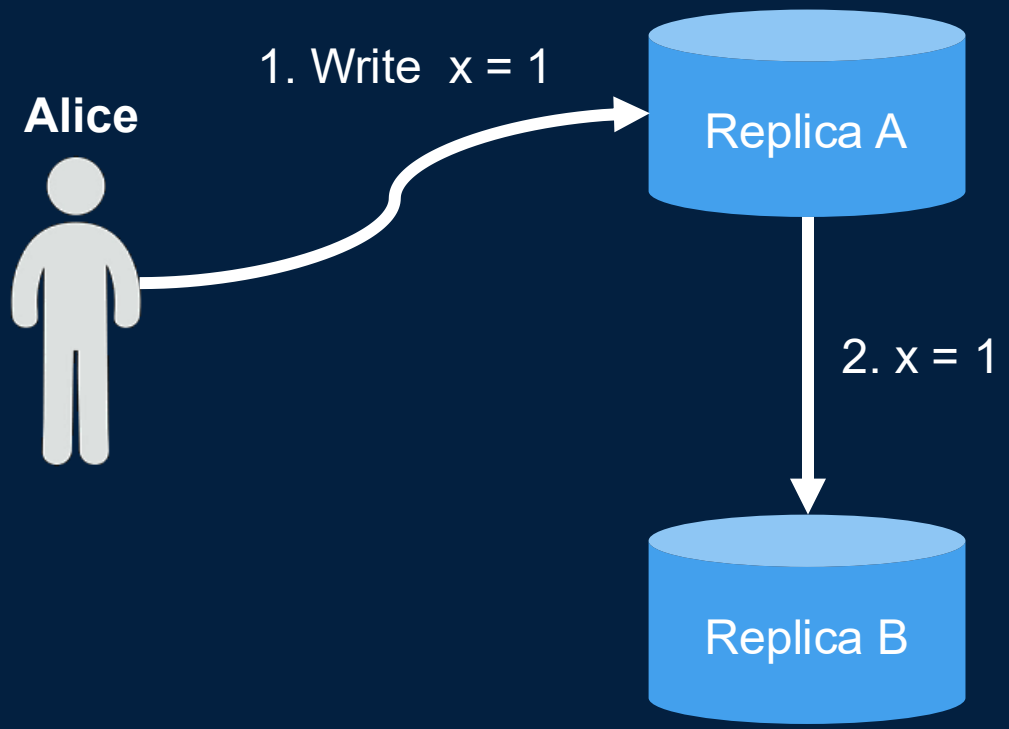
The black box is a distributed system



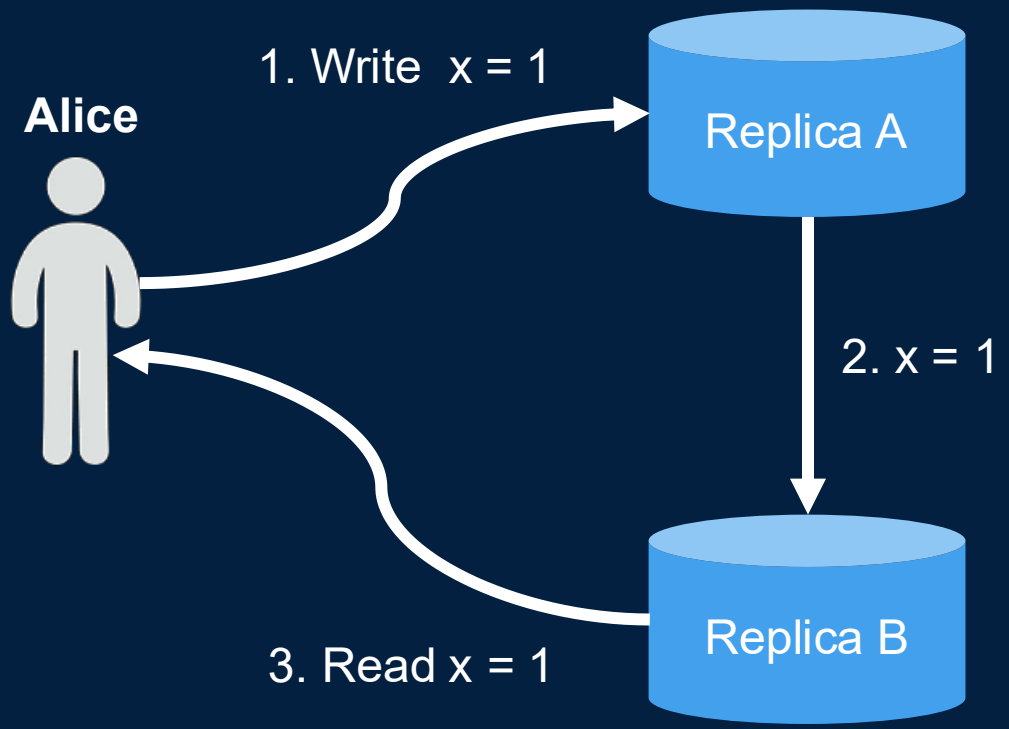
What will the read return?



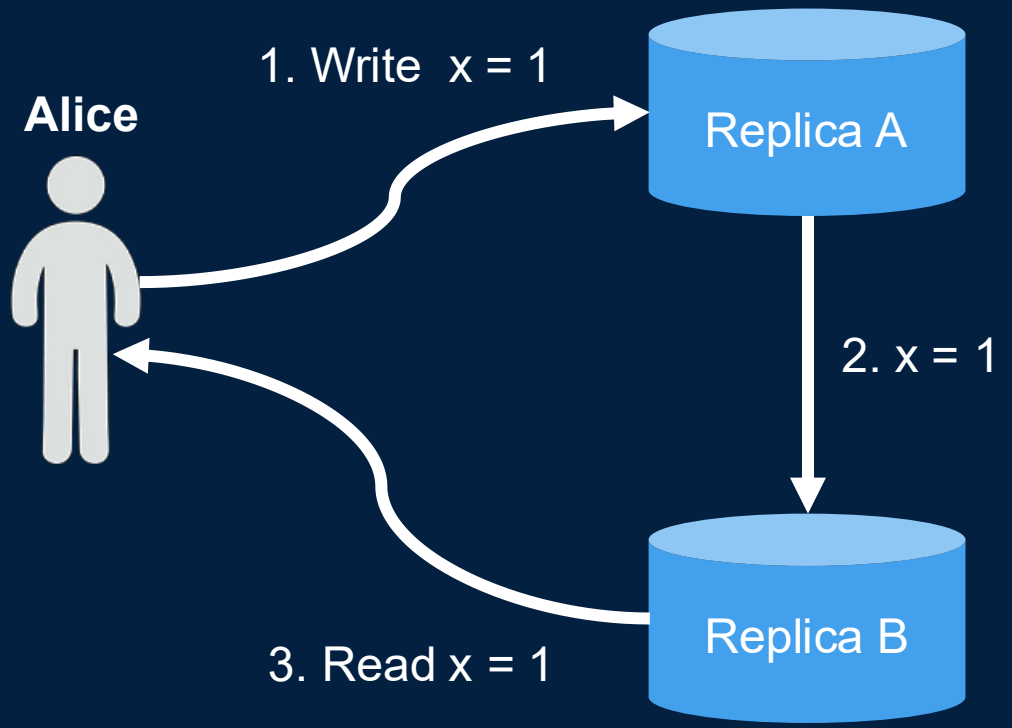
What will the read return?

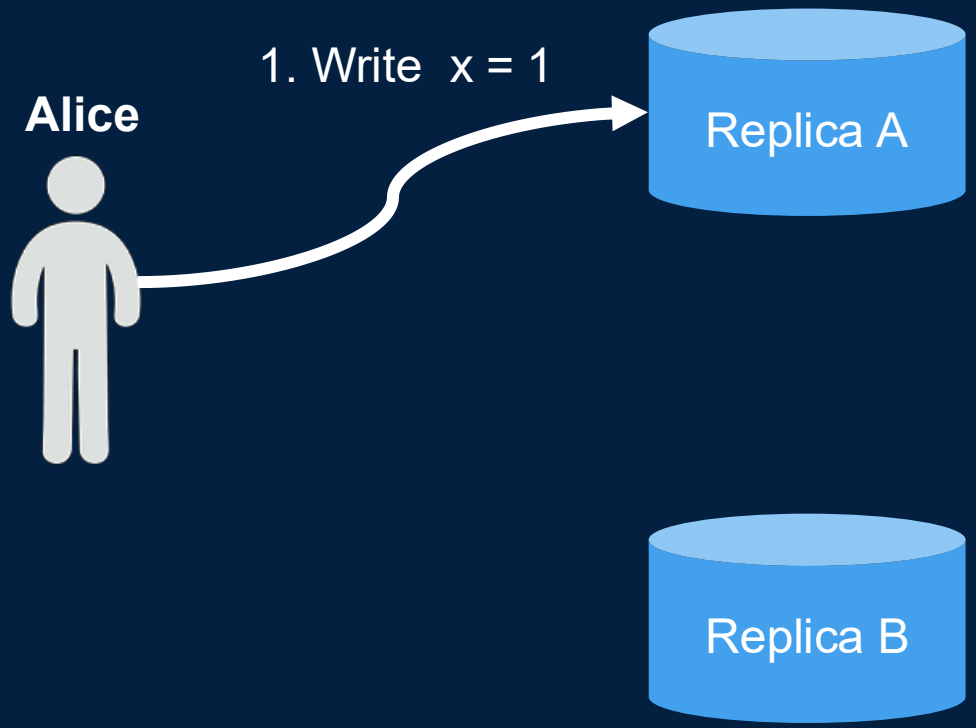


What will the read return?

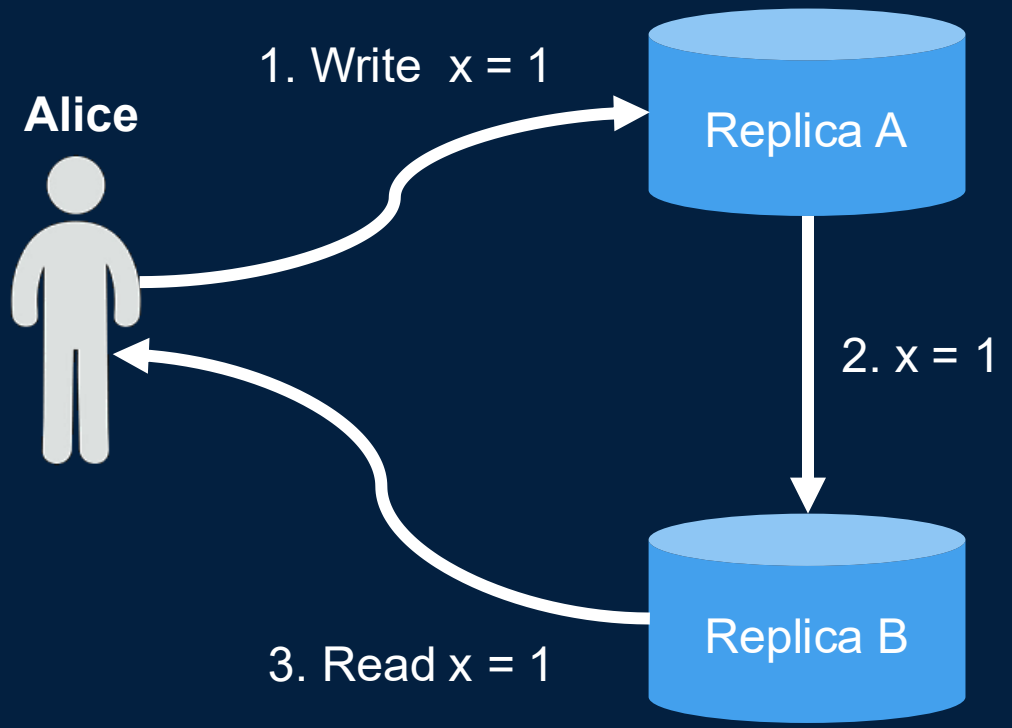


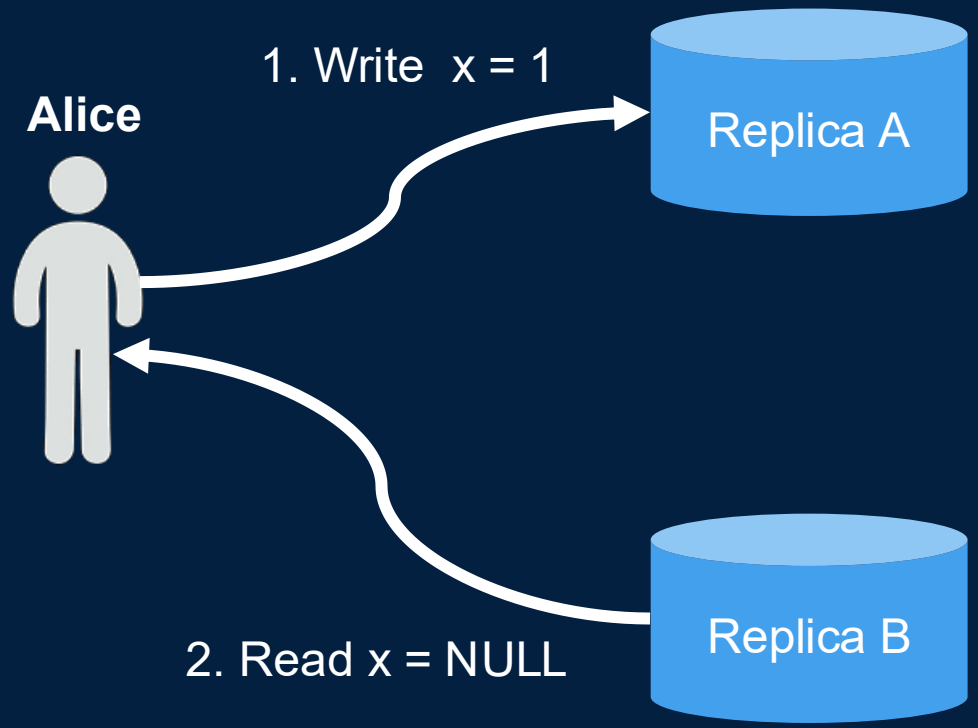
What will the read return?



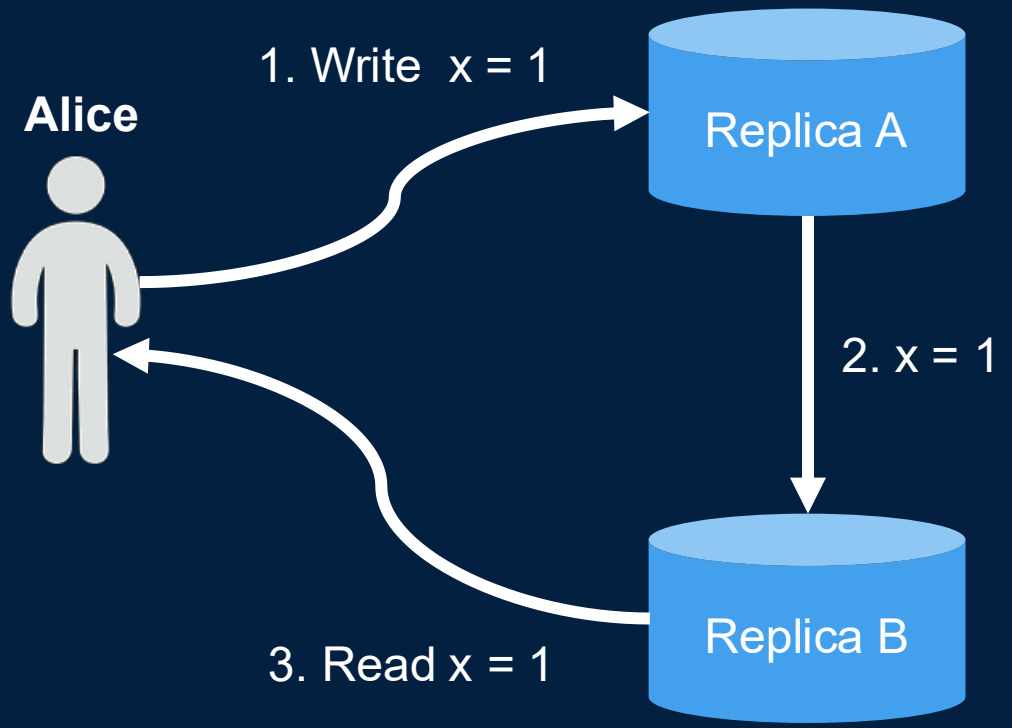


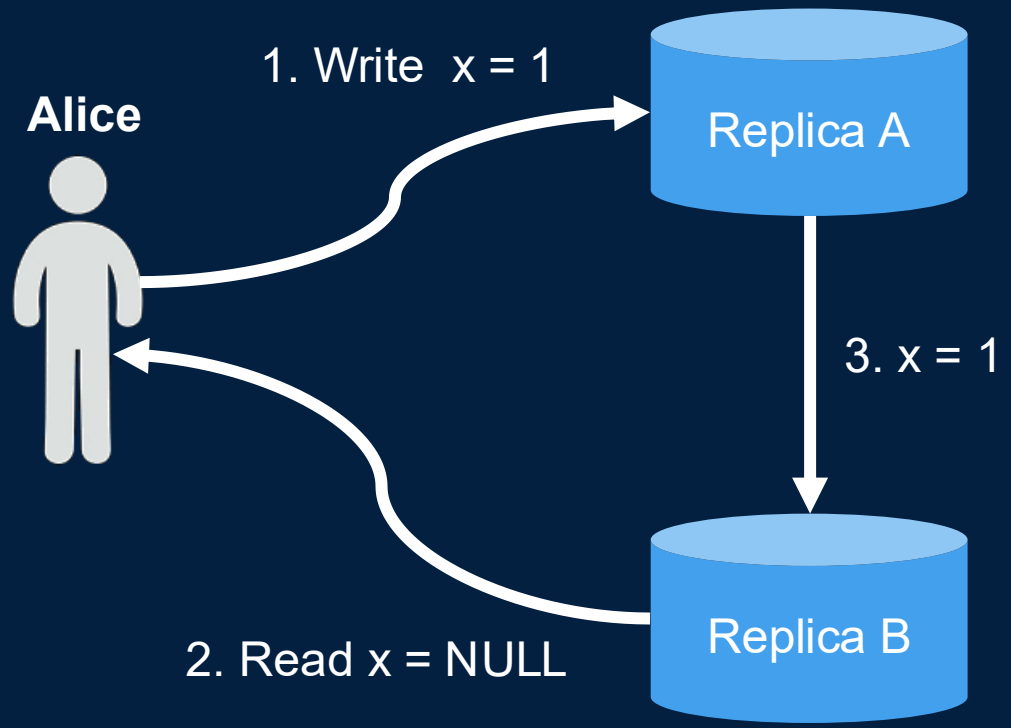
What will the read return?



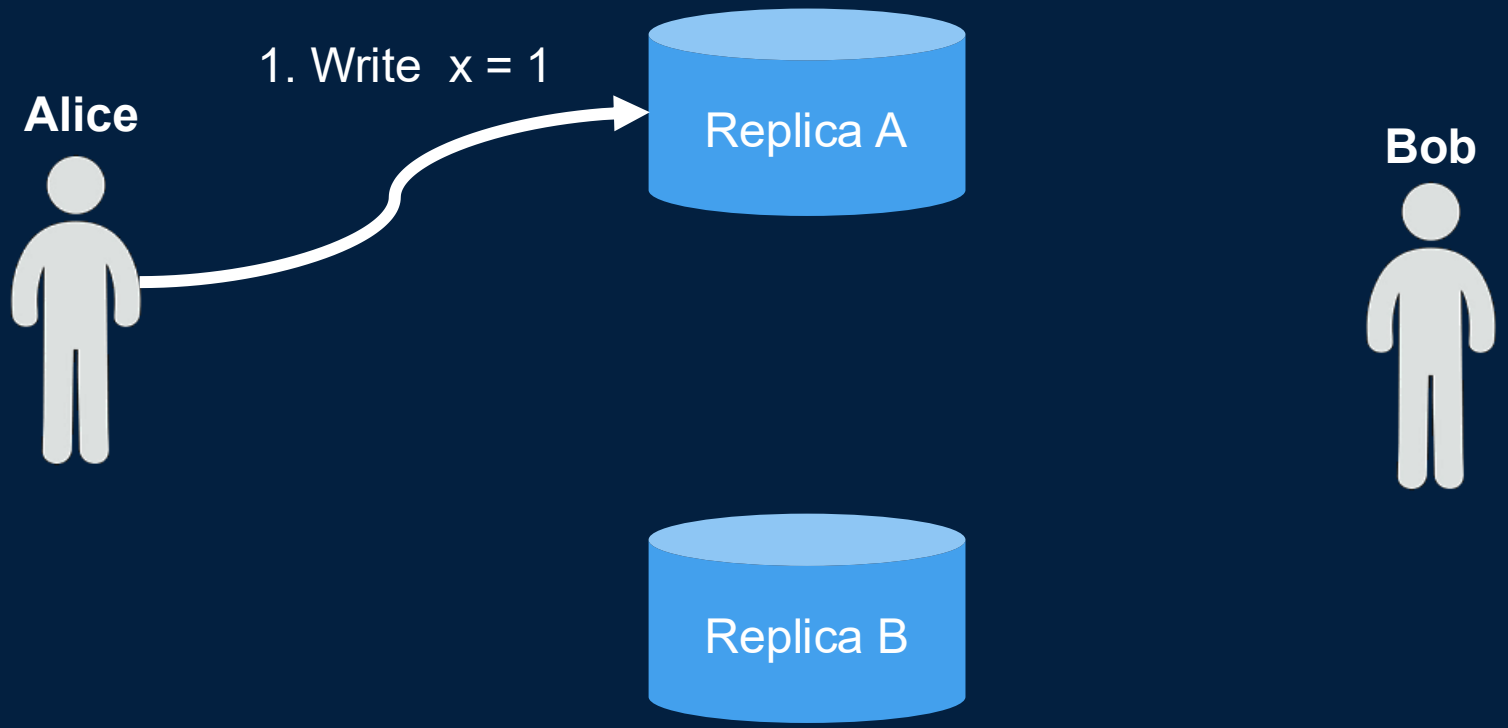


What will the read return?

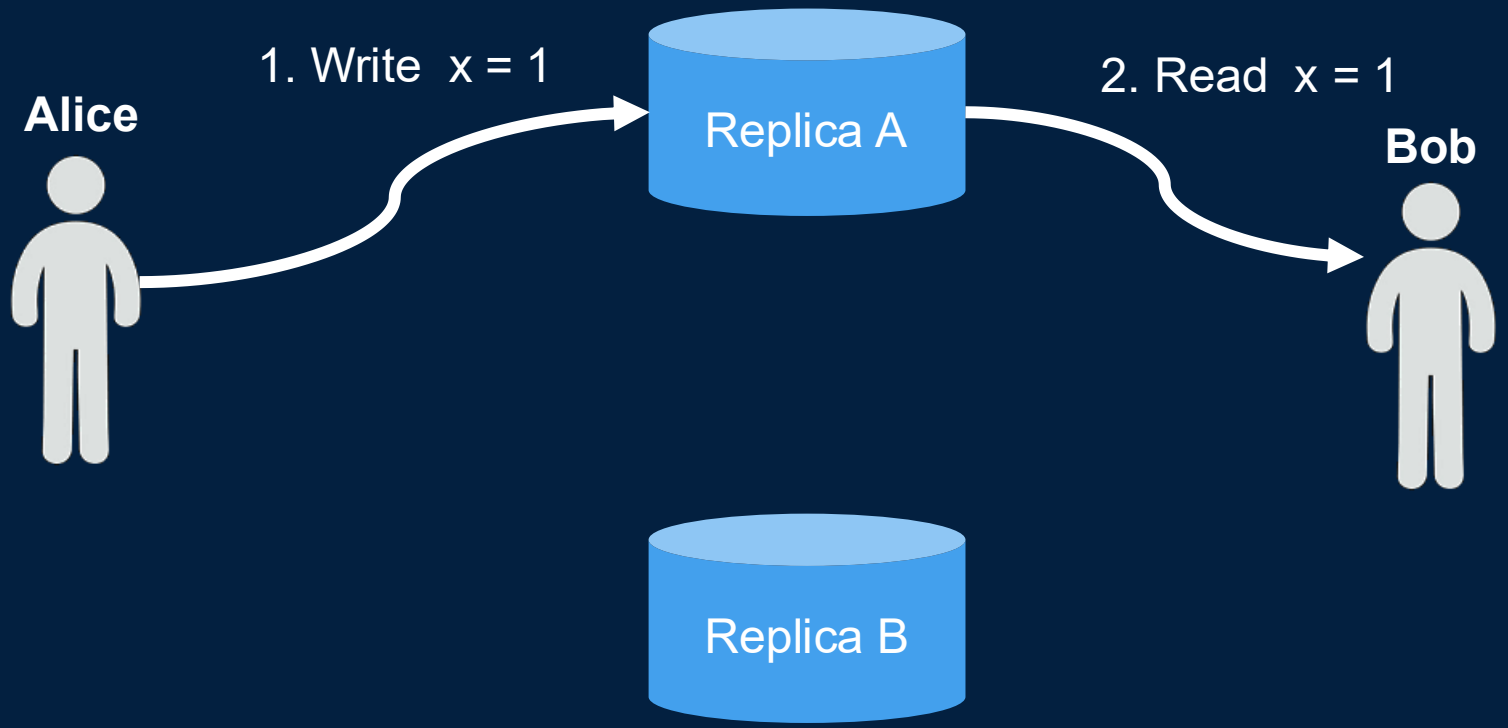




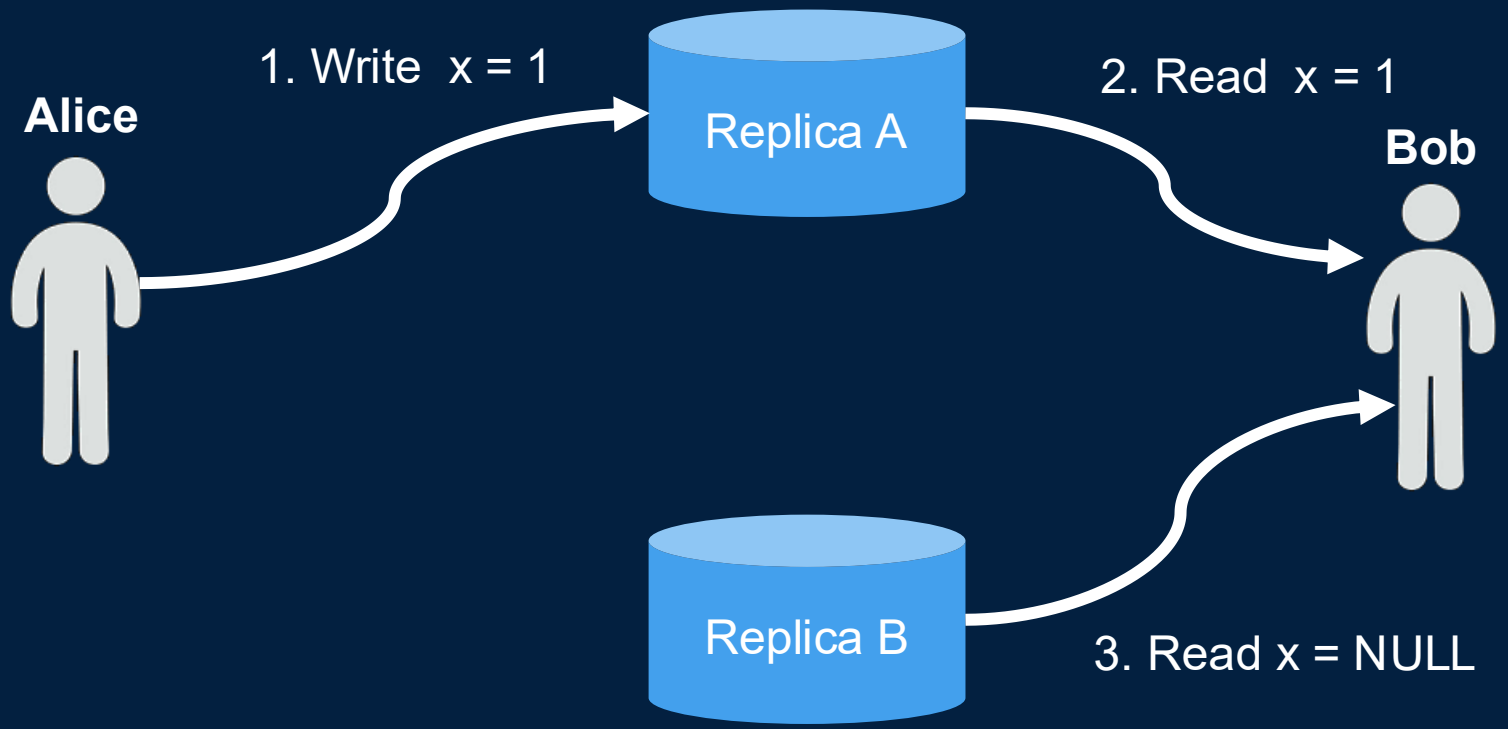
Non-monotonic reads



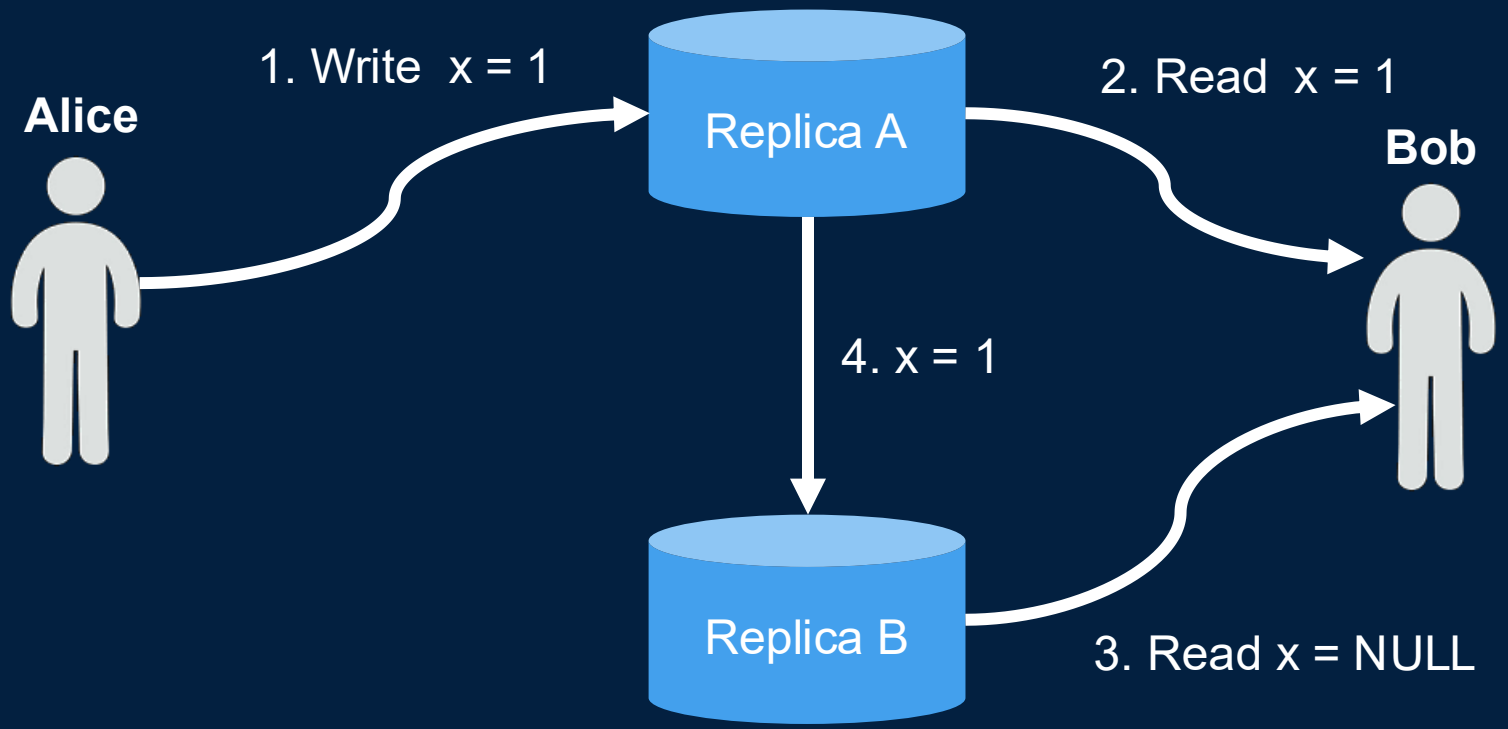
Non-monotonic reads



Non-monotonic reads

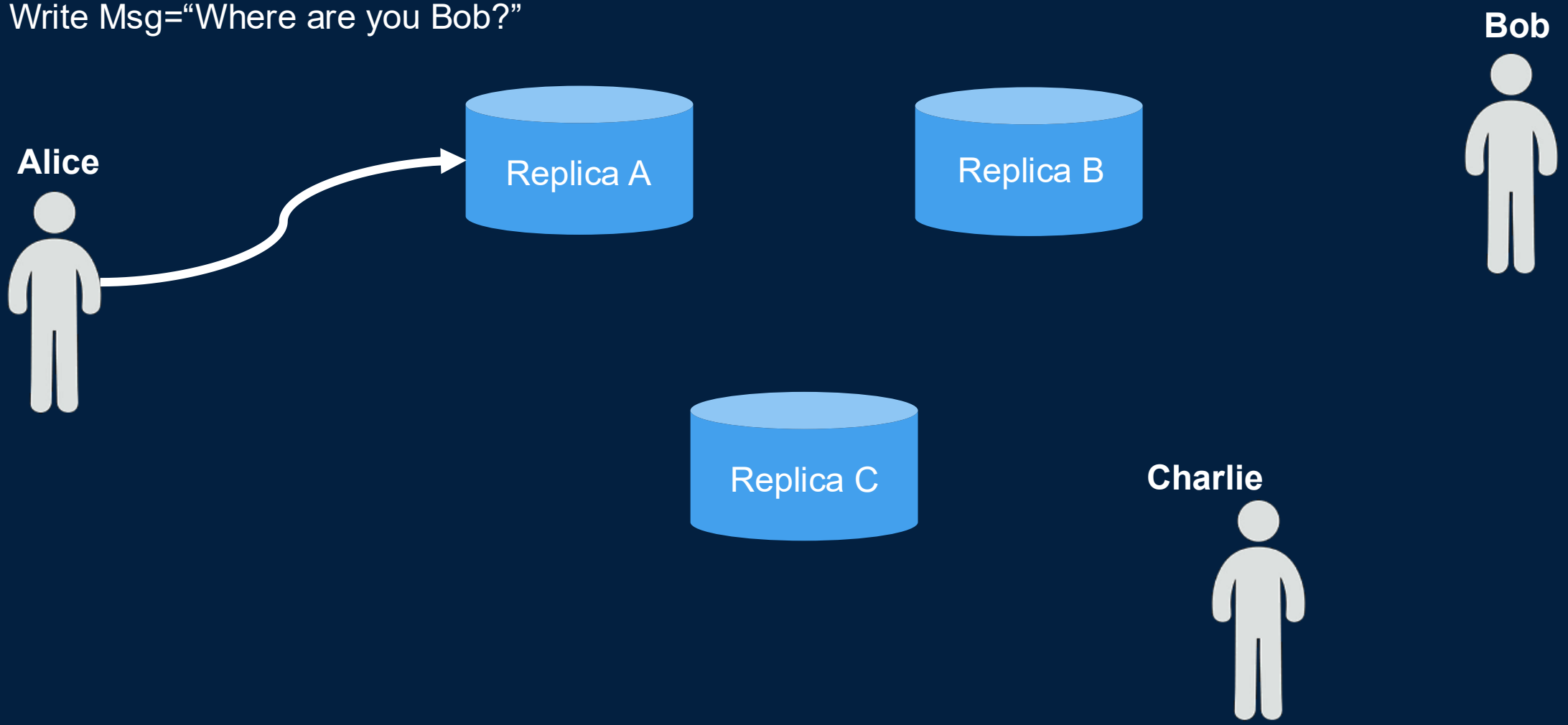


Non-monotonic reads



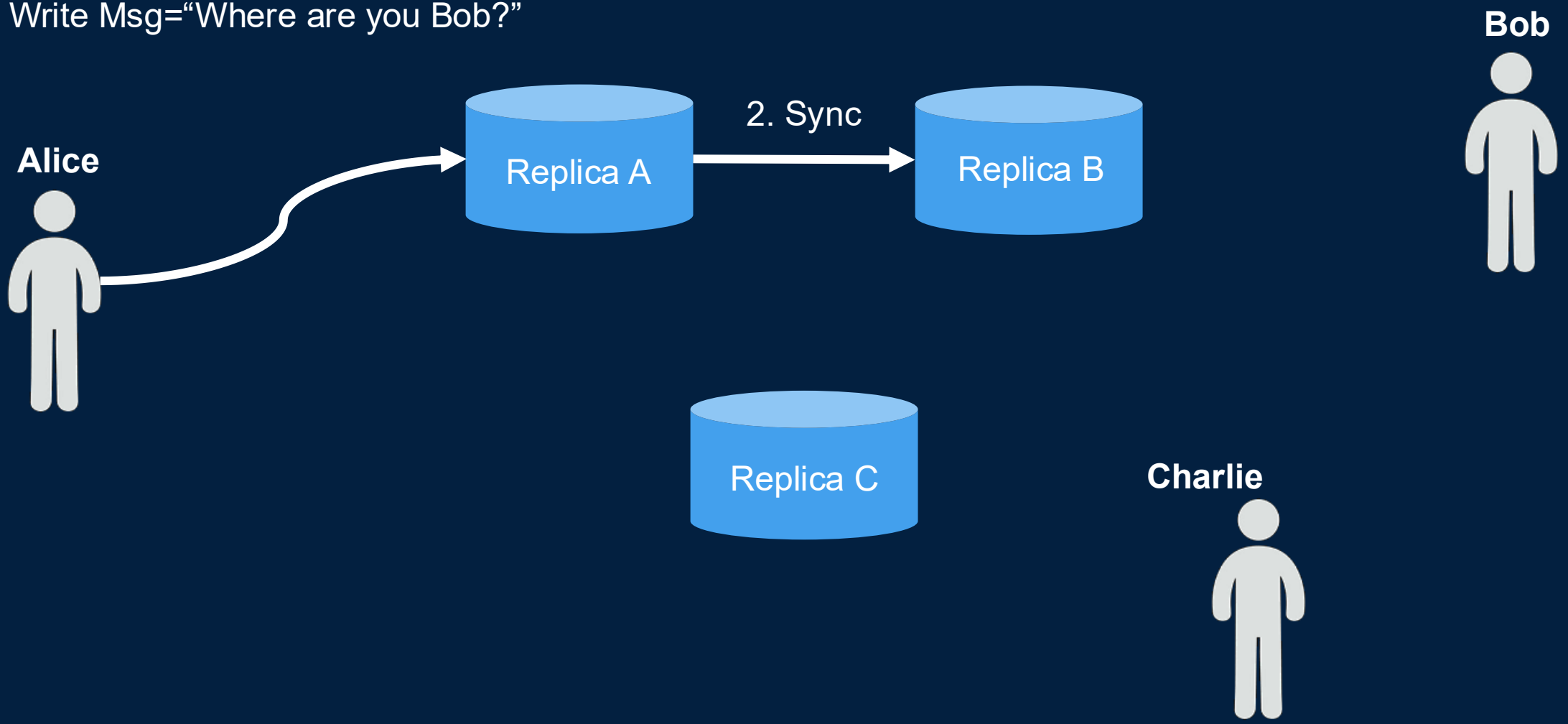
Violation of causality

1. Write Msg="Where are you Bob?"



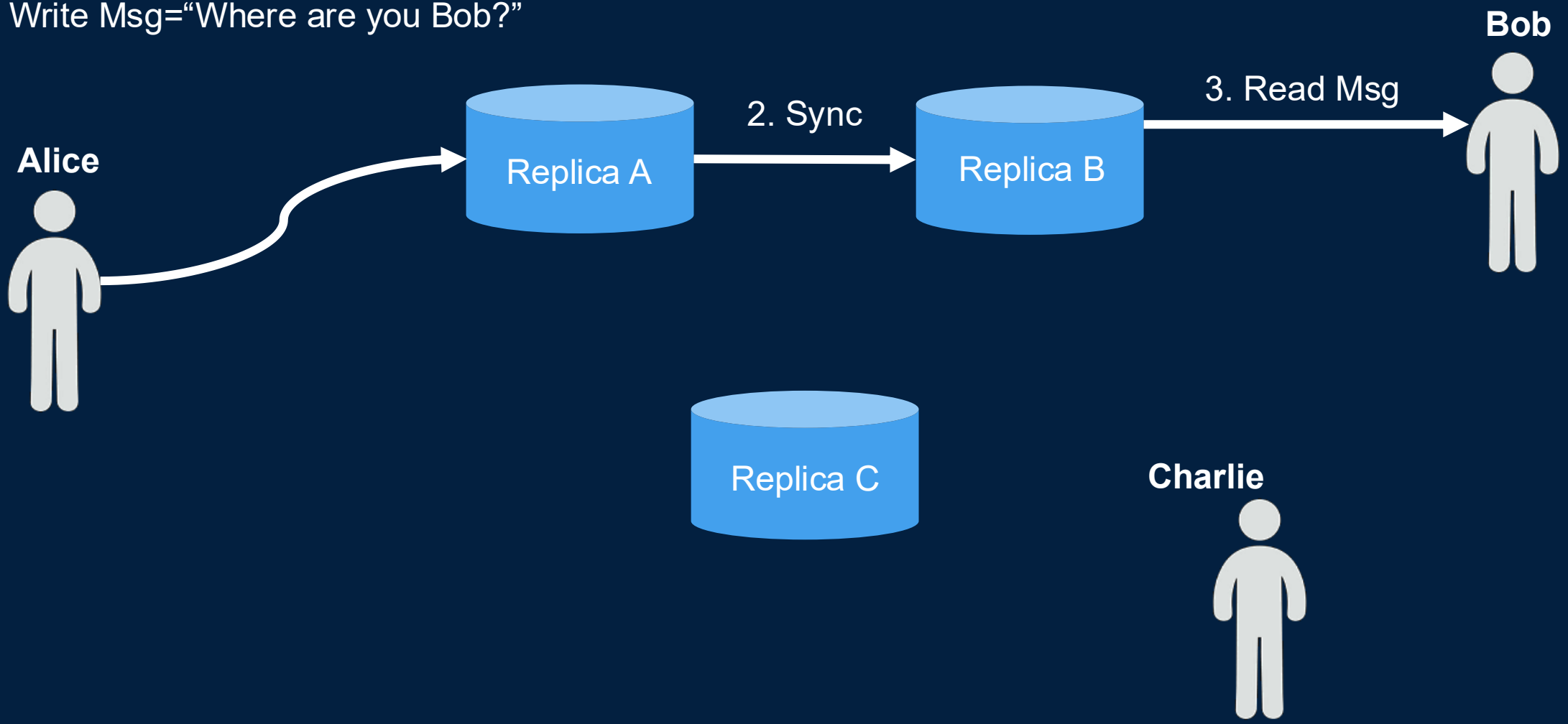
Violation of causality

1. Write Msg="Where are you Bob?"



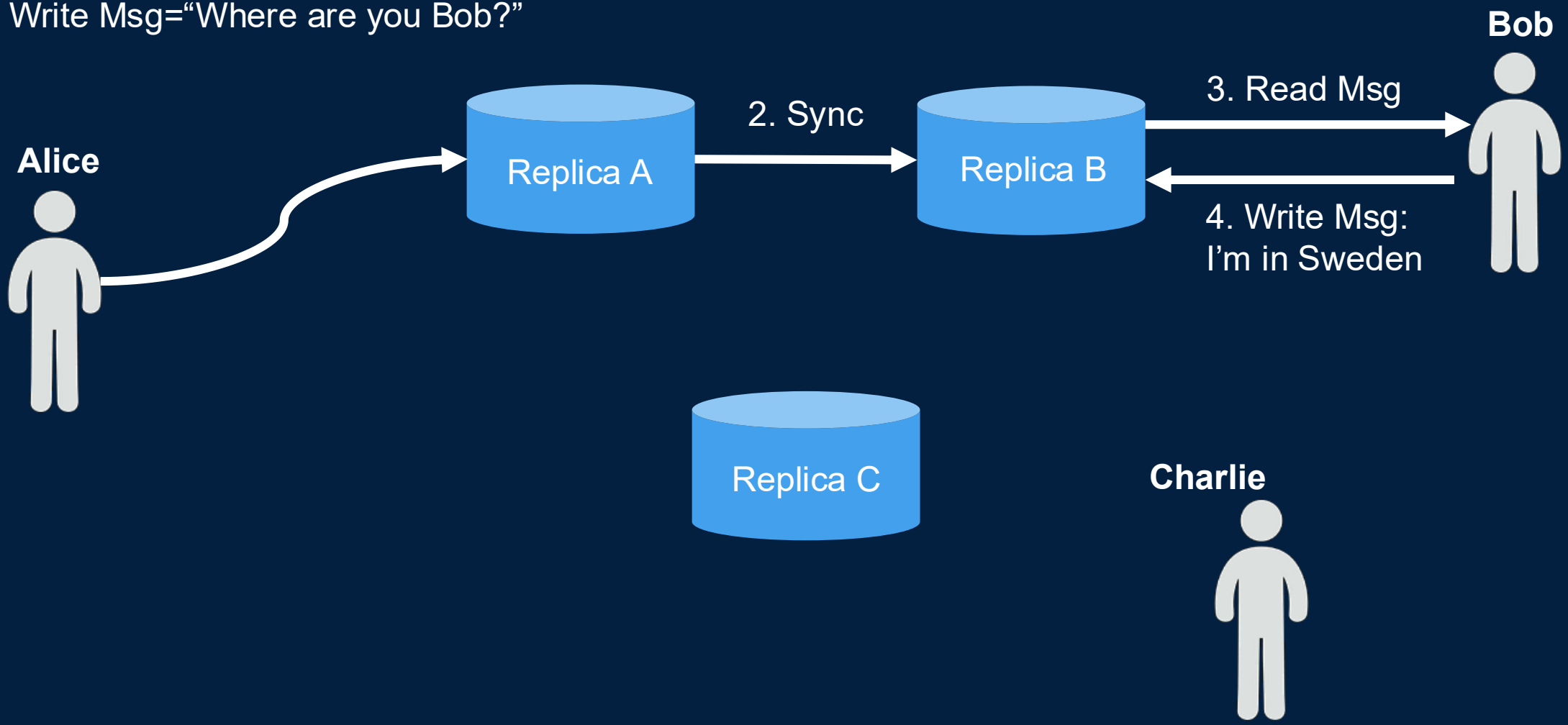
Violation of causality

1. Write Msg="Where are you Bob?"

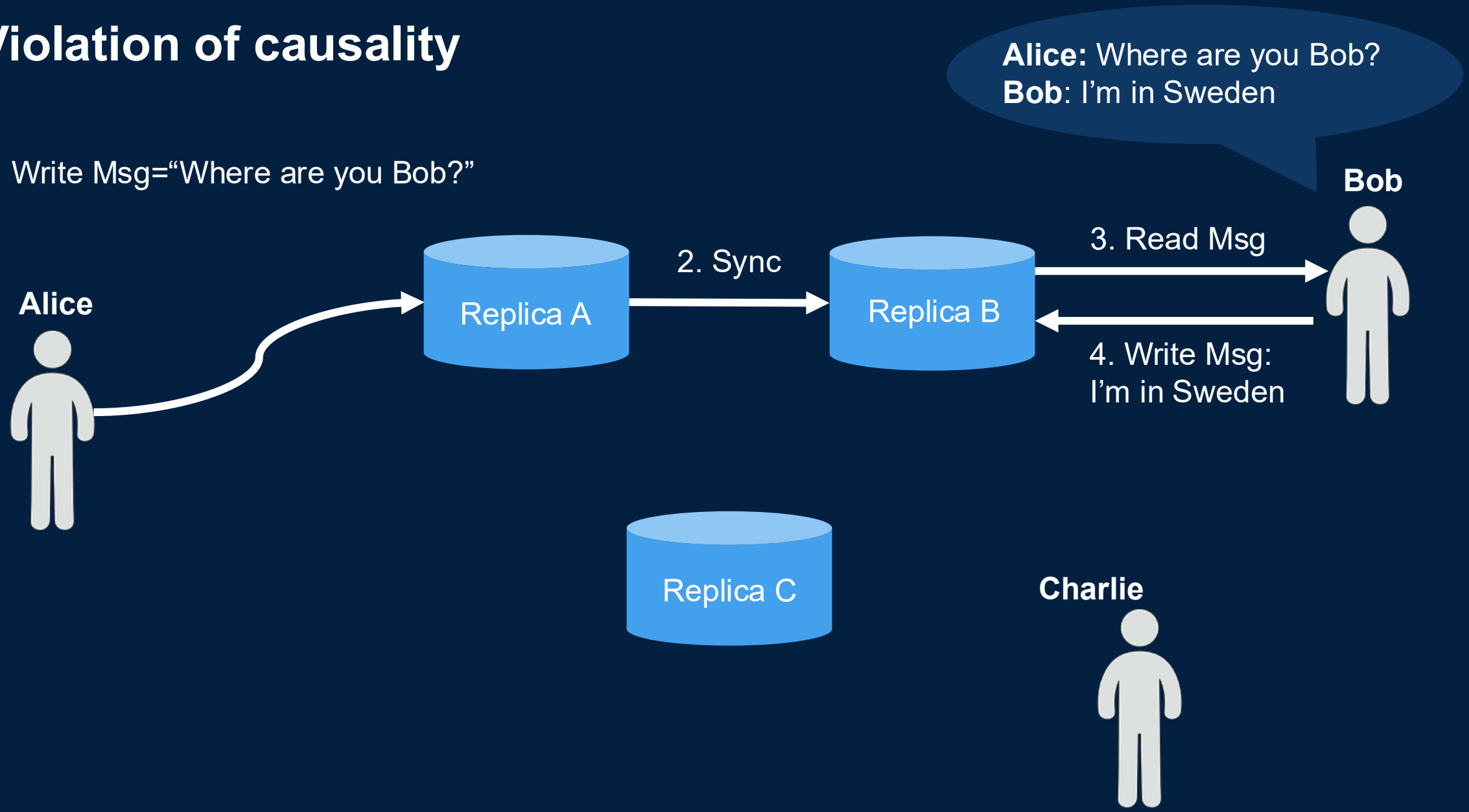


Violation of causality

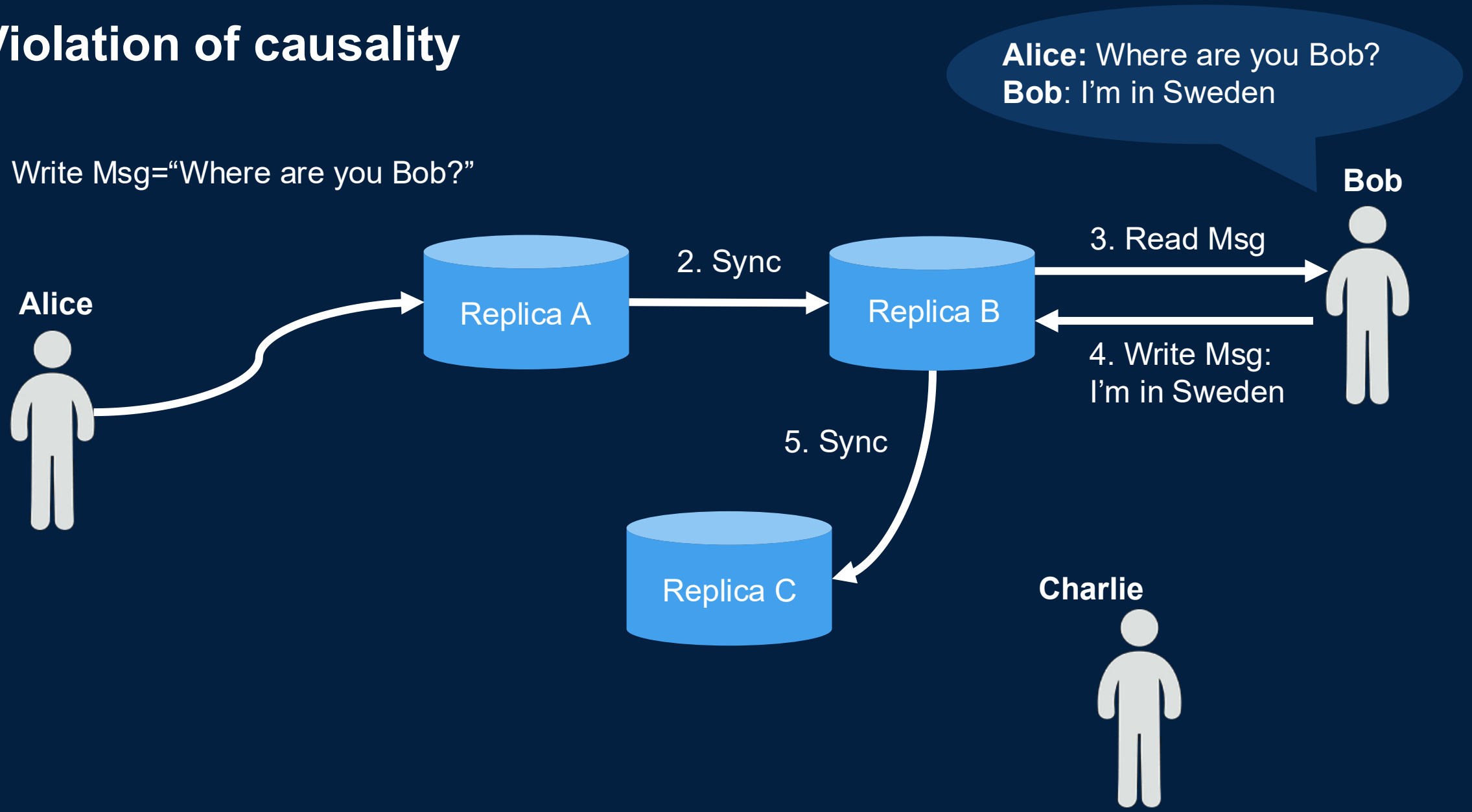
1. Write Msg="Where are you Bob?"



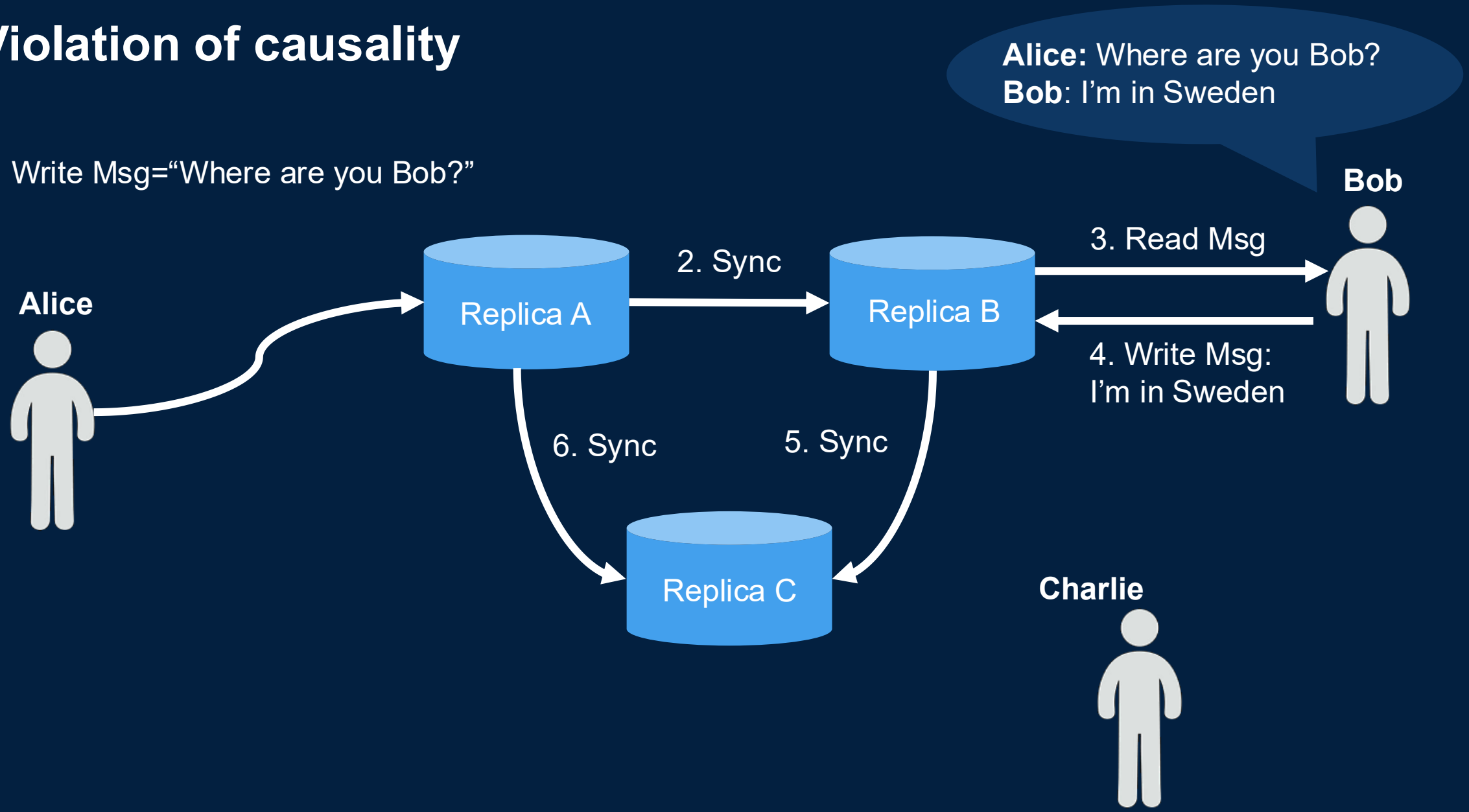
Violation of causality



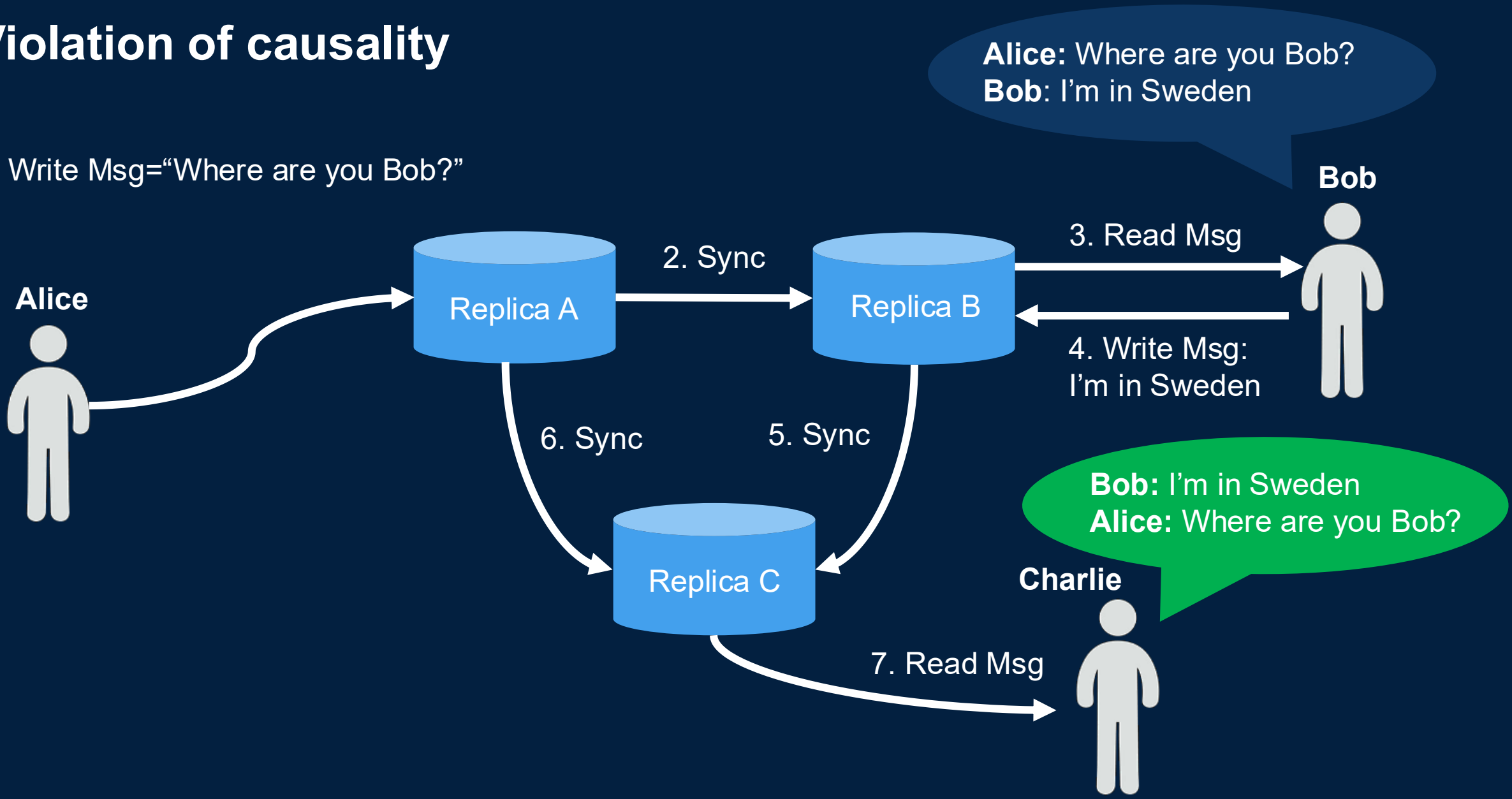
Violation of causality



Violation of causality



Violation of causality

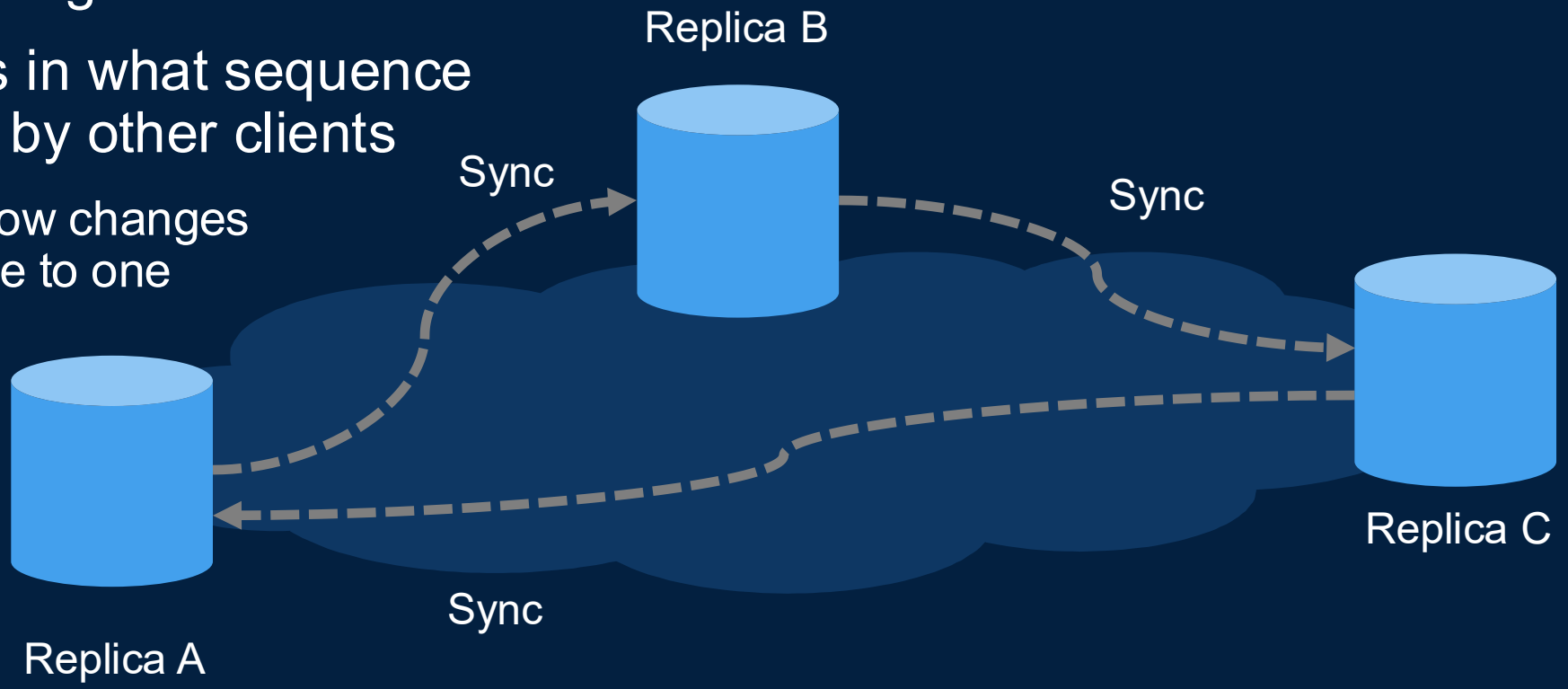


Consistency models

When and in what order updates become visible

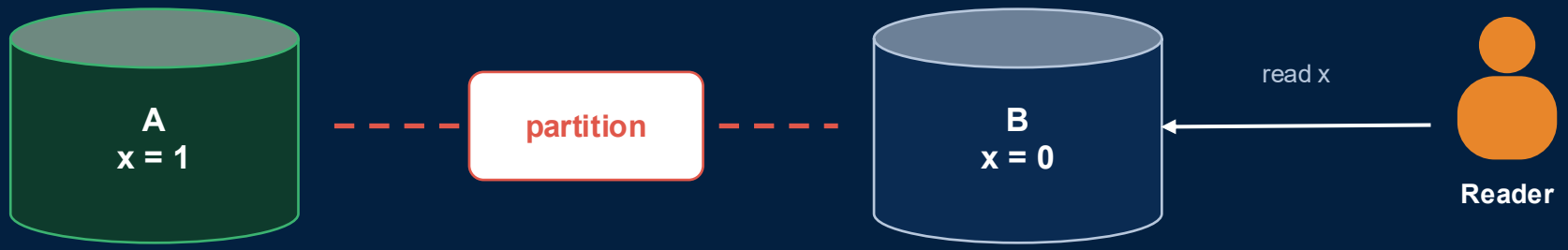
Consistency models

- Due to network latency, partitions, and processing time, users may see updates at different time
- A consistency model dictates what users *should* expect when reading/writing/updating data
- Ordering determines in what sequence operations are seen by other clients
  - It's about when and how changes become visible relative to one another



During a partition, which promise can you keep?

A write x = 1 completed at A. A later read reaches B while A and B cannot talk.



Answer with the old value (available)

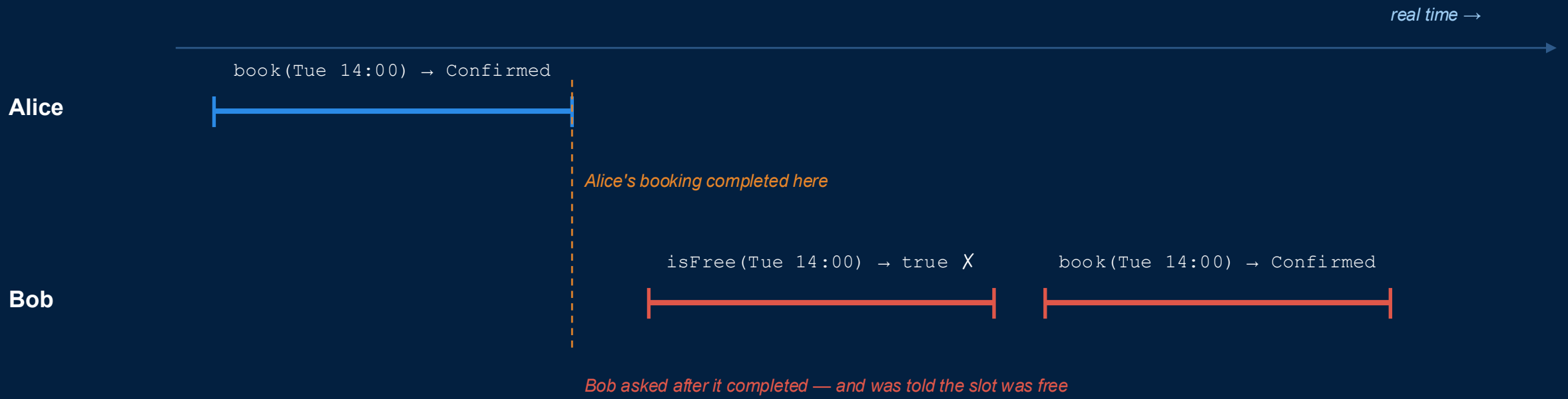
B returns x = 0. A completed write is missed: linearizability is violated. Fine for a note; wrong for a booking.

Wait or refuse (consistent)

The read cannot complete until the partition heals. The operation is unavailable at B — but it never lies.

CAP: "C" means linearizability, "A" applies at non-failing nodes. Decide per operation, not per system.

Linearizability



What went wrong

The replica that answered Bob had not applied Alice's booking yet. Bob acted on an answer that was already false when he asked. Both replicas will converge — on two bookings.

The promise that prevents it

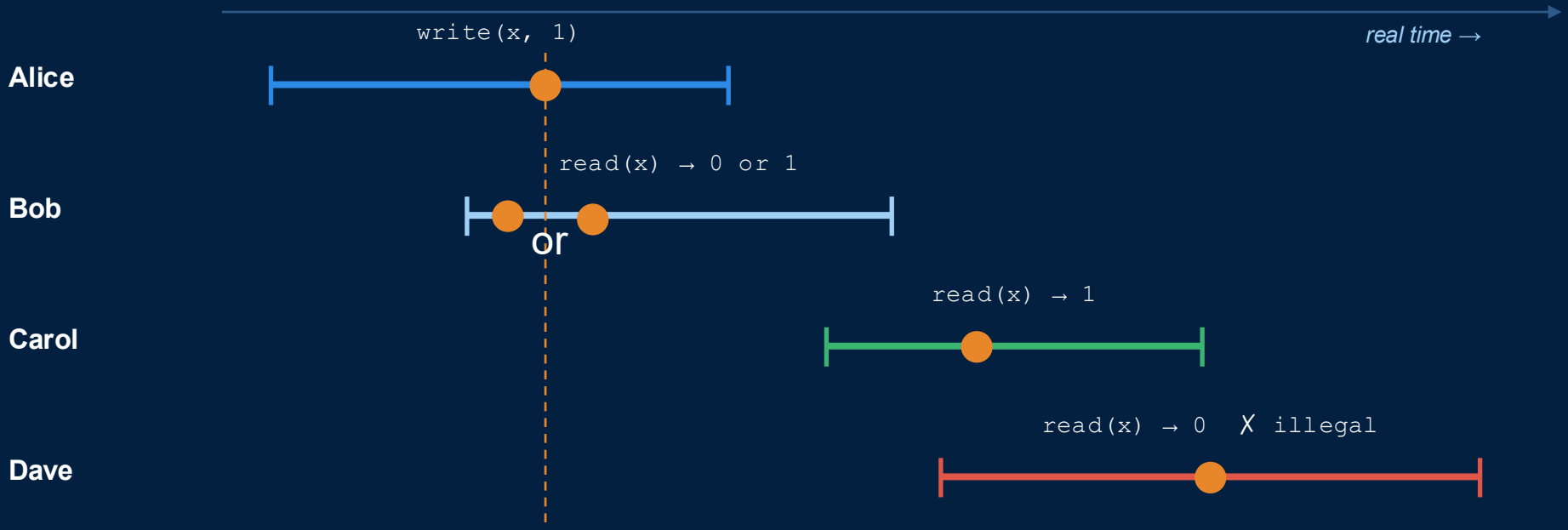
Behave as if there were a single copy: if an operation completed before mine started, mine must see its effect. Needed wherever a decision is taken on the basis of a read: bookings, balances, "is this name taken?", leader election.

Eventual agreement is not enough when someone acts on the answer. That is the case linearizability exists for.

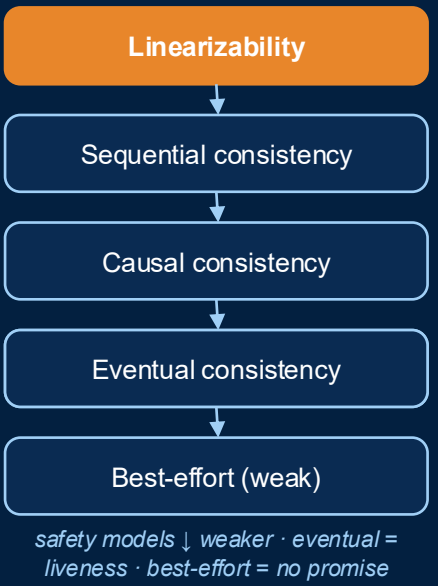
Herlihy & Wing, 1990: https://doi.org/10.1145/78969.78972

Linearizability, step by step

Intuition: the system behaves as if there were one single copy of the data, and every operation happened on it at one instant — in real time.



① every operation is an interval ② it takes effect at one point inside it ③ the points, in time order, must be a legal sequential execution



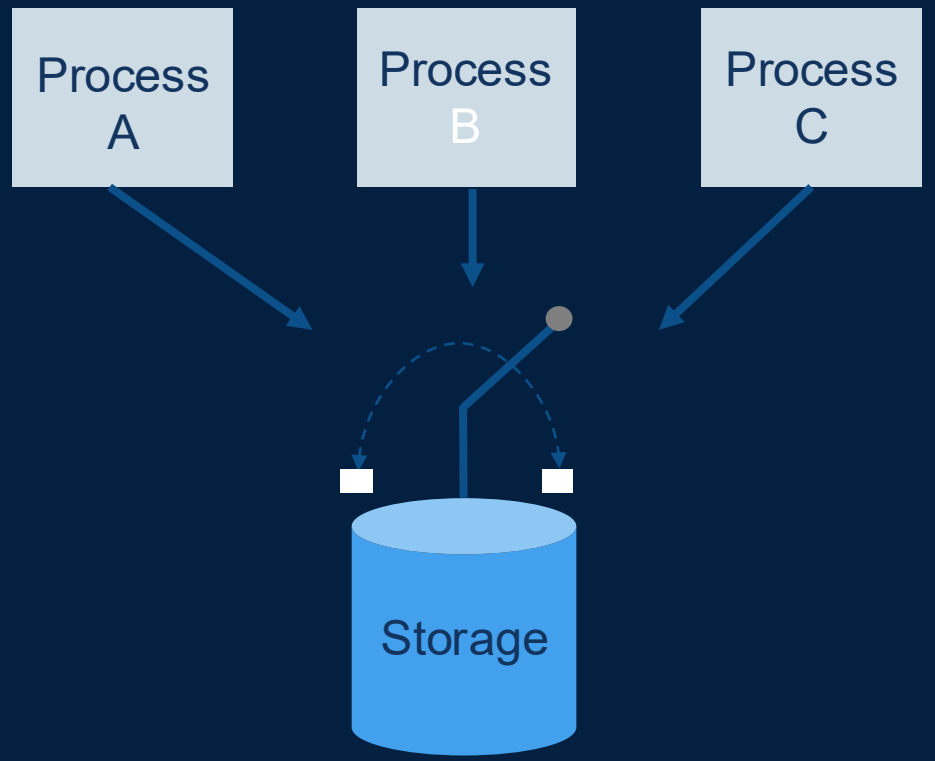
Bob overlaps the write: 0 or 1, both legal.

"Single server" alone is also sequential consistency; real time is what makes it linearizable.

Test: can you place one point per operation so the sequence explains every answer? If not, the execution is not linearizable.

Linearizability

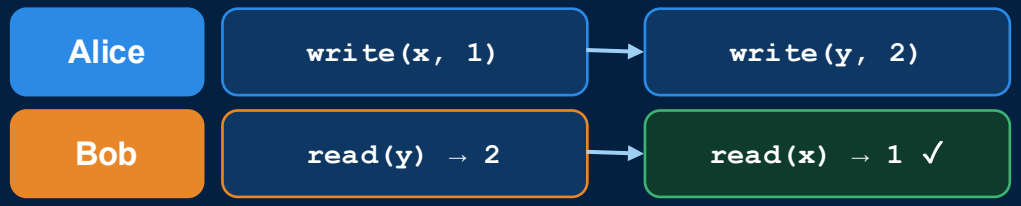
- All operations appear to execute ***atomically*** in ***real-time order***
- The system behaves as if it were not distributed at all, all operations behave as if executed on a single copy of the data, even though data is replicated across multiple nodes
- Total order respecting real-time order
- Completed and acknowledge operations are instantly visible to all clients



Program order: the order in which one client issues its operations

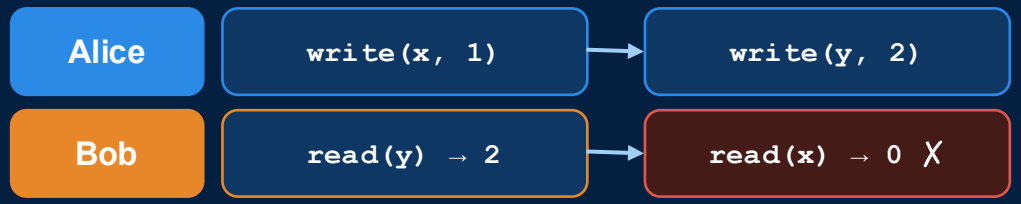
Definition: for one process, the sequence of its own operations in the order it issued them. Alice: write(x, 1) then write(y, 2). It orders nothing across processes.

Example 1 — program order is respected



One sequence explains it: w(x,1) · w(y,2) · r(y)=2 · r(x)=1. Both program orders kept.

Example 2 — program order is violated

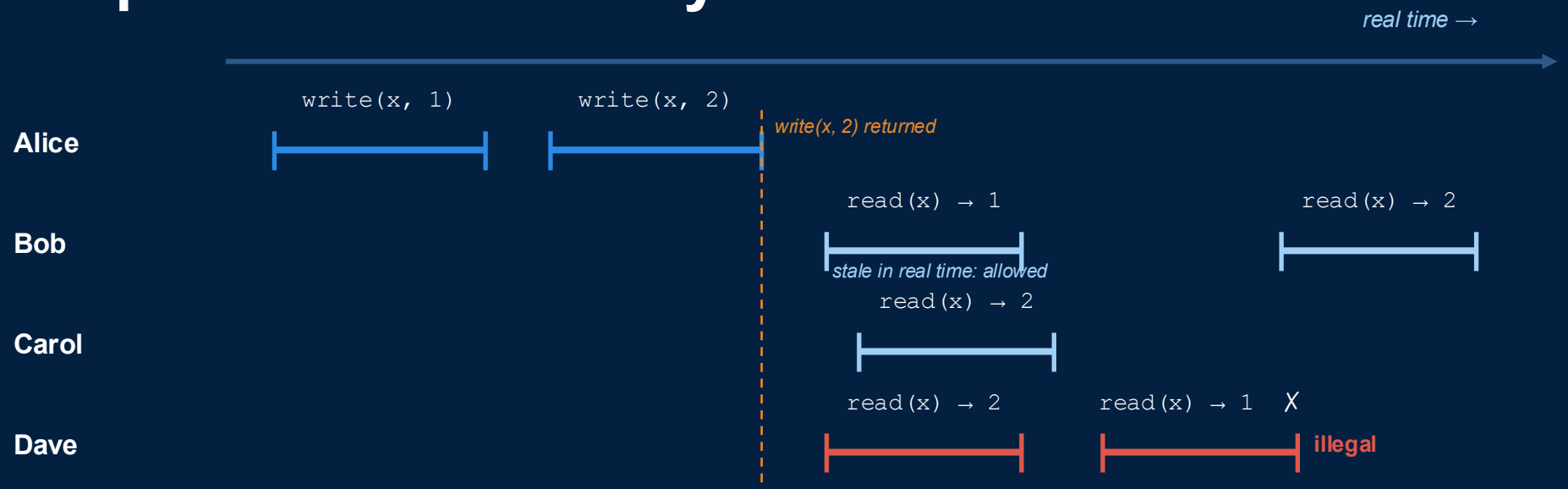


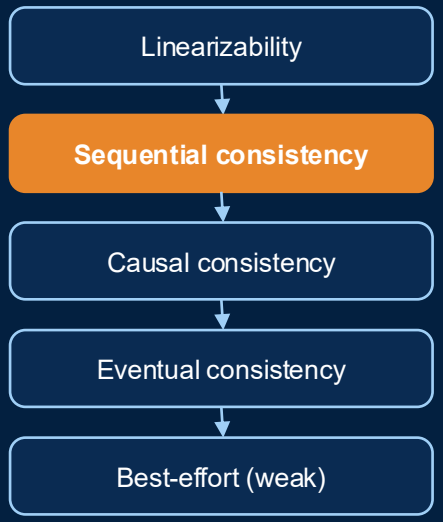
No single sequence explains it: y = 2 was written after x = 1, yet Bob saw the second write and not the first. Legal only under eventual / weak consistency (a replica applied Alice's writes out of order).

Model	Program order	Causal order	Real-time order
Linearizability	yes	yes	yes
Sequential consistency	yes (one global sequence)	yes	no
Causal consistency	yes	yes	no
Eventual consistency	no	no	no

Lamport (1979): https://doi.org/10.1109/TC.1979.1675439

Sequential consistency





safety models ↓ weaker · eventual = liveness · best-effort = no promise

One sequence explains Alice, Bob and Carol: `w(x,1) · Bob r→1 · w(x,2) · Carol r→2 · Bob r→2`. Bob's first read is stale in real time, but real time is not part of the promise.

Dave cannot be placed in any sequence: he saw 2 before 1, but Alice's program order has 1 before 2. Not sequentially consistent — legal only under eventual consistency (a replica applied the writes in the wrong order for a while).

Definition

Some single total order of all operations respects every client's program order — but not real time.

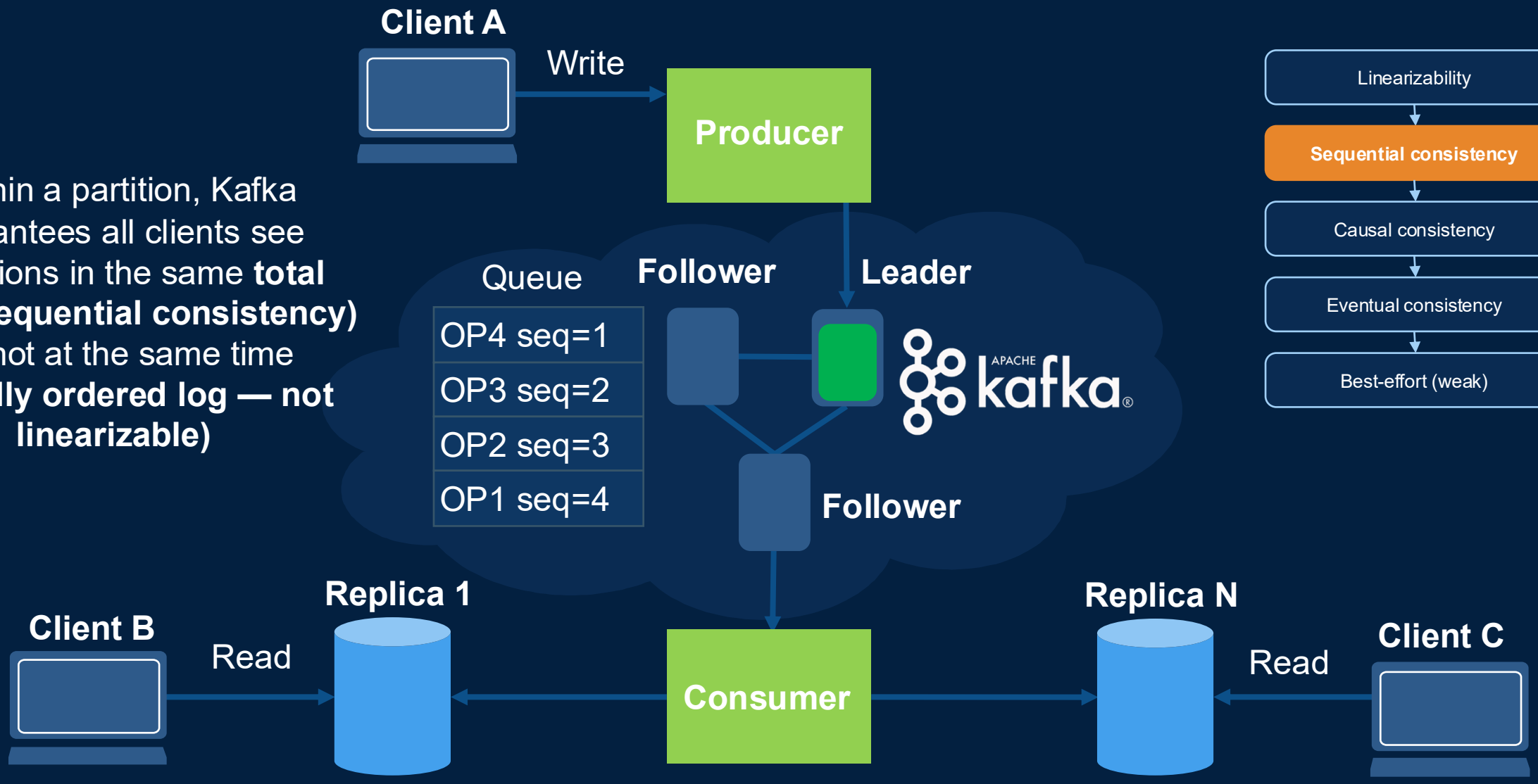
Why it is still expensive

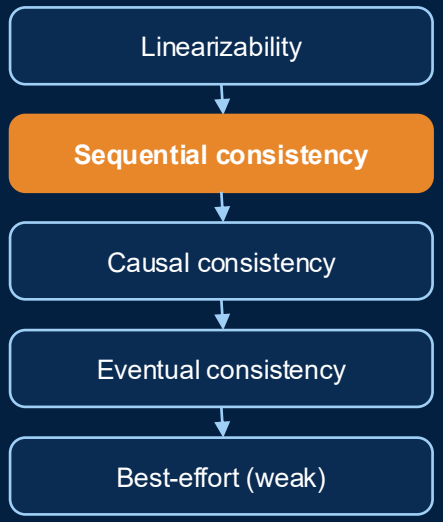
One global order must be agreed (a totally ordered log): unavailable under partition; reads or writes local, never both.

Everyone hears the same story, not at the same time. Different order per client (Dave) breaks it; being late (Bob) does not.

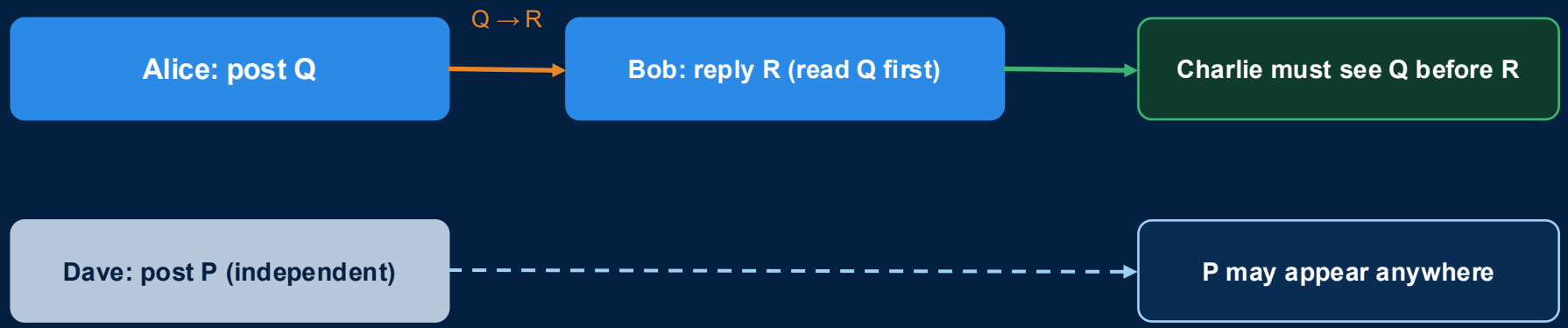
Sequential consistency in practice: a replicated log (Kafka)

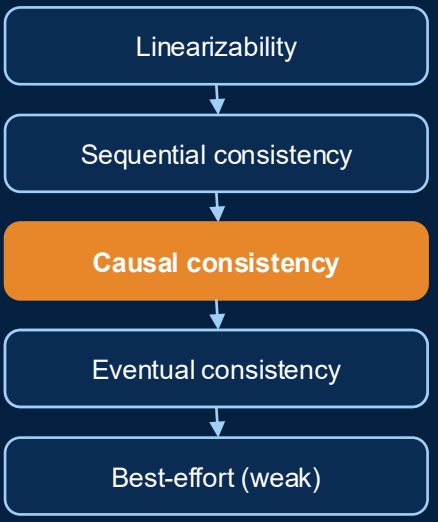
Within a partition, Kafka guarantees all clients see operations in the same **total order (sequential consistency)** but not at the same time **(a totally ordered log — not linearizable)**





Causal consistency





safety models ↓ weaker · eventual = liveness · best-effort = no promise

Definition

Writes ordered by happens-before are seen in that order by all; concurrent writes may be seen in different orders. Causal+ adds convergent handling of the concurrent ones.

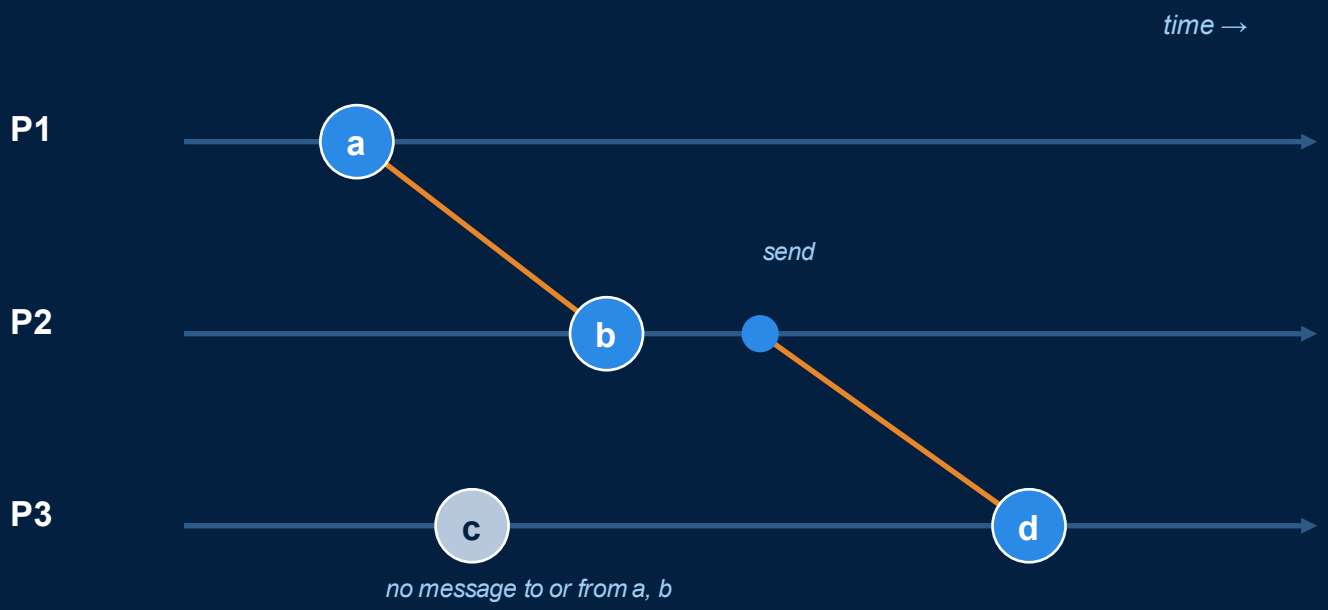
Why it matters

The strongest model that stays available under partition with local reads and writes. Implementation: dependency tracking (vector clocks) and buffering until dependencies are applied.

Causality constrains visibility; it does not choose a winner among concurrent writes. That choice is Part 2.

Lamport 1978: https://doi.org/10.1145/359545.359563

Recap: total order versus causal order



Happens-before: a → b → d, c → d. c ∥ a, c ∥ b

Total orders consistent with it: a b c d · a c b d · c a b d

a total order picks one of them; a causal order keeps all three.

Total order

Every pair of events is comparable.

One global sequence everyone agrees on.

This is what a replicated log gives: sequential consistency. It costs coordination.

Causal order

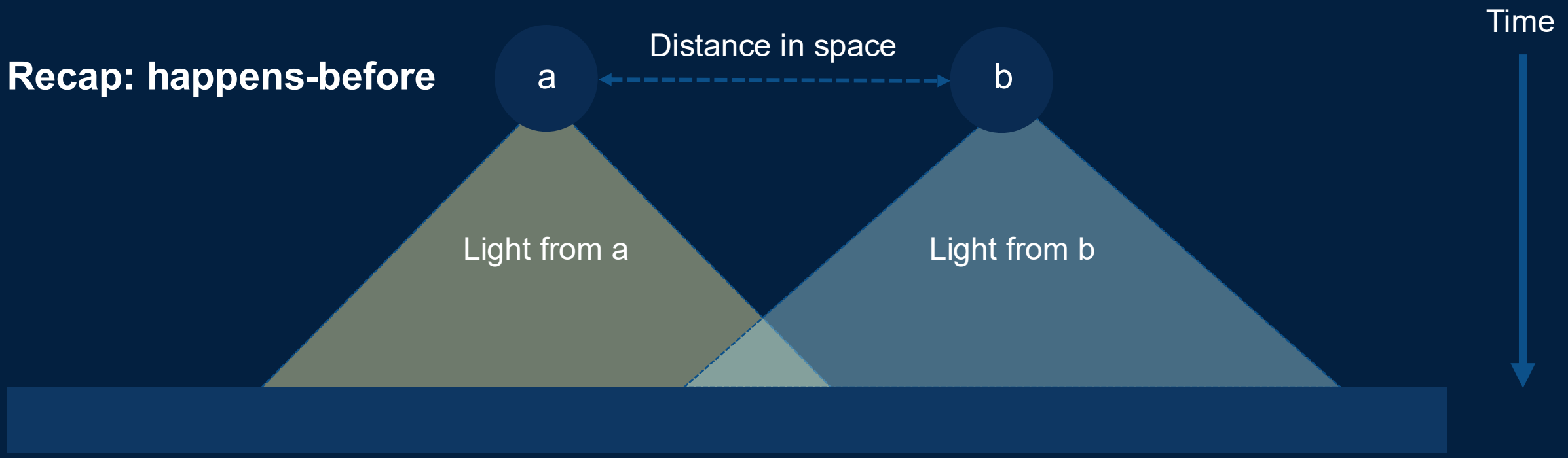
Events are ordered only if one happens-before the other (Lamport 1978).

Concurrent events may be seen in different orders by different replicas.

This is what causal consistency promises; it stays available under partition. Vector clocks decide the case.

A total order settles every pair; a causal order settles only the pairs that could have influenced each other — and leaves the rest to reconciliation.

Recap: happens-before



Happened-before (denoted $\rightarrow$) encodes potential causuality
Relation between two events so that one event happened before the other event

When $a \rightarrow b$, then event **a might** have caused event **b**
When $a \parallel b$, then event **a cannot** have caused event **b**

Transitivity

$\forall a, b, c \quad a \rightarrow b$ and $b \rightarrow c \Rightarrow a \rightarrow c$

Asymmetry

$\forall a, b \quad a \rightarrow b \Rightarrow b \nrightarrow a$

Irreflexively

$\forall a, a \nrightarrow a$

Recap: logical clocks (Lamport timestamps)

- Mechanism to capture causal relationships in distributed systems
  - First Logical Clock was developed by Leslie Lamport in 1978 (Turing Award 2013)
- Lamport Timestamp
  - Assigns a numerical timestamp (L) to each event to preserve causal relationships
  - If $a \rightarrow b \Rightarrow L(a) < L(b)$ $L(a) < L(b) \not\Rightarrow a \rightarrow b$

Since a and b could happen concurrently

Local Event

```
def on_local_event():
   local_clock += 1
   return local_clock
```

Sending

```
def send_message(msg):
    local_clock += 1
    msg.timestamp = local_clock send(msg)
```

Receiving

```
def receive_message(msg):
   local_clock = max(local_clock, msg.timestamp)+1
   process(msg)
```

Fidge, 1988: https://zoo.cs.yale.edu/classes/cs426/2012/lab/bib/fidge88timestamps.pdf
Mattern, 1989: https://www.vs.inf.ethz.ch/publ/papers/VirtTimeGlobStates.pdf

Recap: vector clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

$$VC_i[i] = VC_i[i] + 1$$
$$msg.VC = VC_i$$

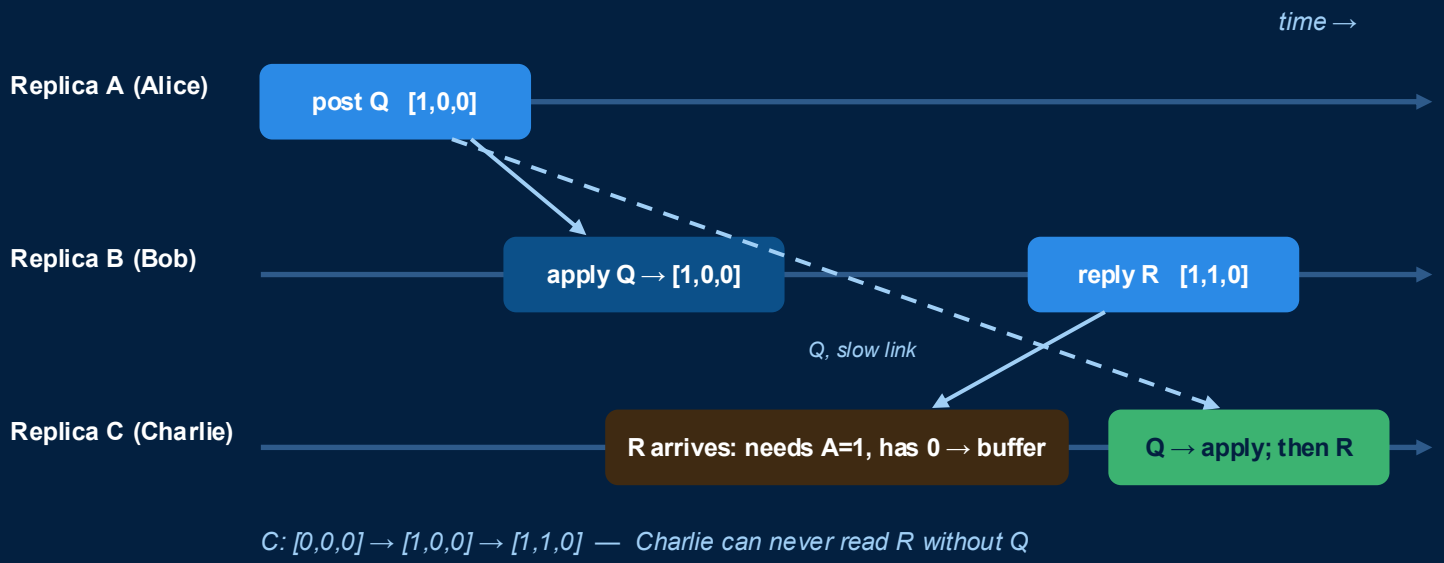
Receiving

$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$
$$VC_i[i] = VC_i[i] + 1$$

Implementing causal consistency: a vector clock as a dependency check

Tag every write with the writer's vector clock — it names everything the write may depend on.

Apply a write from replica j with tag V only when $V[j] = local[j] + 1$ (nothing from j is missing) and $V[k] \le local[k]$ for all $k \neq j$ (every dependency already applied). Otherwise buffer it.



C: [0,0,0] → [1,0,0] → [1,1,0] — Charlie can never read R without Q

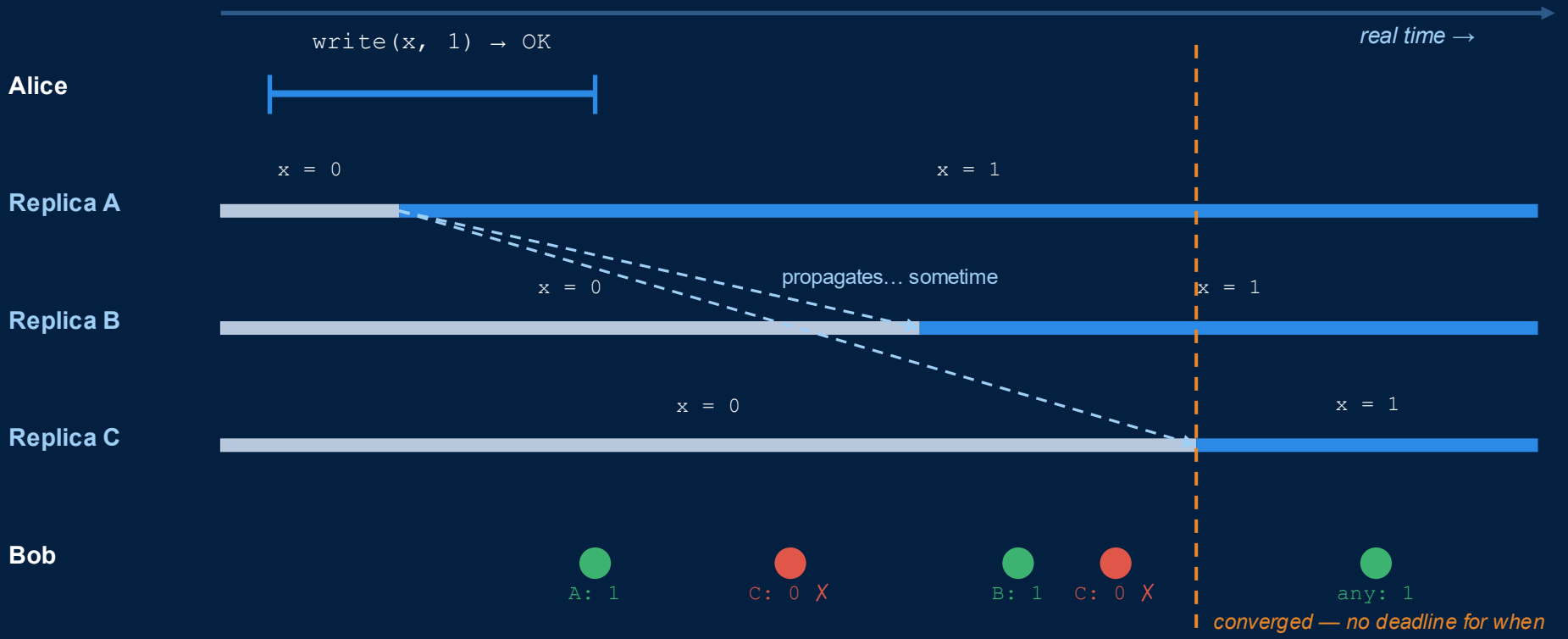
What it buys

No coordination on the write path: local reads and writes, available under partition. The strongest model with that property.

A vector clock turns "happens-before" into a delivery rule: hold a write until everything it depends on has been applied.

Birman, Schiper & Stephenson (1991): https://doi.org/10.1145/128738.128742 · COPS (2011): https://doi.org/10.1145/2043556.2043593

Eventual consistency on a timeline: anything goes until the replicas converge





Before convergence: a read may return any value the replica has — old after new, new after old.

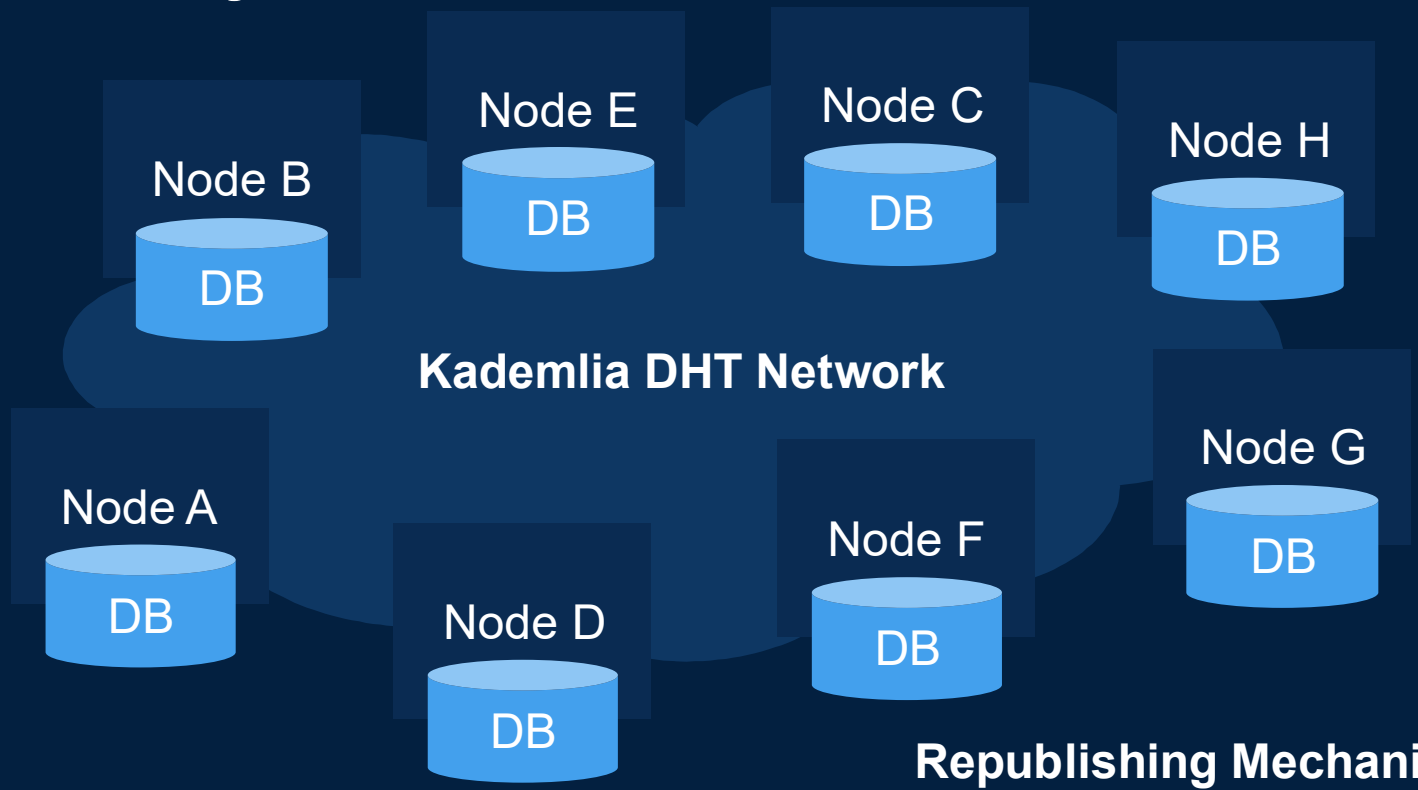
After convergence: all replicas hold the same value — if updates stop and messages get through.

Vogels (2009): https://doi.org/10.1145/1435417.1435432 · Bailis & Ghodsi (2013): https://doi.org/10.1145/2447976.2447992

Eventual consistency in the lab: Kademlia

Store operation:

1. Find k closest nodes to key
2. Store on those k nodes
3. No waiting for confirmation from all
4. Other nodes learn gradually through lookups



Republishing Mechanism:

1. Original publisher republishes every hour
2. Each node checks its stored data hourly
3. If node has data, find k closest nodes to key
4. Send to any of k closest that don't have it
5. Maintain k copies despite node failures
6. Data expires after 24 hours without refresh

Eventual consistency in the lab: a worked timeline

Event 1: Order Received (7:00 PM):
DHT.Store("order:12345", { "status": "Order received", "time": "7:00 PM" })
→ **Stored on nodes [A, B, C]**

Event 2: Pizza in Oven (7:15 PM):
DHT.Store("order:12345", { "status": "Cooking", "time": "7:15 PM" })
→ **Stored on nodes [D, E, F]** ***Different nodes!***

Event 3: On the way (7:30 PM):
DHT.Store("order:12345", { "status": "On the way", "time": "7:30 PM" })
→ **Stored on nodes [A, G, H]** ***Node A gets update***

Node A: "On the way!" (Latest)
Node B: "Order received" (Never got updates)
Node C: "Order received" (Never got updates)
Node D: "Cooking" (Missed delivery update)
Node E: "Cooking" (Missed delivery update)

NOTE: Republishing ensures eventual consistency, e.g., after 1h all nodes will converge to Event 3

Where eventual consistency is the right choice: Kademlia sweet spots

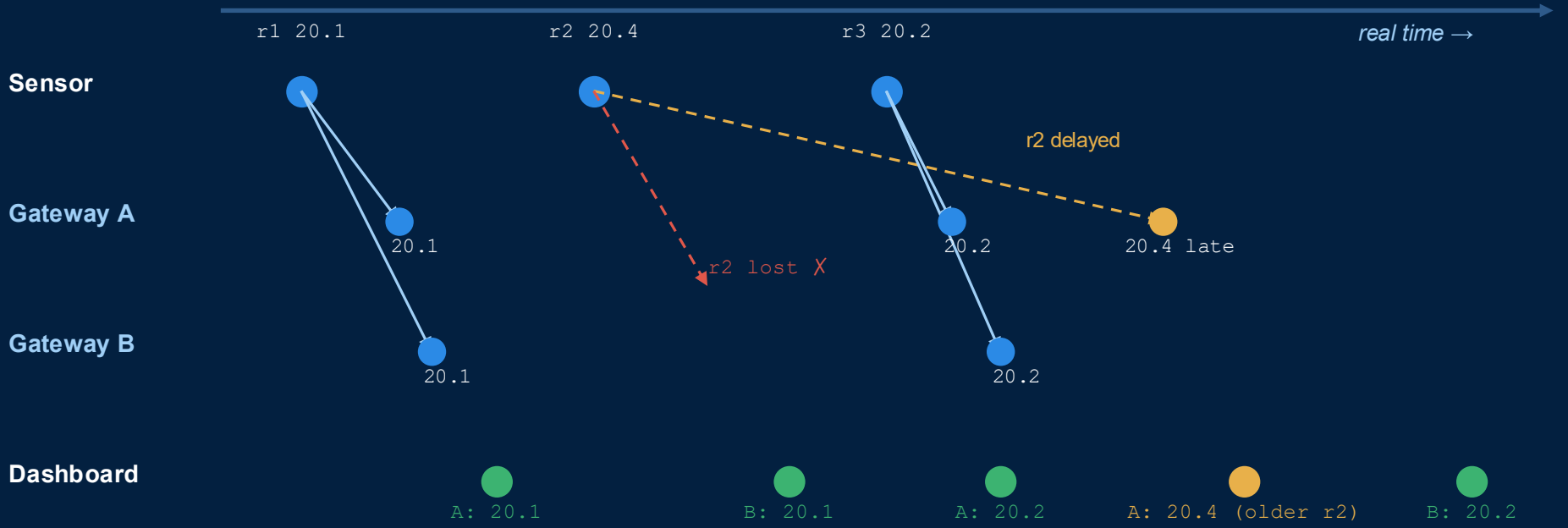
Perfect Use Cases

- **BitTorrent DHT** - Finding peers who have files
- **CDN Cache** - Locating content servers
- **Content Addressing** - IPFS-style hash to location
- **Public Keys** - Cryptographic key distribution
- **Blockchain Headers** - Immutable block data

Why these works?

- Immutable data - Content never changes
- Read-heavy - Infrequent updates
- Multiple sources good - More peers = better availability
- Order doesn't matter - Any valid copy works

Best-effort consistency on a timeline: a temperature sensor





Nobody retransmits, buffers or reconciles. Gateway B never learns r2; gateway A shows r2 after r3. The dashboard keeps the newest timestamp and interpolates the rest.

Game at 60 Hz: a late position update is discarded, a dropped one is dead-reckoned — only the newest state matters.

Delete

How does a removal reach a replica that was offline — and how do we tell "deleted" from "never seen"?

A deletion must reach devices that were offline

1 · Before

A	B (offline)
holds X	holds X

2 · A deletes X

A	B (offline)
X gone	still holds X

3 · B reconnects

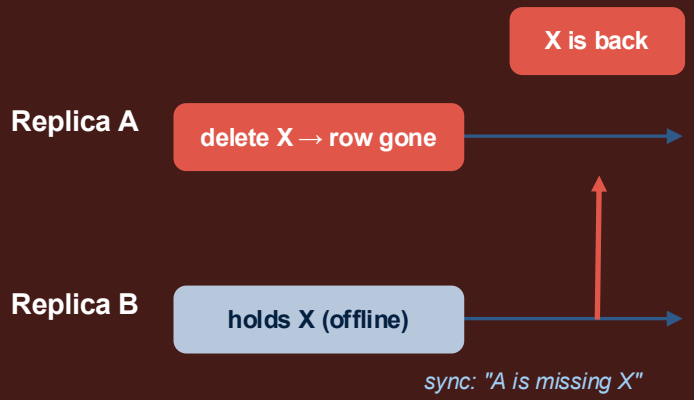
A	B (offline)
X absent…	"A is missing X?"

Without a record of the deletion, absence looks like missing data — and B's old copy resurrects X on both sides.

Delete is not just another write: three ways it goes wrong

1 · Resurrection

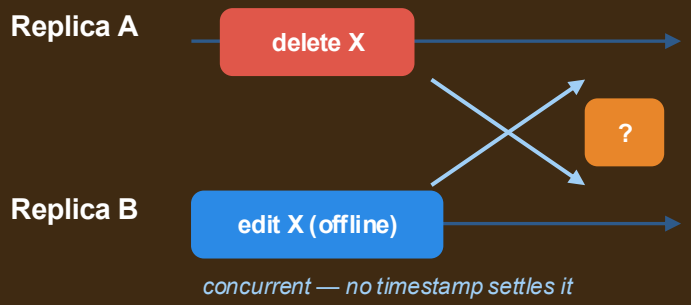
A removes the row. A later repair pass (anti-entropy, defined in Part 4) finds X only on B — and copies it back.



Absence carries no information: "I never had X" and "I deleted X" look the same.

2 · Delete versus concurrent edit

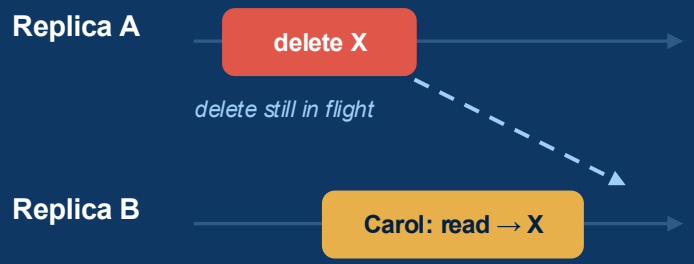
Bob edits X offline while Alice deletes it. Which one wins when they meet?



A wall-clock timestamp cannot tell "edited after seeing the delete" from "edited without knowing".

3 · Stale read

Carol reads from B before the delete has reached it. The task is back — for her only.



A delete is only as visible as the read guarantee you bought in the Read part.

A delete removes the evidence of itself. Every fix on the next slides is a way of keeping that evidence — or of never letting the race happen.

Bayou (Terry et al. 1995): https://doi.org/10.1145/224056.224070 · DeCandia et al. 2007: https://doi.org/10.1145/1294261.1294281

A deletion must reach devices that were offline

1 · Before

A	B (offline)
holds X	holds X

2 · A deletes X

A	B (offline)
X gone	still holds X

3 · B reconnects

A	B (offline)
X absent…	"A is missing X?"

Without a record of the deletion, absence looks like missing data — and B's old copy resurrects X on both sides.

Keep a tombstone

A "deleted X at version v" record wins over B's older copy. Retain it while old updates can still return.

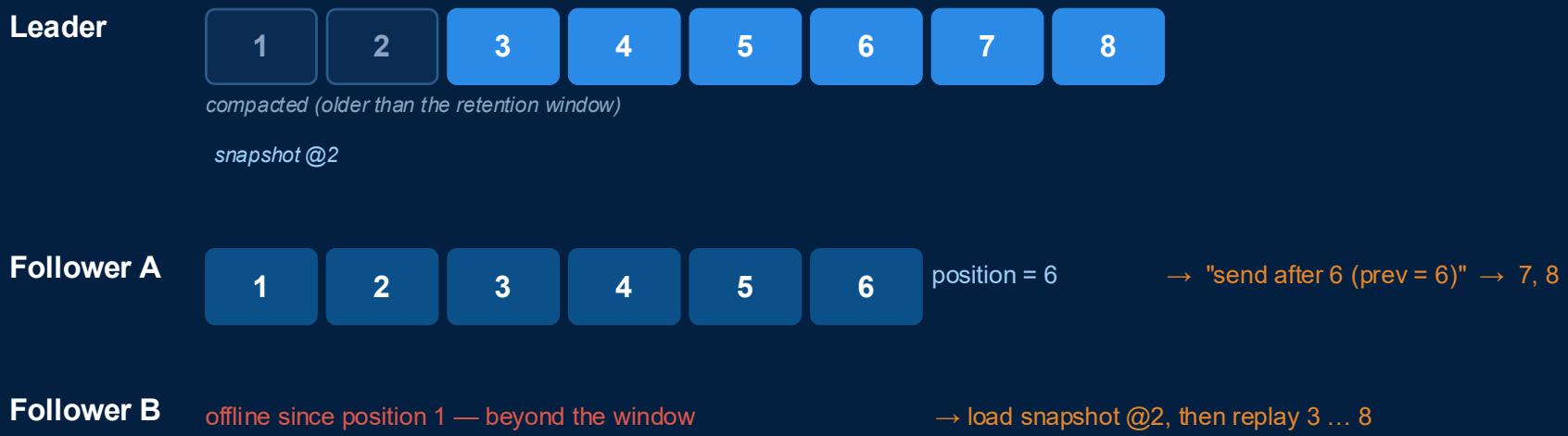
Compact with a rule

Drop tombstones only after every replica has seen them — or force clients older than the retention window to fully resync.

Synchronize and reconciliation

Which changes is a replica missing, and how do concurrent edits combine once they meet?

Log shipping: followers replay the leader's log from a resume position



Position = a complete prefix

Raft ships (prevLogIndex, prevLogTerm) with each batch; the follower rejects it if it lacks that entry. A gap is detected, never skipped.

Snapshot + log tail

Replay inside the retention window; beyond it, install a snapshot and resume from its position. Retention and the supported offline period are one decision.

Where it is used

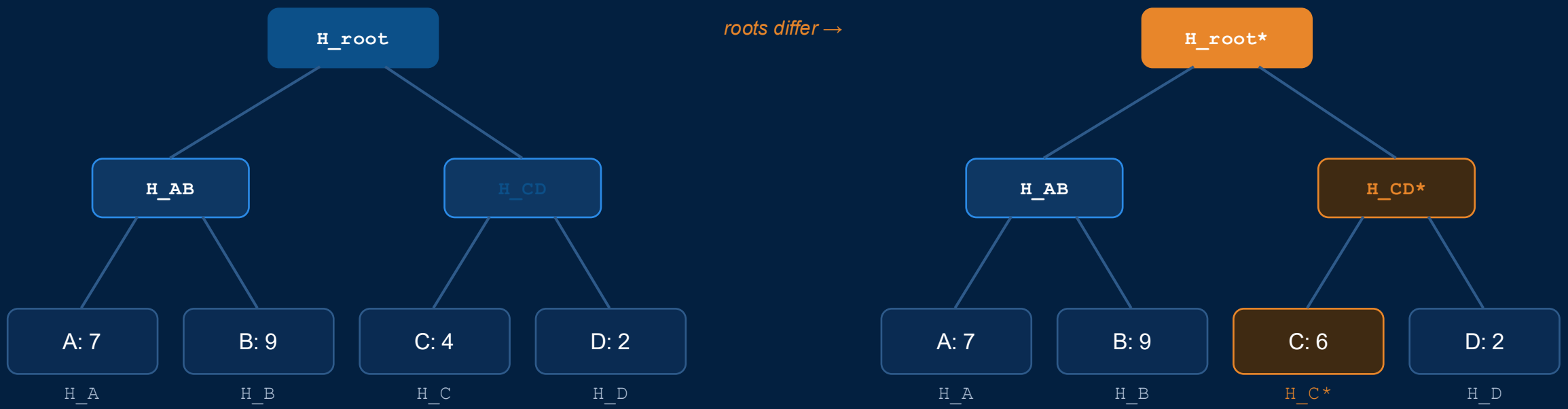
PostgreSQL WAL streaming, MySQL binlog, Kafka offsets, Raft AppendEntries, change-data-capture (Debezium). Several leaders ⇒ one position per leader: a version vector.

One log, one order, one integer to resume from — the price is a single writer per log, and follower lag is the staleness of the Read part.

Raft (Ongaro & Ousterhout 2014): https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

Merkle trees find the differences without sending every value

ALGORITHM · REFERENCE
Merkle-tree anti-entropy (Dynamo §4.7)



Compare roots → descend into the differing child → skip matching subtrees → fetch only C. A differing hash finds work; it cannot say which value is right.

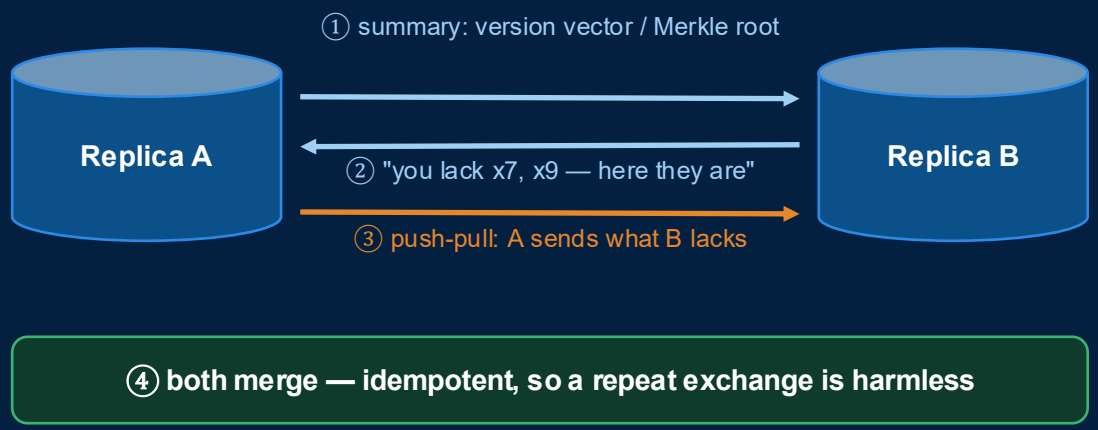
Leaf = hash(key, version, deletion marker, value). Include the metadata, or two replicas can agree on the value and disagree on whether it was deleted.

Beyond Merkle: a Bloom filter or invertible Bloom lookup table (Eppstein et al. 2011) reconciles a small difference in one round instead of O(log n) round trips; range-based reconciliation (Negentropy) does the same over sorted keys.

DeCandia et al. (2007): https://doi.org/10.1145/1294261.1294281 · Merkle (1987): https://doi.org/10.1007/3-540-48184-2_32 · Eppstein et al. 2011: https://doi.org/10.1145/2018436.2018462

Anti-entropy: replicas compare what they hold and repair the difference

Definition (Demers et al., 1987): a background process in which each replica periodically selects a peer and compares or exchanges database state, resolving differences between them. In the push–pull variant, both replicas obtain any updates they lack. “Entropy” informally means inconsistency among replicas; anti-entropy drives replicas toward convergence, including by recovering updates whose earlier transmission was lost.



Peer chosen at random each round: after O(log n) rounds every replica has everything (epidemic spread).

Conflict-Free Replicated Datatypes (CRDT)

In traditional distributed systems, ***we need coordination (locks, consensus protocols)*** to avoid conflicts to handle concurrent changes

- This creates bottlenecks and failure points.

The CRDT Solution: Mathematical properties allow us to design data structures where conflicts are mathematically impossible

- All replicas will converge to the same state
- No information is lost during merging
- No coordination protocols are needed
- Network delays and failures can't break consistency

Use Cases: Offline applications, multi-device sync, sync play lists …

A CRDT makes agreement follow from the data type

ALGORITHM · REFERENCE
CRDTs (Shapiro et al. 2011)

Local updates

Supported edits happen immediately, without waiting for any other replica.

Defined merge

The type specifies its metadata and how any two states combine.

Convergence

Replicas that have seen the same updates hold the same state — regardless of order.

Merge laws — and the delivery fault each one makes harmless

Commutative

$a \sqcup b = b \sqcup a$

Messages arrive in a different order.

Associative

$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$

Messages are batched or split differently.

Idempotent

$a \sqcup a = a$

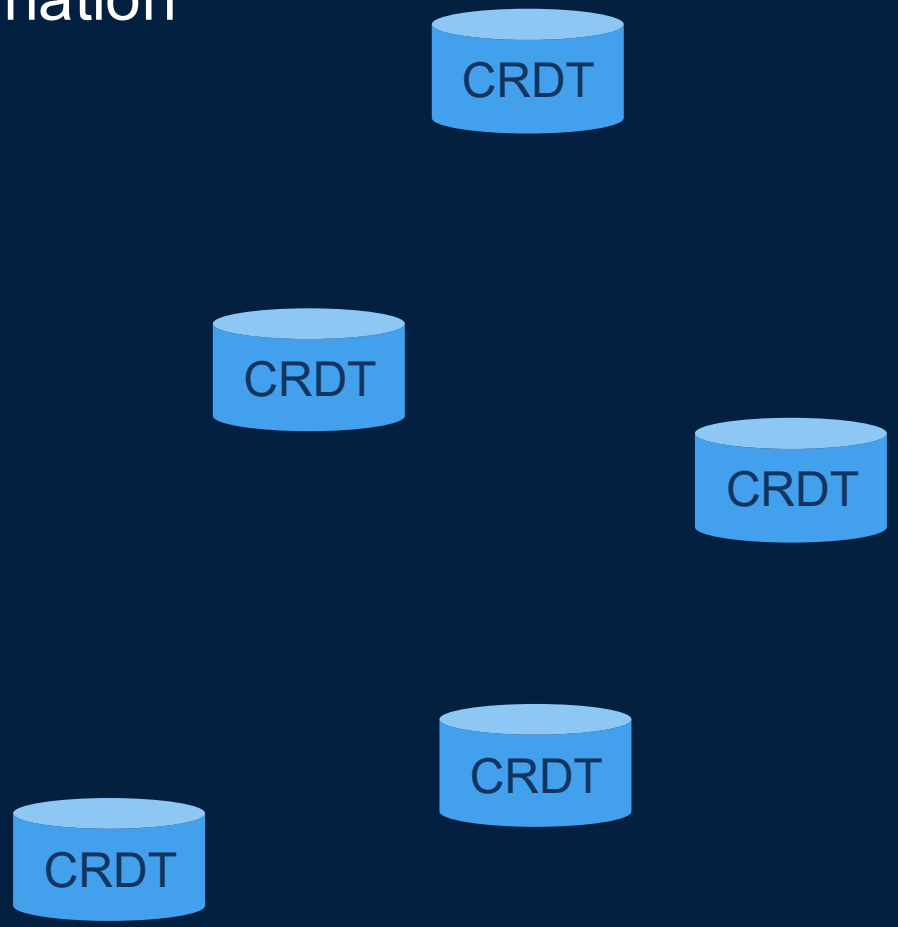
A message is delivered twice.

Convergence is conditional on delivery. An isolated replica does not magically know about remote edits.

Shapiro et al. (2011): https://doi.org/10.1007/978-3-642-24550-3_29 · Shapiro et al. (2011): https://inria.hal.science/inria-00555588

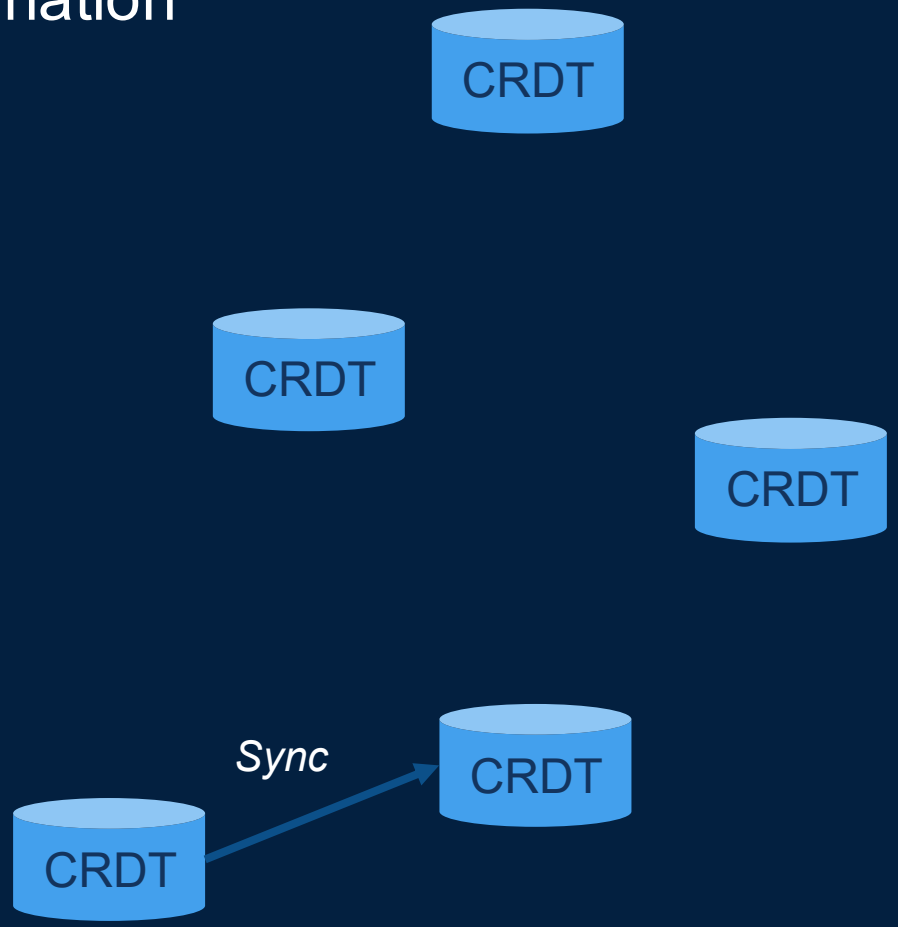
CRDT: Decentralized Data Synchronization

- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*



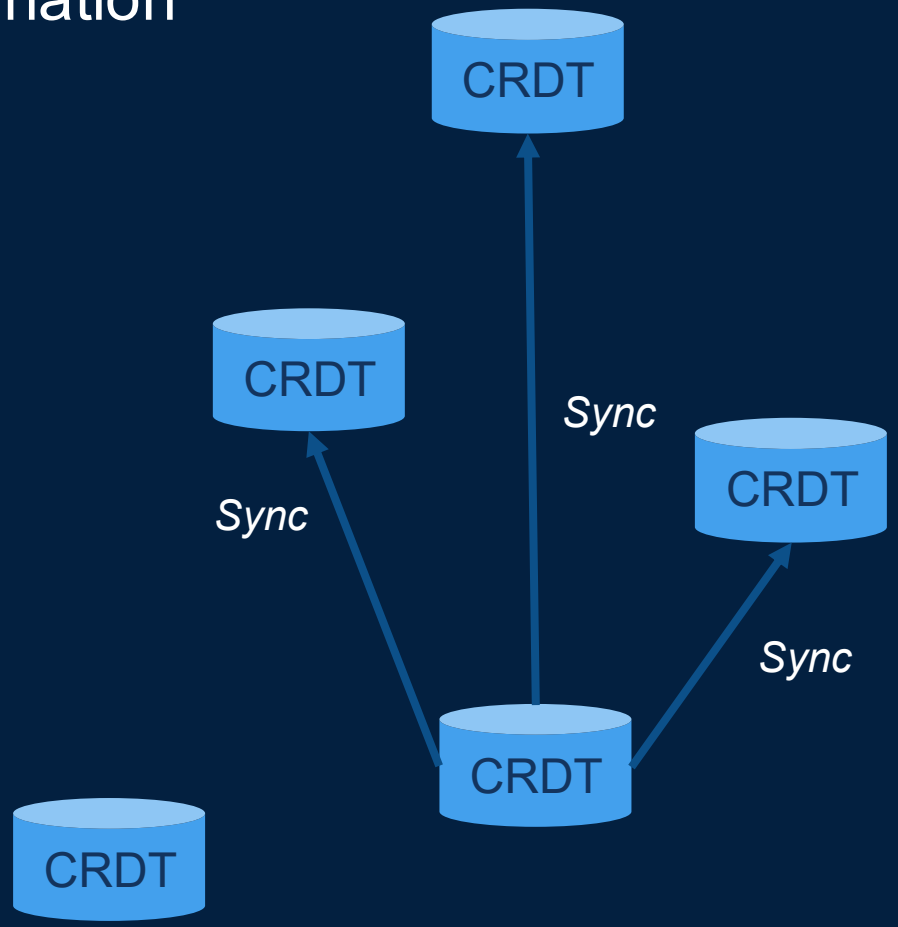
CRDT: Decentralized Data Synchronization

- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*



CRDT: Decentralized Data Synchronization

- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*



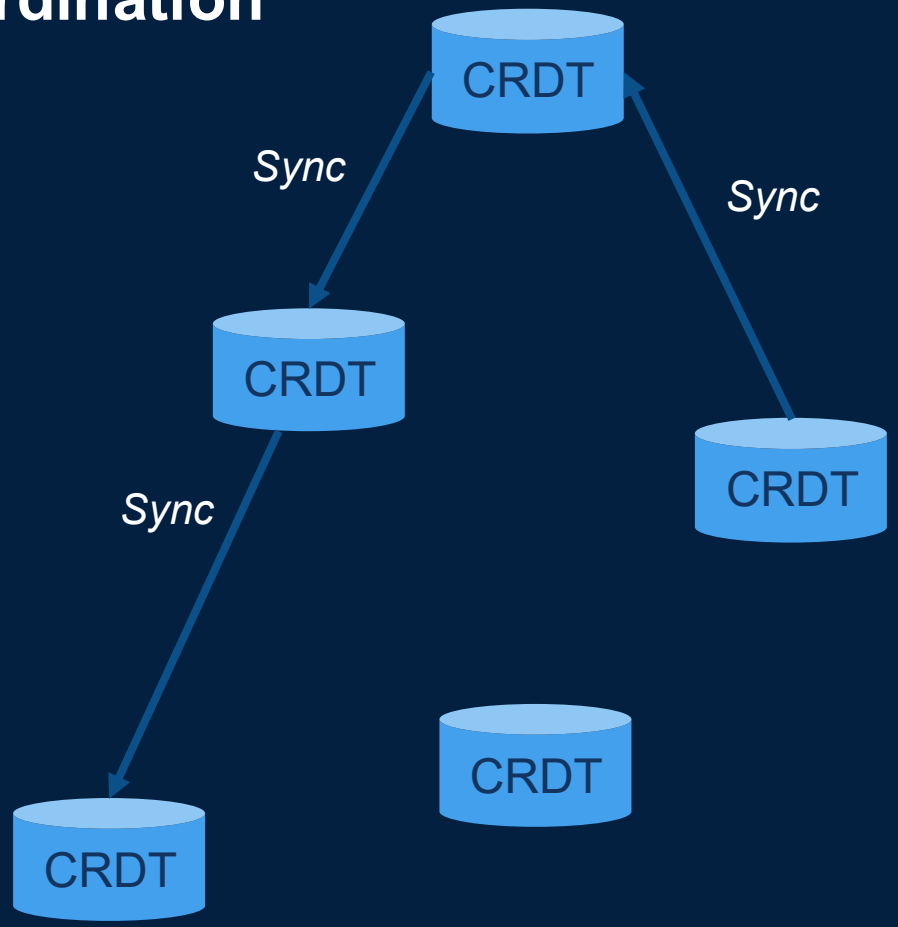
CRDT: Decentralized Data Synchronization

Data Synchronization without central coordination

- Across multiple devices
- Across users
- Across systems

Building block for Modern Apps

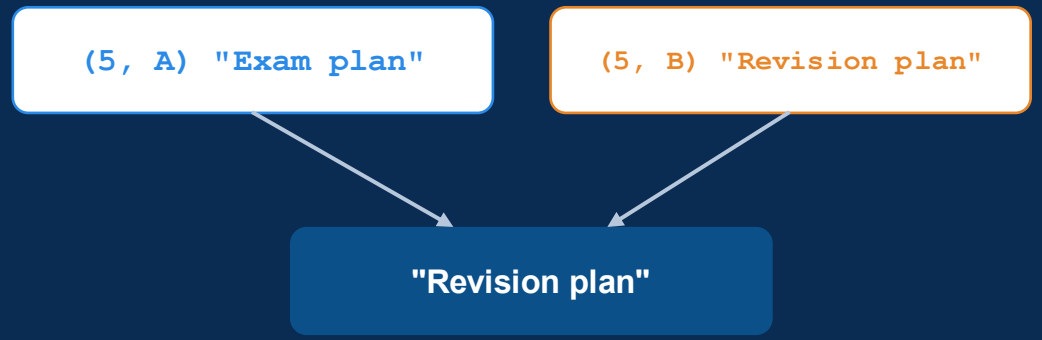
- Mobile First: *Intermittent connectivity, P2P sync*
- Offline First*: Work without central authority*



A title can select a winner or expose the conflict

Alice writes "Exam plan". Bob, concurrently, writes "Revision plan".

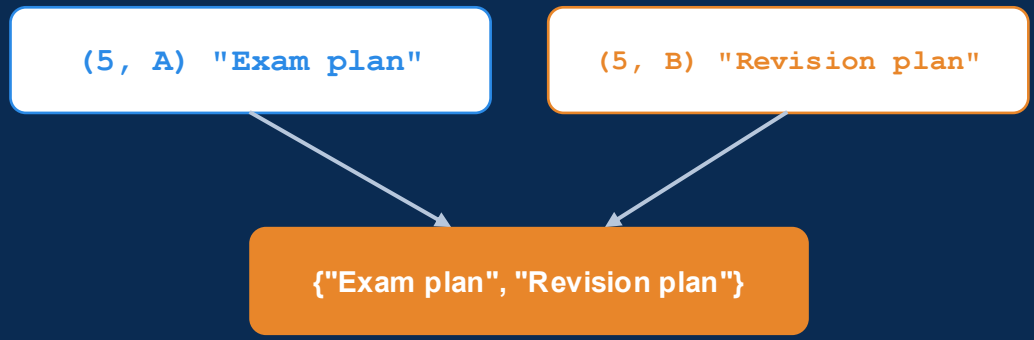
Last-writer-wins register



Total order (counter, replica ID): B > A, so Bob wins. Alice's title is silently gone. Deterministic, simple, lossy.

Tie-braker

Multi-value register



Both concurrent values are kept and shown. A later write that has seen both replaces them. Nothing lost; the UI must handle conflicts.

"Last" means last in the chosen order, not last on the wall clock. If users must see the conflict, use a multi-value register.

Shapiro et al. (2011) §3.2: https://doi.org/10.1007/978-3-642-24550-3_29 · Kulkarni et al. 2014: https://doi.org/10.1007/978-3-319-14472-6_2

Reconcile text

How do two people edit the same sentence at the same time?

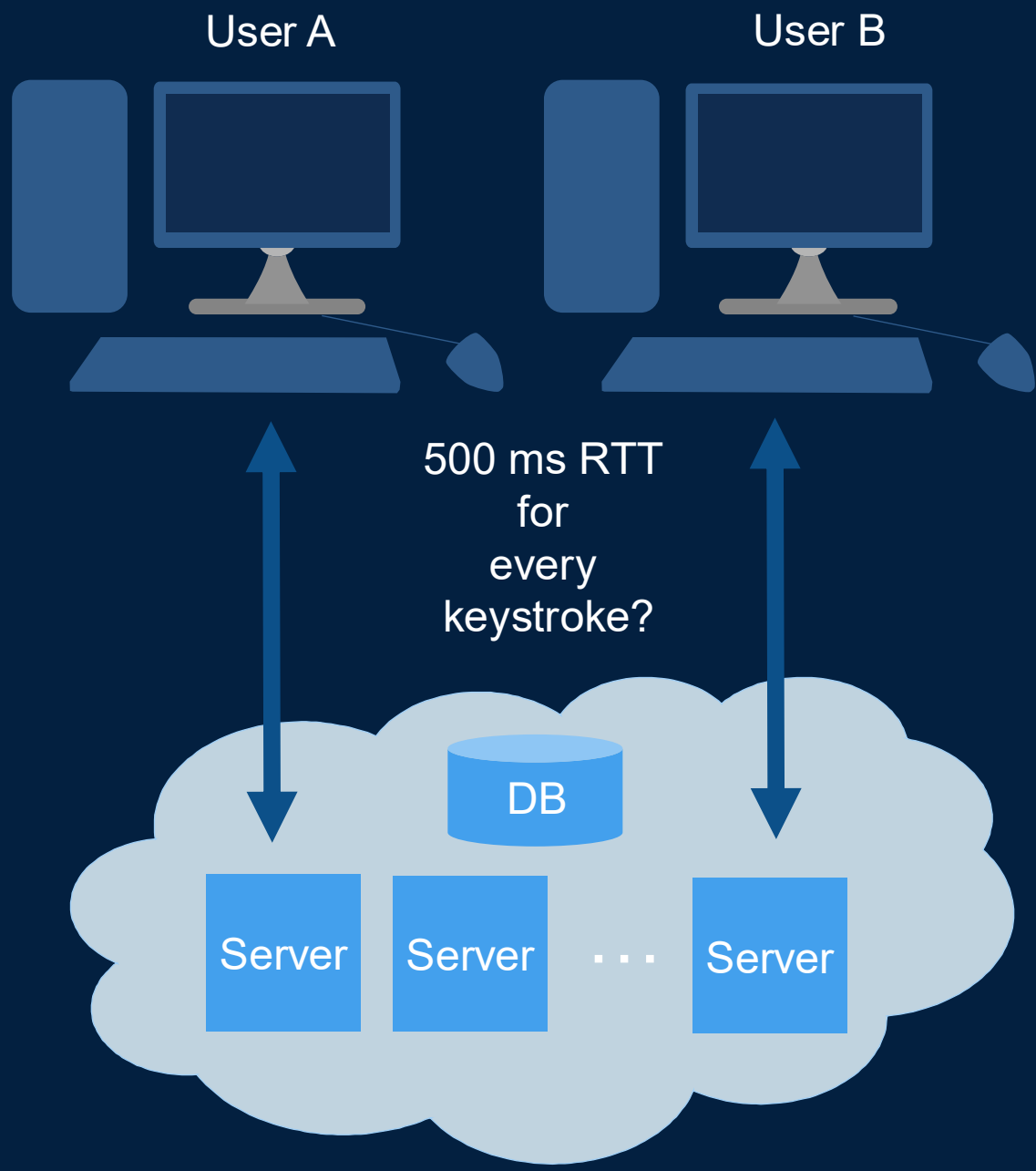
Lock-based solutions are too slow

Network latency kills user experience

- Every keystroke → Server roundtrip → 50-200ms delay
- Server processing: 10-50ms per operation
- Total per action: 50 - 400ms of waiting

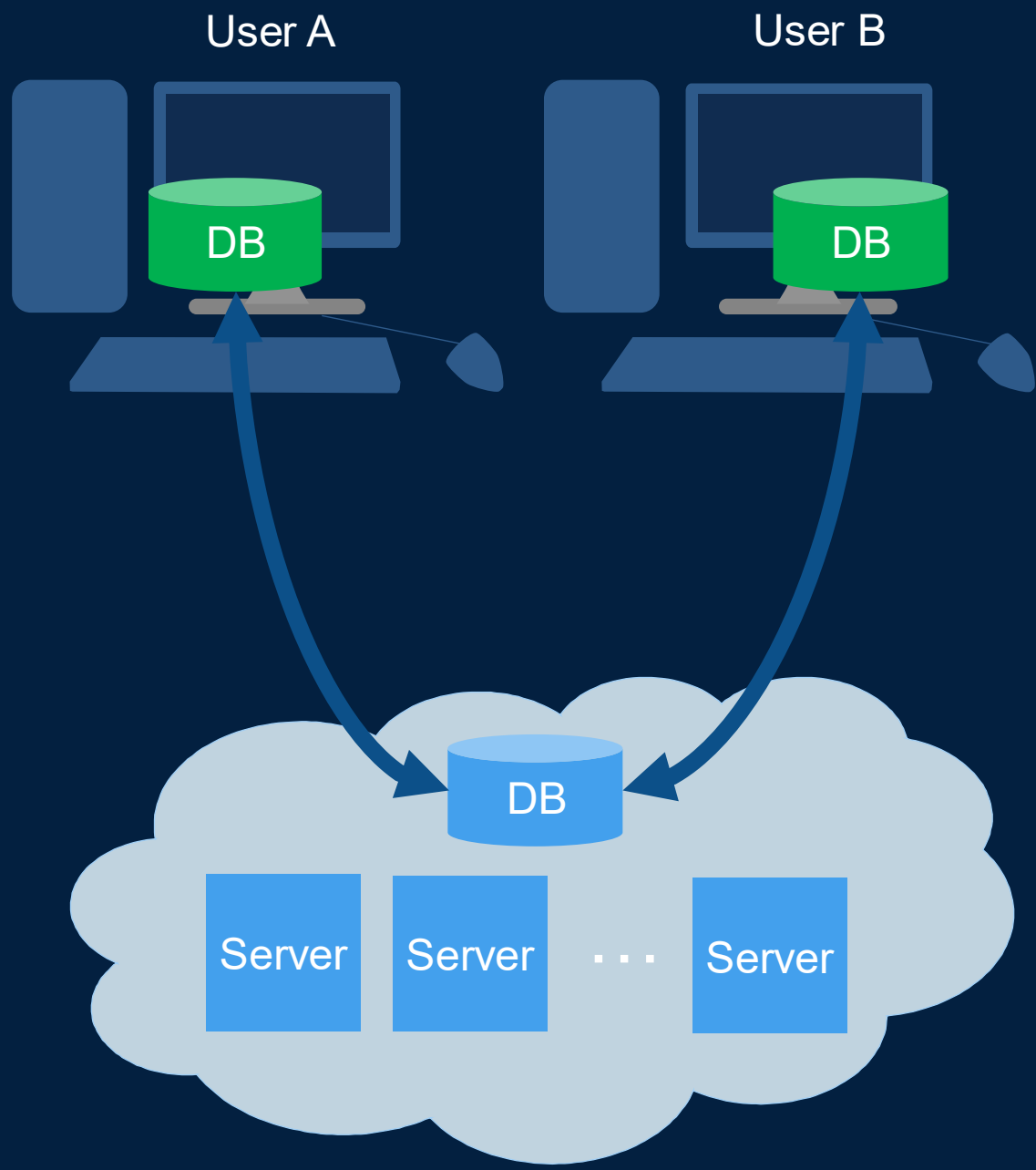
Why wait for server fails?

- Typing feels broken - Characters appear after noticeable lag
- Cursor jumps around - Position conflicts with server state
- Unresponsive UI - Click... wait... update
- Unusable on mobile networks (high latency)



Principles of collaborative editing

1. **Apply changes locally** (optimistic updates)
   Execute operations immediately on local data without waiting for server confirmation
2. **Sync with other users** (in the background)
   Propagate local changes to other users and receive their changes asynchronously
3. **Resolve Conflicts (**automatically)
   When concurrent edits happen, use algorithms to merge them without user intervention
4. **Preserve user intent**
   The system understands what the user was trying to do and maintains that goal even when the document changes.



Operational transformation

A **concurrency control algorithm** used in **real-time collaborative editing** systems

Enables **multiple users** to edit the same document simultaneously **without conflicts**

Key Concepts

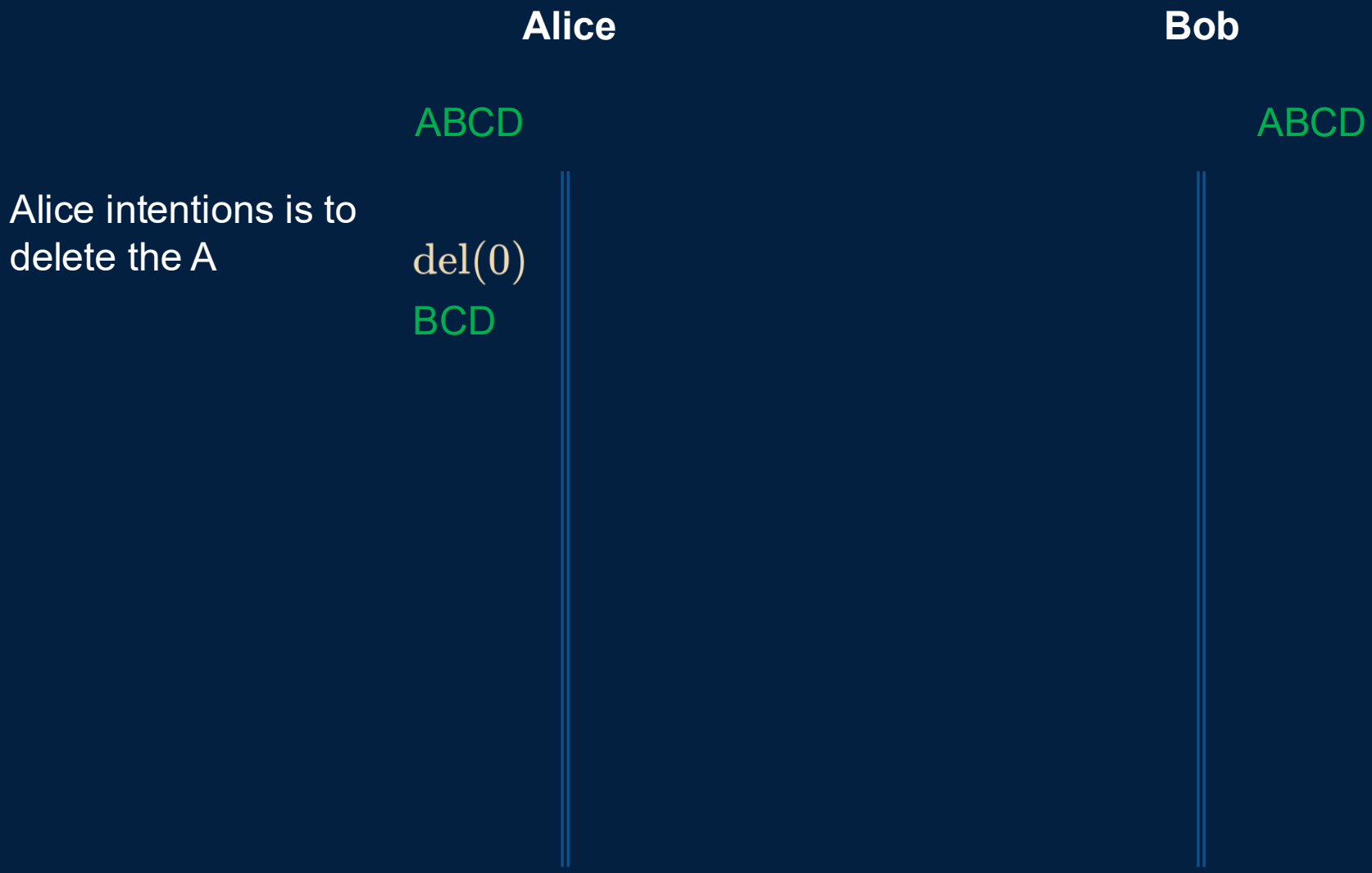
- **Operations:** Insert, delete, update actions on shared data.
- **Transformation:** Adjusts operations to maintain consistency across all user views.
- **Intention Preservation:** Ensures each user's intended changes are preserved, even when edits overlap

Ellis & Gibbs, 1989: https://dl.acm.org/doi/10.1145/66926.66963

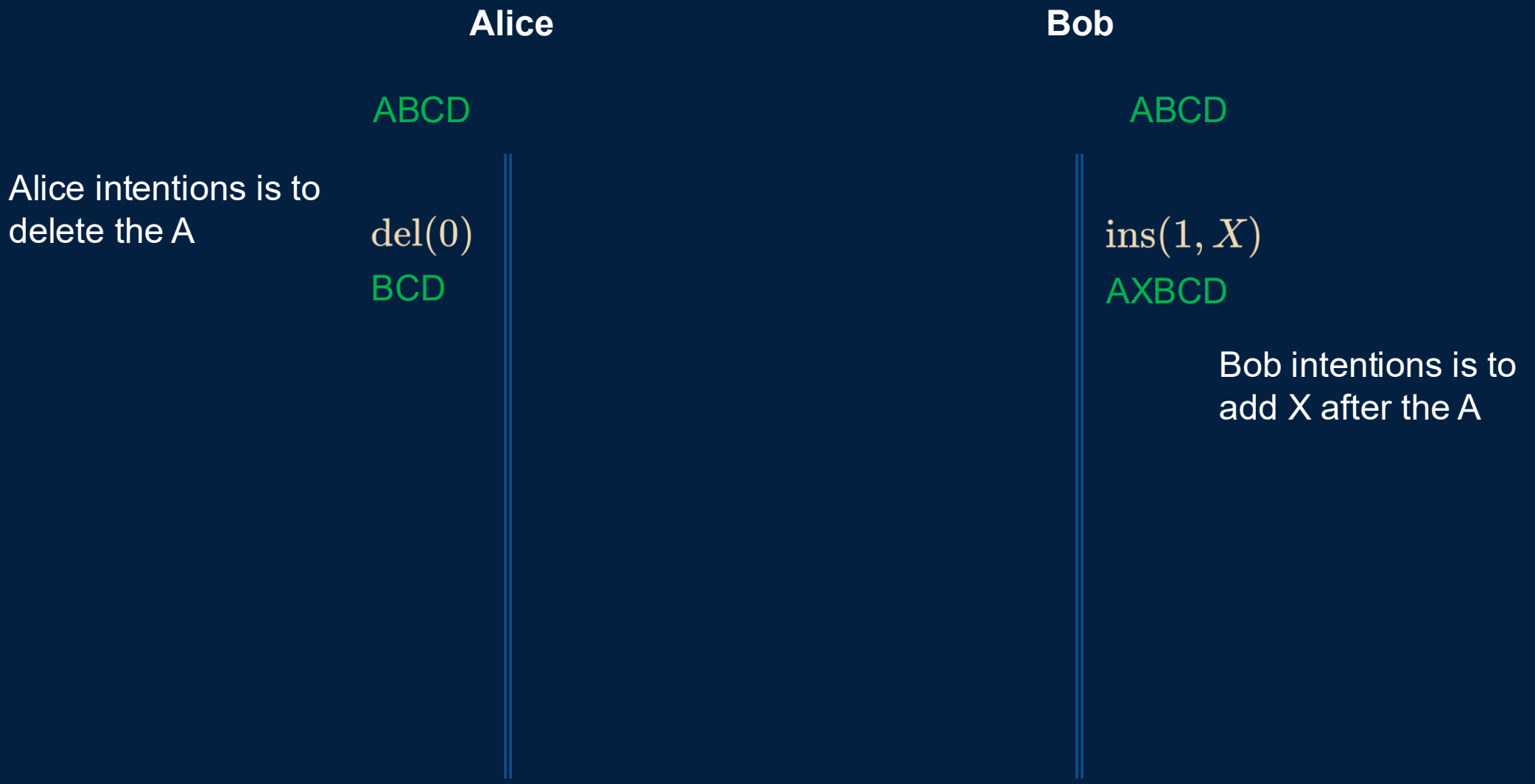
Example: two concurrent edits on ABCD



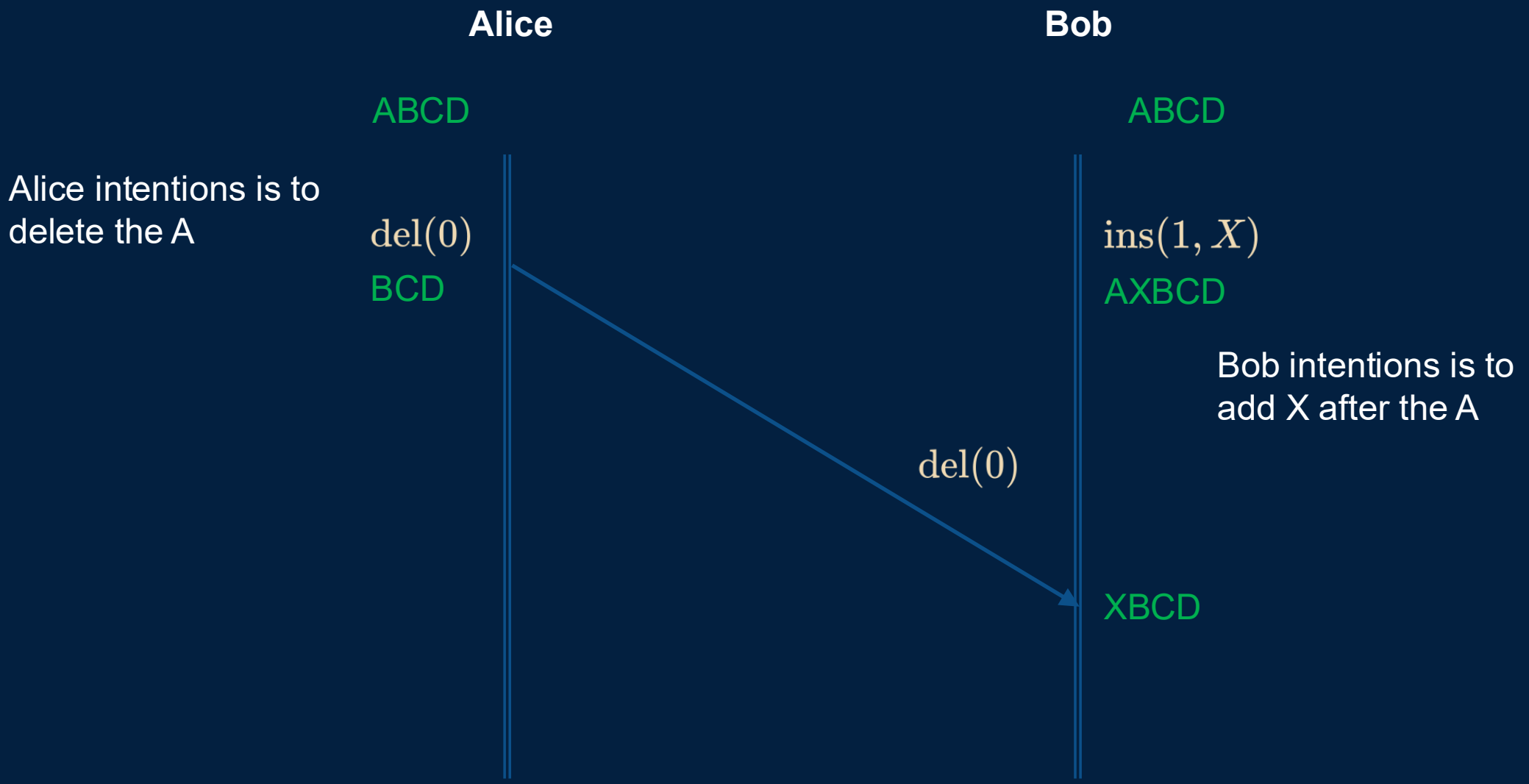
Example: Alice deletes A



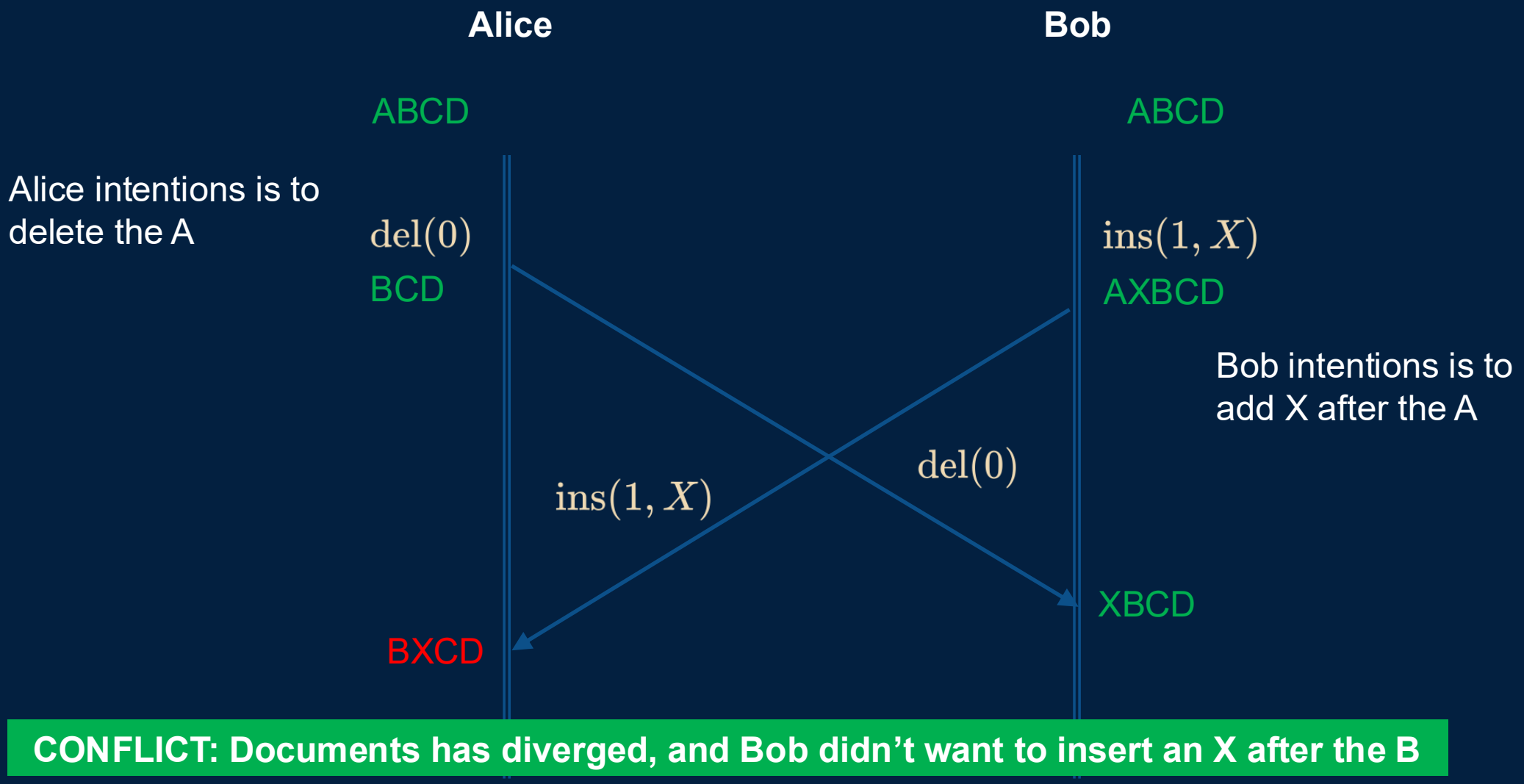
Example: Bob inserts X after A



Example: each side receives the other's operation

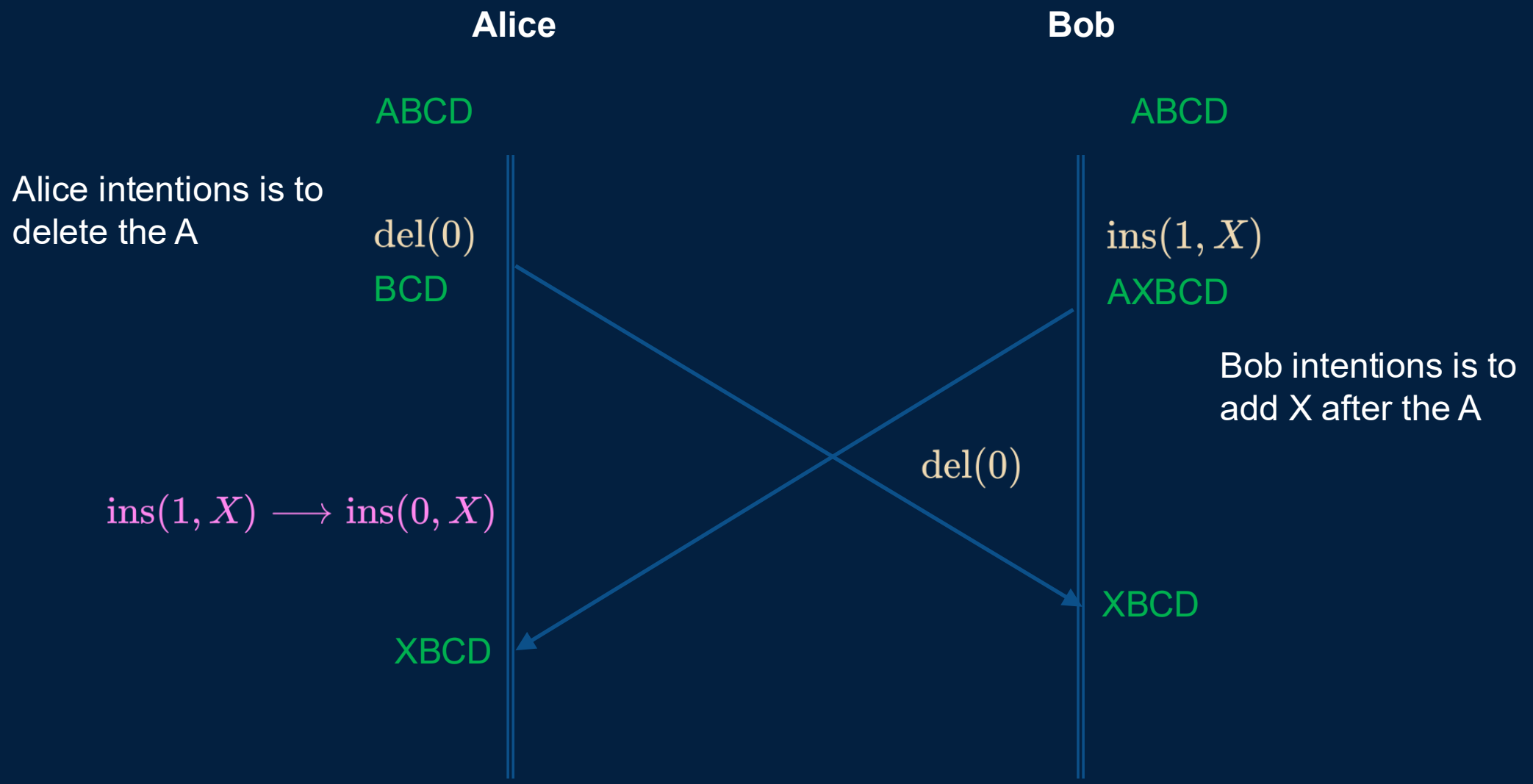


Example: the documents diverge



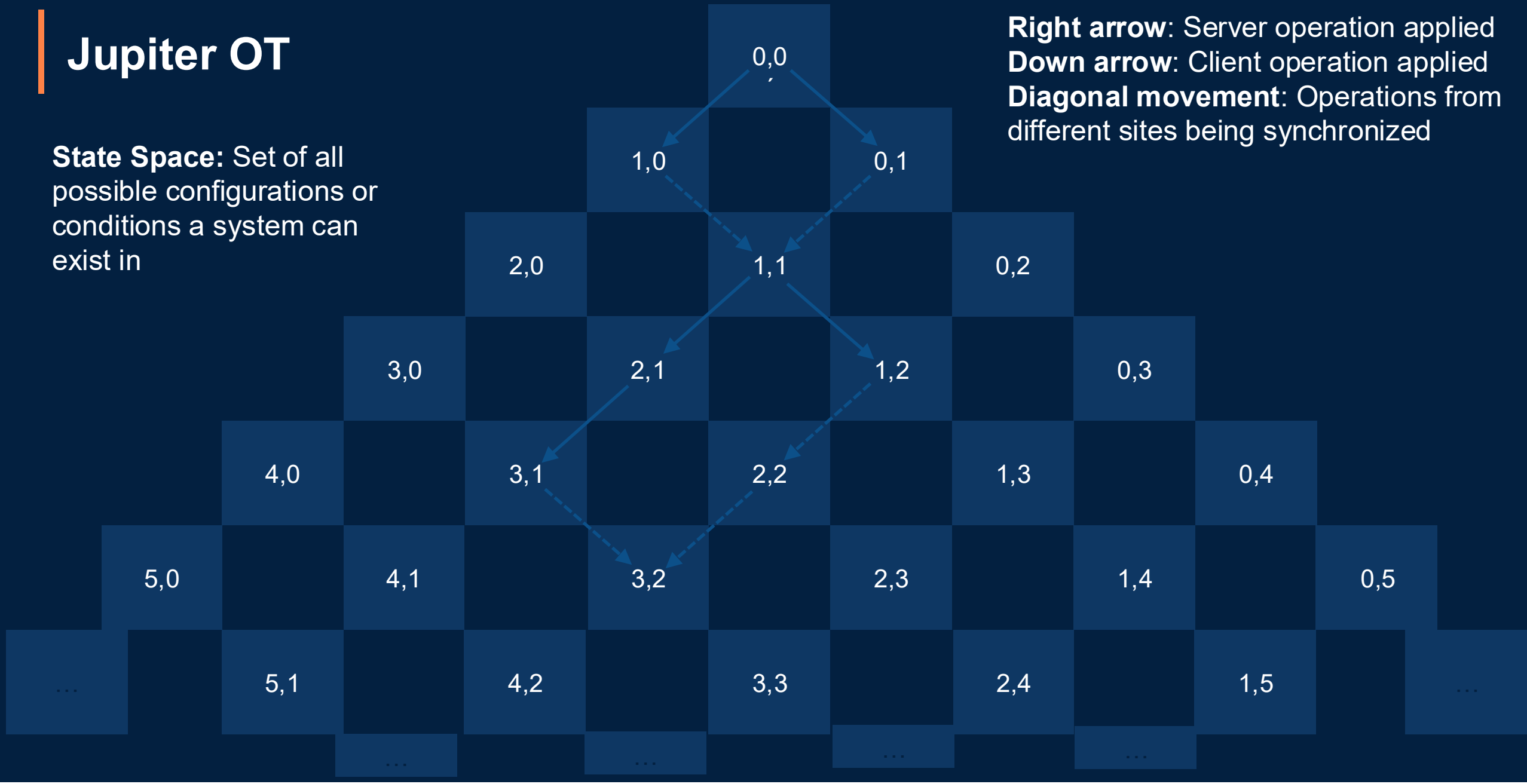
CONFLICT: Documents has diverged, and Bob didn't want to insert an X after the B

Operational Transformation



Jupiter OT

State Space: Set of all possible configurations or conditions a system can exist in



Right arrow: Server operation applied
Down arrow: Client operation applied
Diagonal movement: Operations from different sites being synchronized

Transformation functions for insert and delete

The Transformation Function $T : \mathrm{O}_{\text{target}} \times \mathrm{O}_{\text{ref}} \rightarrow \mathrm{O}^{T}_{\text{target}}$ adjusts the position of operation $\mathrm{O}_{\text{target}}$ to account for the positional changes caused by operation O_{ref}, ensuring it can be applied correctly while preserving its original intentions

$$T(\mathrm{ins}(p_1, c_1), \mathrm{ins}(p_2, c_2)) = \begin{cases} \mathrm{ins}(p_1, c_1) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 + 1, c_1) & \text{if } p_1 > p_2 \end{cases}$$

$$T(\mathrm{ins}(p_1, c), \mathrm{del}(p_2)) = \begin{cases} \mathrm{ins}(p_1, c) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 - 1, c) & \text{if } p_1 > p_2 \end{cases}$$

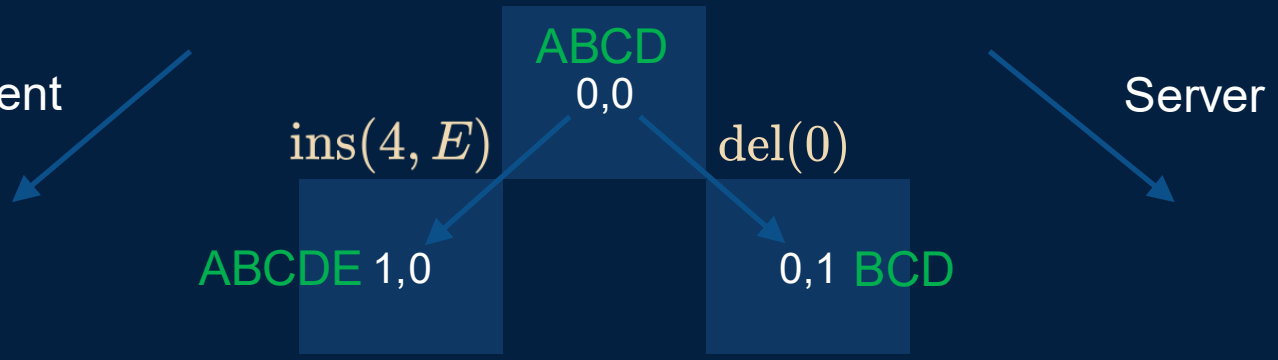
$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

$$T(\mathrm{del}(p_1), \mathrm{del}(p_2)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 - 1) & \text{if } p_1 > p_2 \\ \mathrm{nop}() & \text{if } p_1 = p_2 \end{cases}$$

Jupiter OT

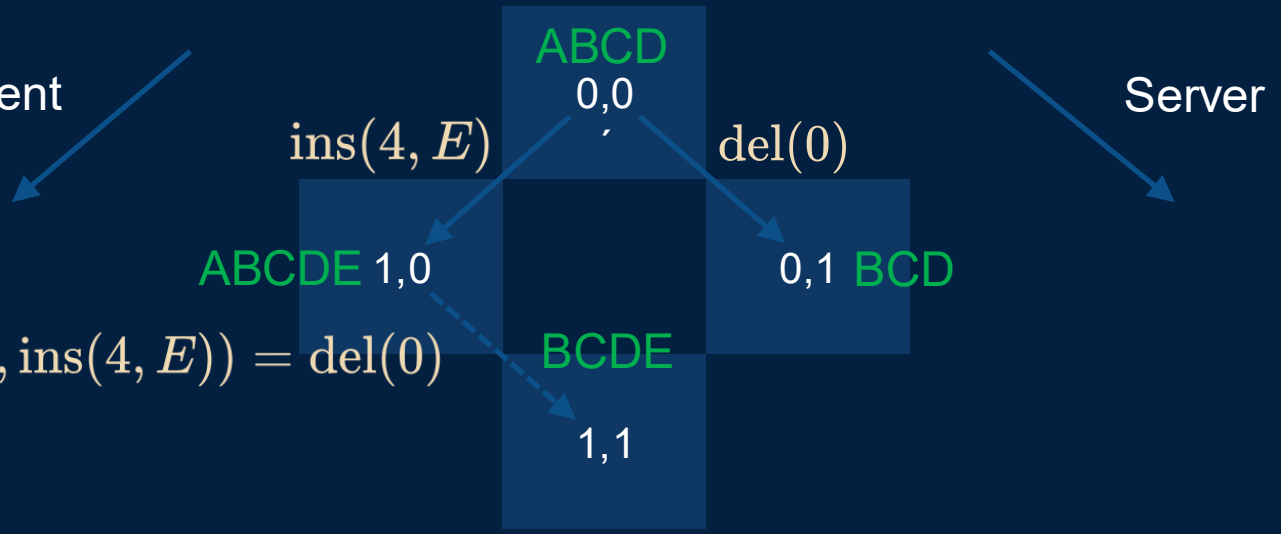


Jupiter OT



Client and Server (via other clients) edits the text simultaneously

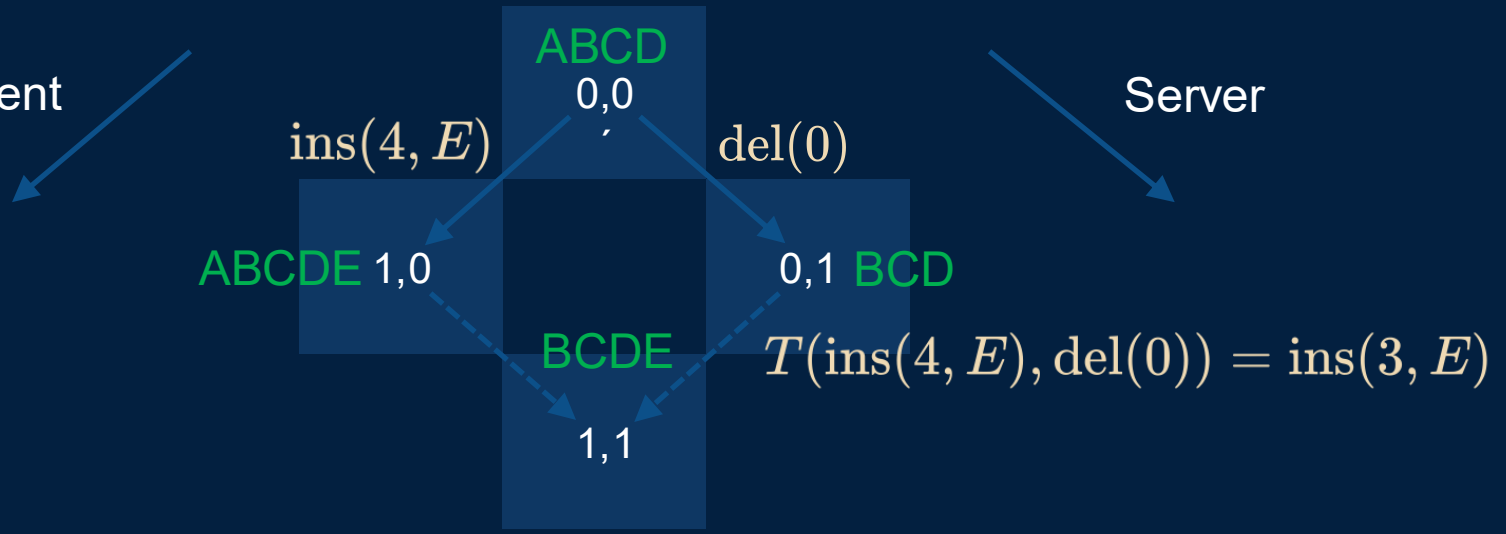
Jupiter OT



$$T(\mathrm{del}(0), \mathrm{ins}(4, E)) = \mathrm{del}(0)$$

$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

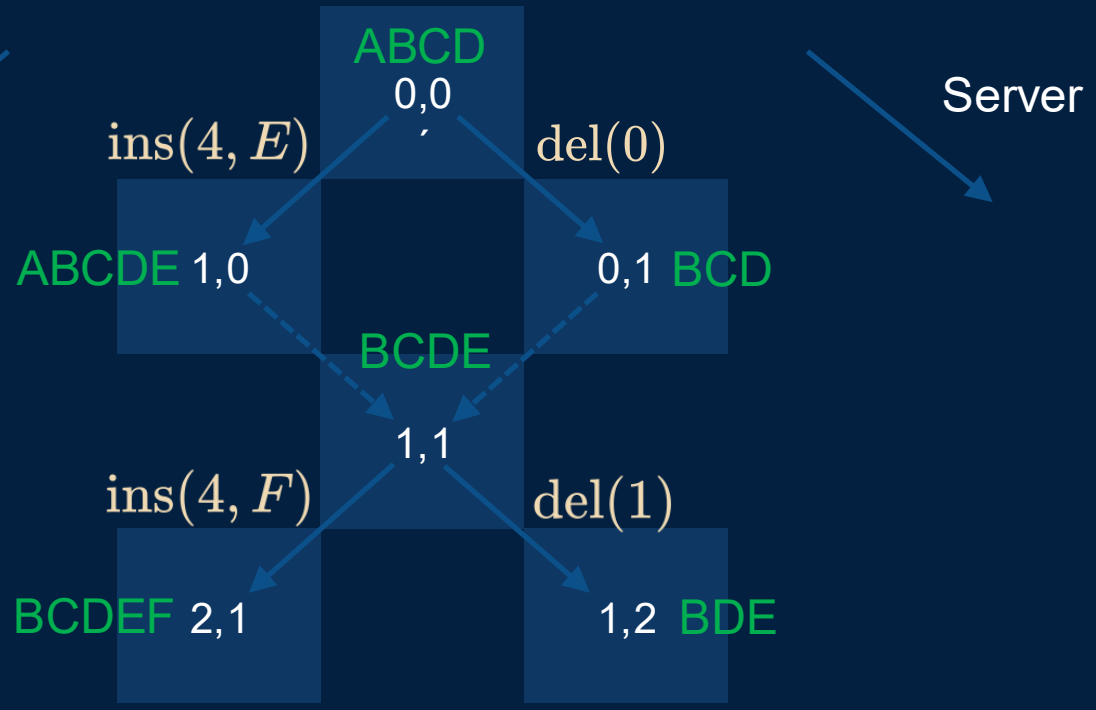
Jupiter OT



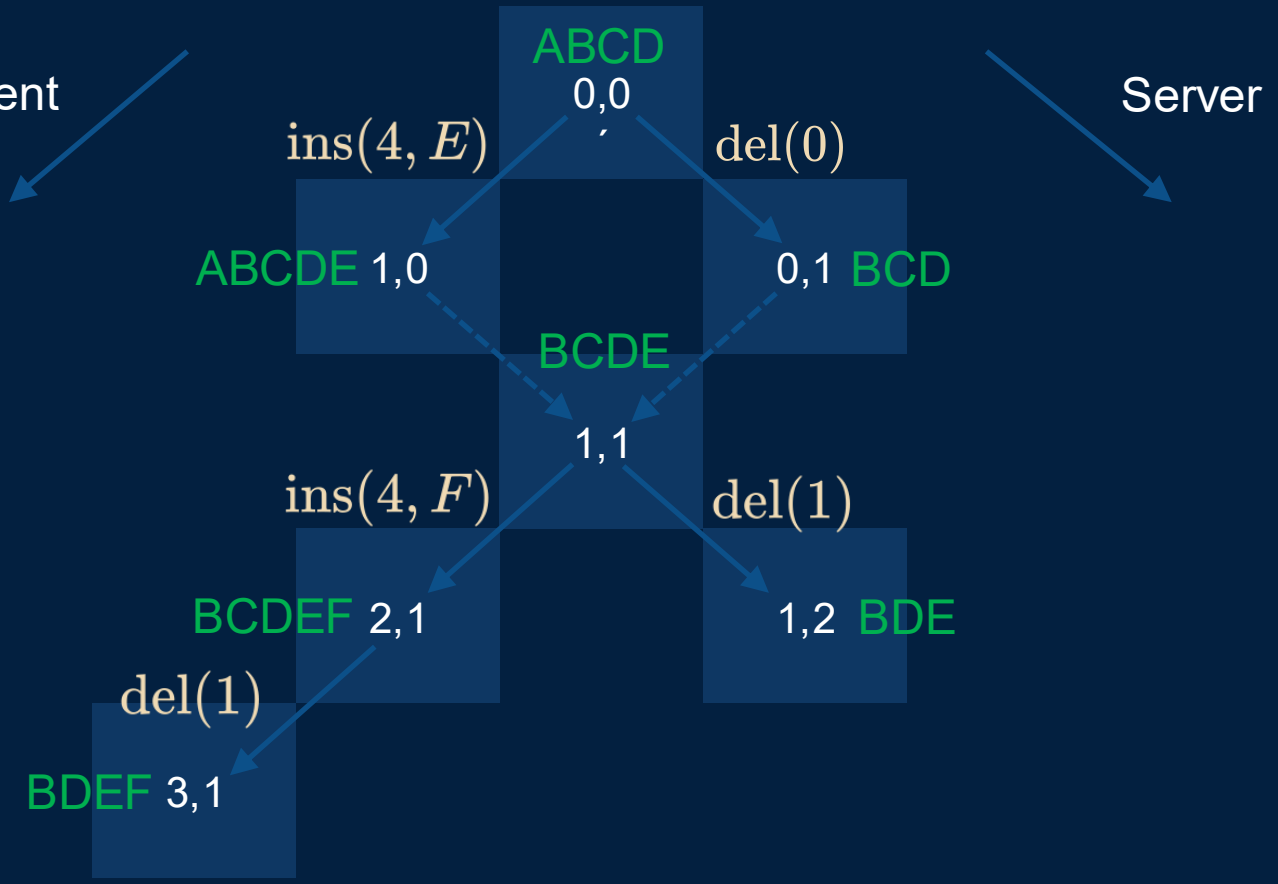
$$T(\mathrm{ins}(4, E), \mathrm{del}(0)) = \mathrm{ins}(3, E)$$

$$T(\mathrm{ins}(p_1, c), \mathrm{del}(p_2)) = \begin{cases} \mathrm{ins}(p_1, c) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 - 1, c) & \text{if } p_1 > p_2 \end{cases}$$

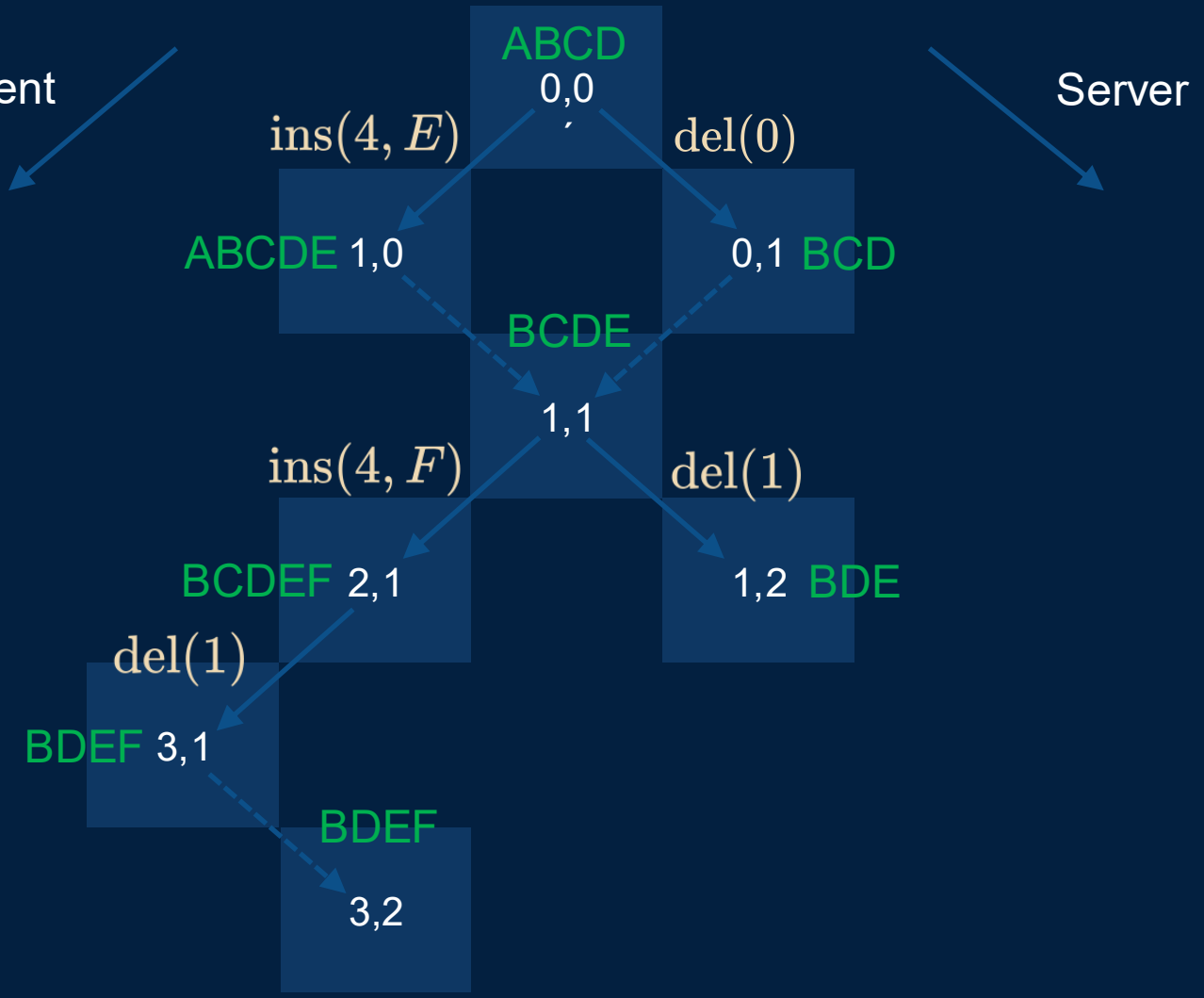
Jupiter OT


Client
Server
ABCD
0,0
$\mathrm{ins}(4, E)$
$\mathrm{del}(0)$
ABCDE 1,0
0,1 BCD
BCDE
1,1
$\mathrm{ins}(4, F)$
$\mathrm{del}(1)$
BCDEF 2,1
1,2 BDE

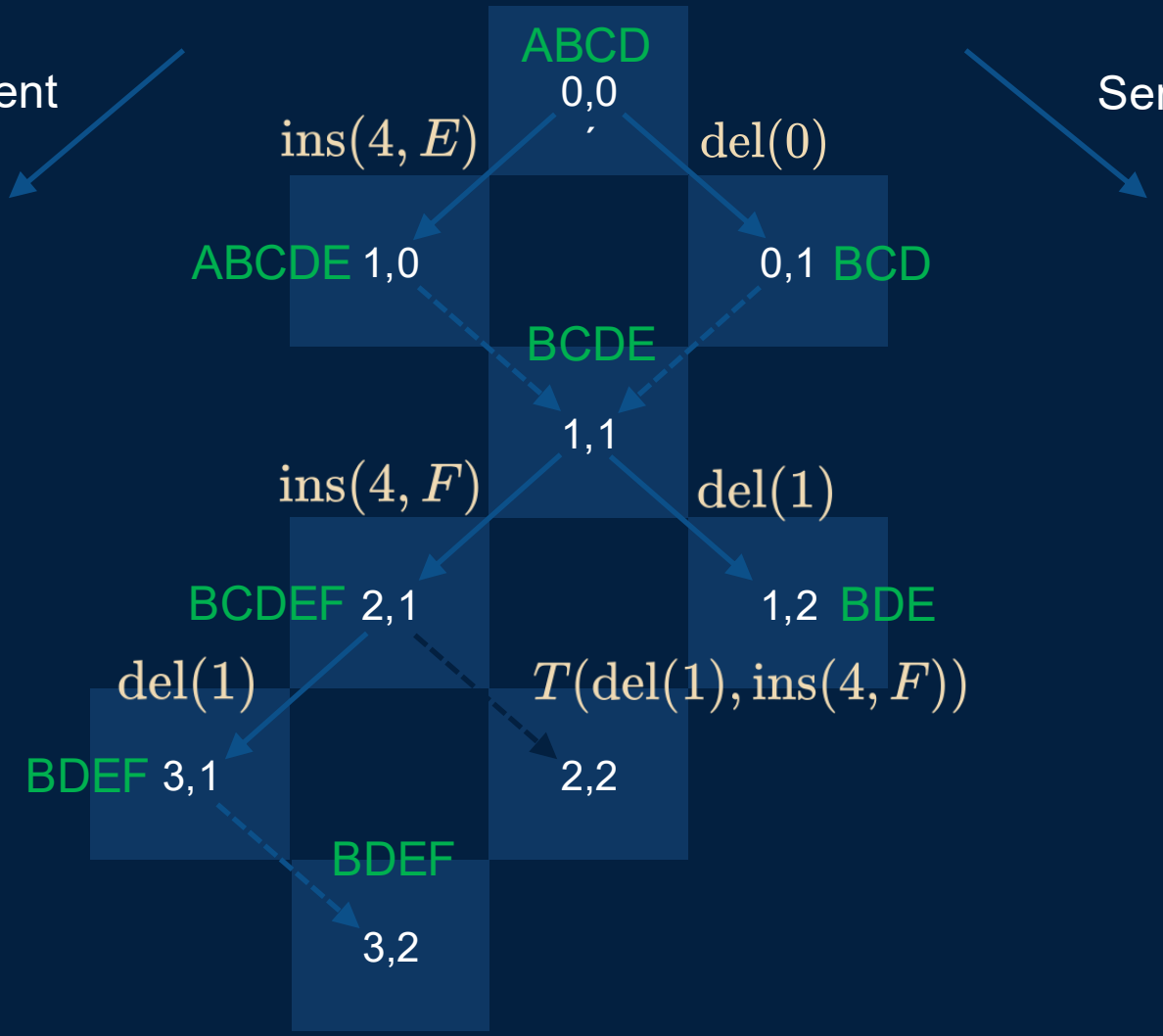
Jupiter OT


Client
Server
ABCD
0,0
$\mathrm{ins}(4, E)$
$\mathrm{del}(0)$
ABCDE 1,0
0,1 BCD
BCDE
1,1
$\mathrm{ins}(4, F)$
$\mathrm{del}(1)$
BCDEF 2,1
1,2 BDE
$\mathrm{del}(1)$
BDEF 3,1

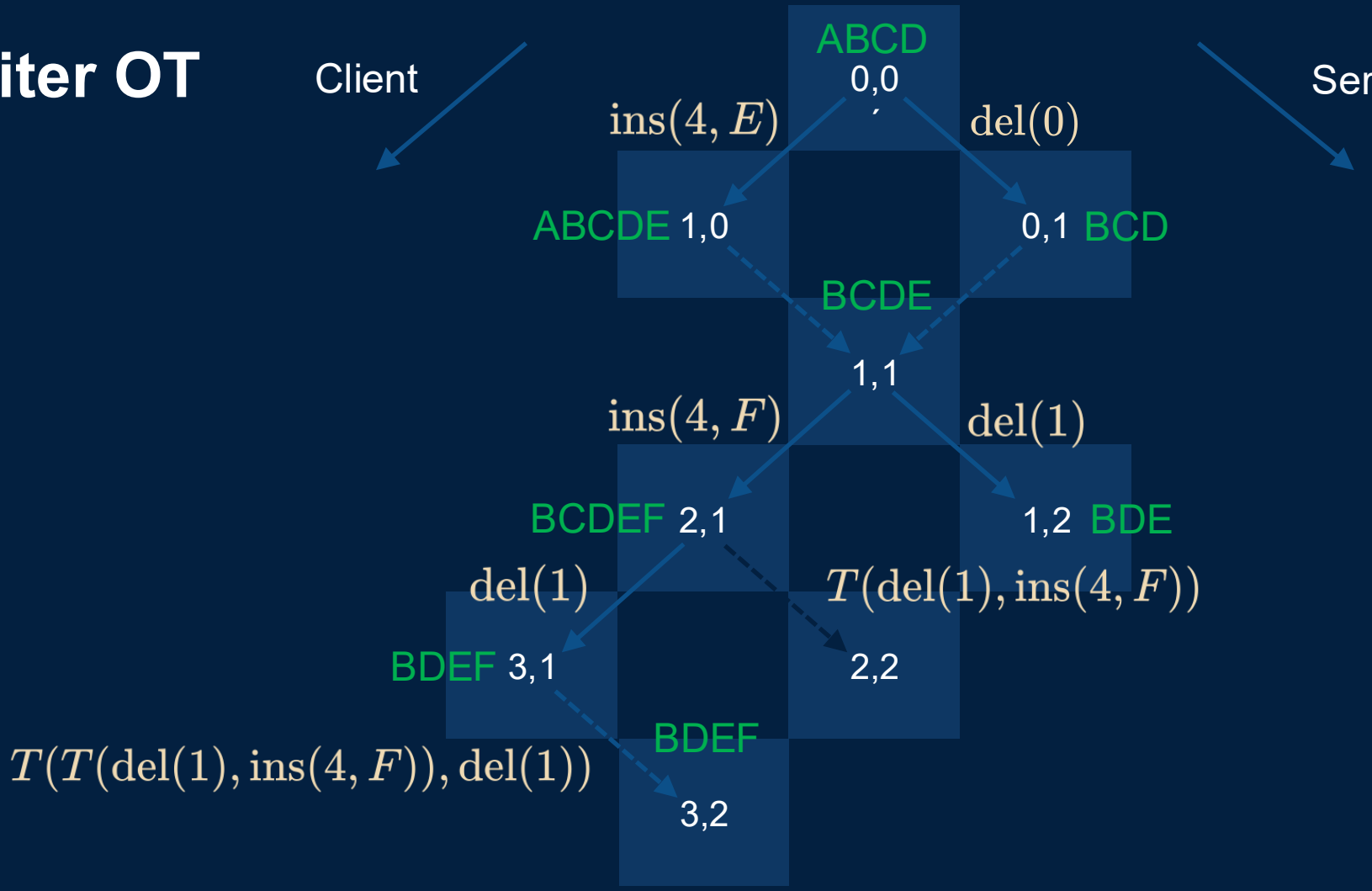
Jupiter OT


Client
Server
ABCD
0,0
$\mathrm{ins}(4, E)$
$\mathrm{del}(0)$
ABCDE 1,0
0,1 BCD
BCDE
1,1
$\mathrm{ins}(4, F)$
$\mathrm{del}(1)$
BCDEF 2,1
1,2 BDE
$\mathrm{del}(1)$
BDEF 3,1
BDEF
3,2

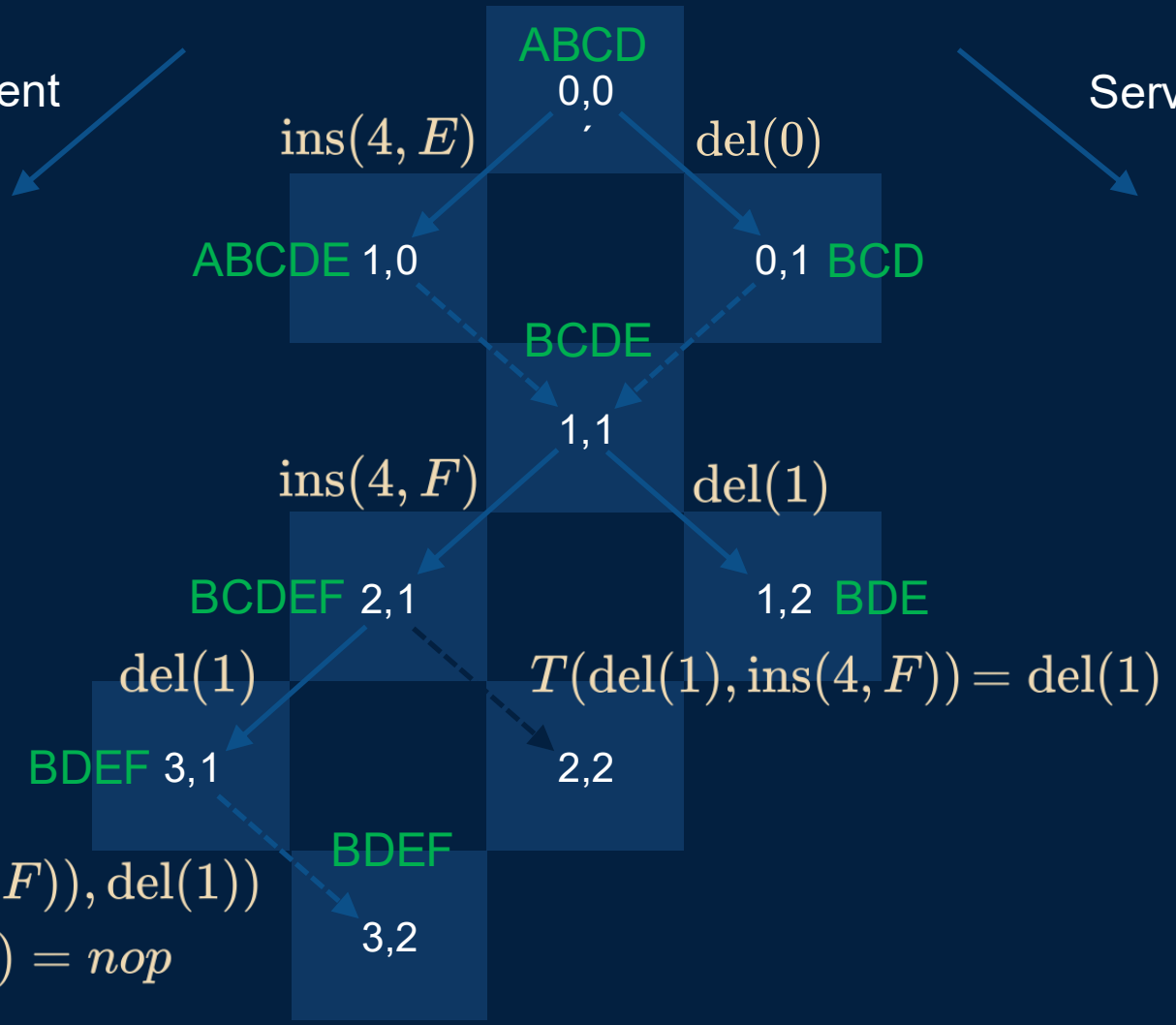
Jupiter OT


Client
Server
ABCD
0,0
$\mathrm{ins}(4, E)$
$\mathrm{del}(0)$
ABCDE 1,0
0,1 BCD
BCDE
1,1
$\mathrm{ins}(4, F)$
$\mathrm{del}(1)$
BCDEF 2,1
1,2 BDE
$\mathrm{del}(1)$
$T(\mathrm{del}(1), \mathrm{ins}(4, F))$
BDEF 3,1
2,2
BDEF
3,2

Jupiter OT


Client
Server
ABCD
0,0
$\mathrm{ins}(4, E)$
$\mathrm{del}(0)$
ABCDE 1,0
0,1 BCD
BCDE
1,1
$\mathrm{ins}(4, F)$
$\mathrm{del}(1)$
BCDEF 2,1
1,2 BDE
$\mathrm{del}(1)$
$T(\mathrm{del}(1), \mathrm{ins}(4, F))$
BDEF 3,1
2,2
BDEF
$T(T(\mathrm{del}(1), \mathrm{ins}(4, F)), \mathrm{del}(1))$
3,2

Jupiter OT

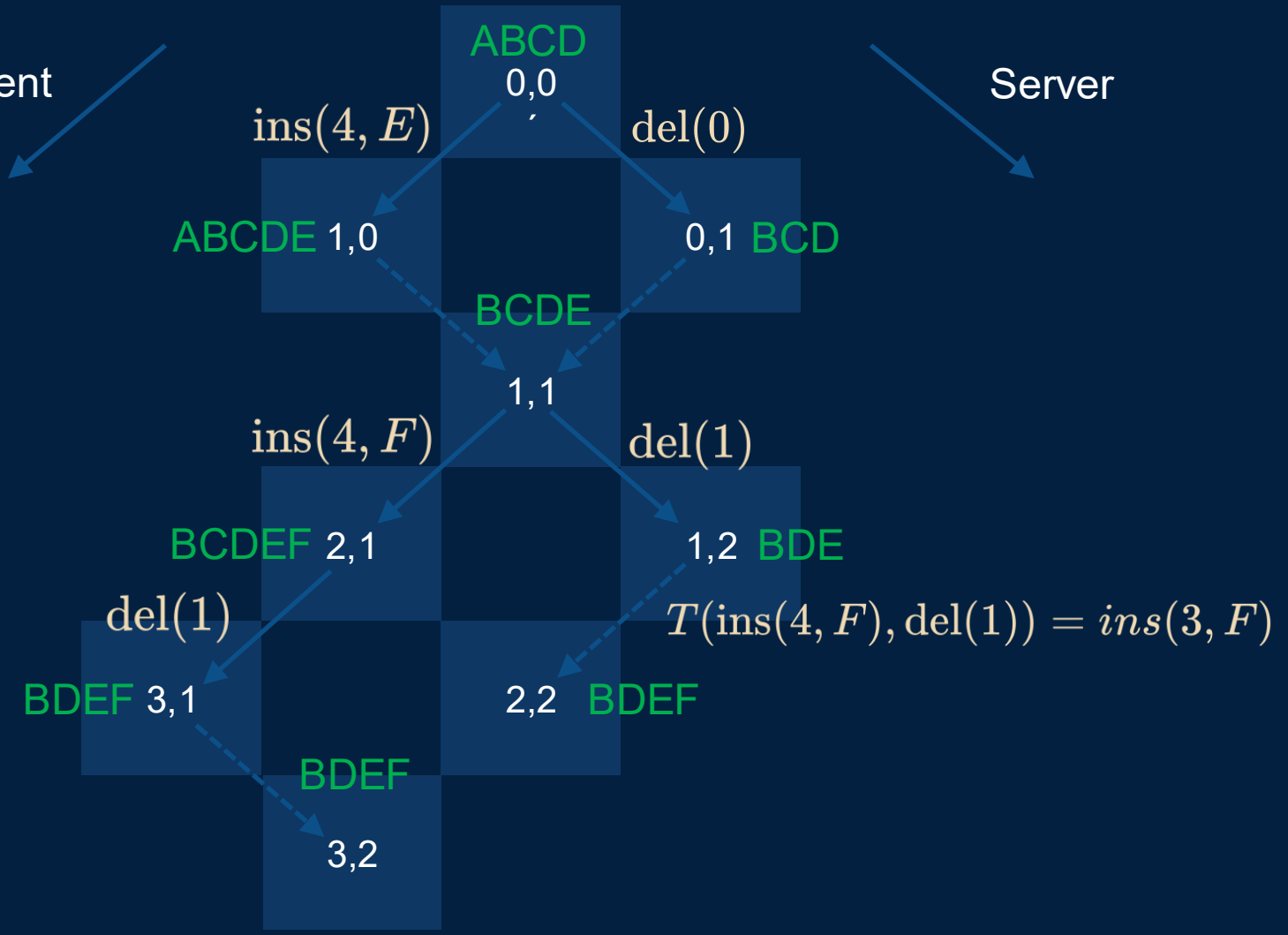


$$T(T(\mathrm{del}(1), \mathrm{ins}(4, F)), \mathrm{del}(1)) = T(\mathrm{del}(1), \mathrm{del}(1)) = nop$$

$$T(\mathrm{del}(p_1), \mathrm{del}(p_2)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 - 1) & \text{if } p_1 > p_2 \\ \mathrm{nop}() & \text{if } p_1 = p_2 \end{cases}$$

$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

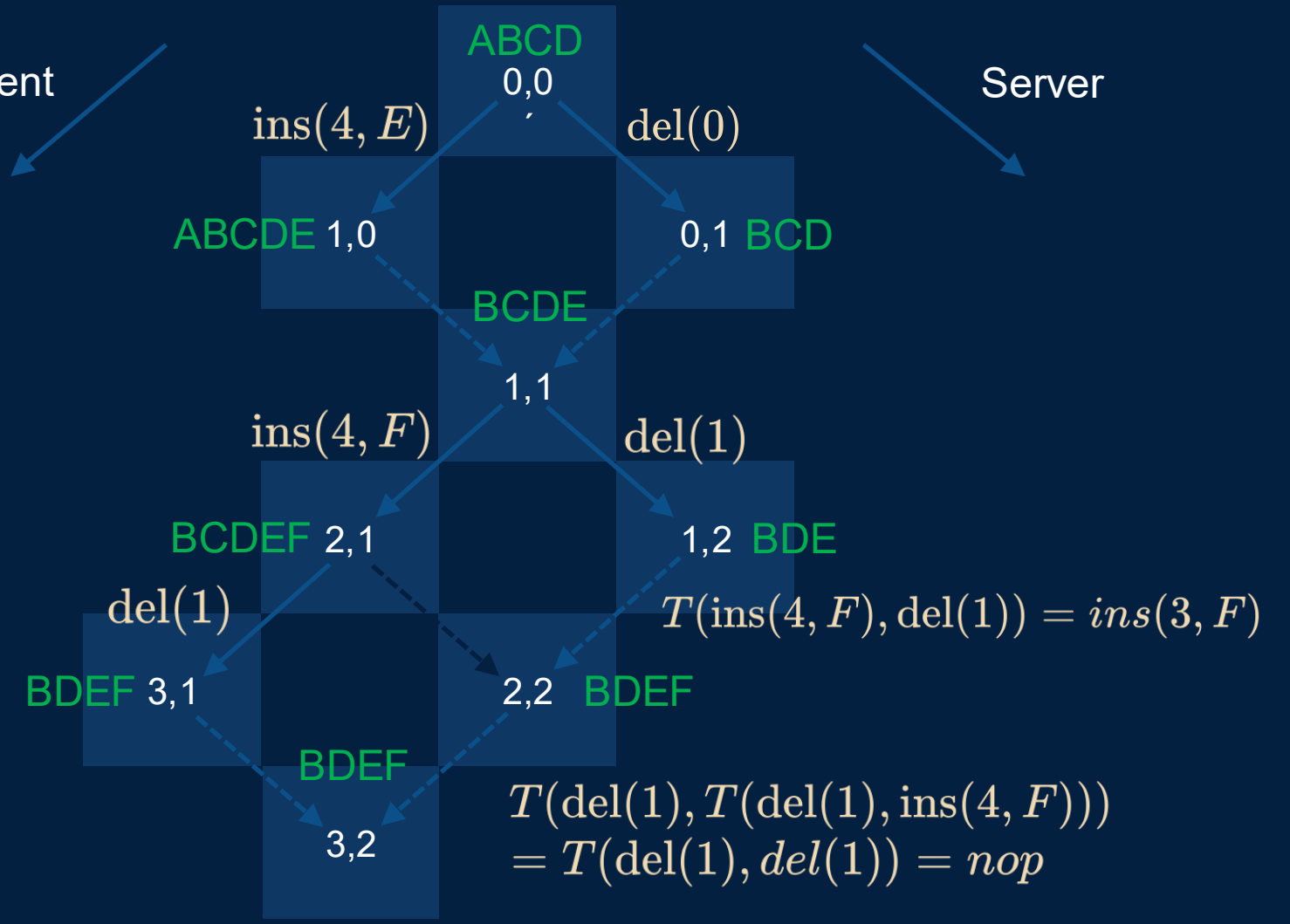
Jupiter OT



$T(\mathrm{ins}(4, F), \mathrm{del}(1)) = ins(3, F)$

$$T(\mathrm{ins}(p_1, c), \mathrm{del}(p_2)) = \begin{cases} \mathrm{ins}(p_1, c) & \text{if } p_1 \leq p_2 \\ \mathrm{ins}(p_1 - 1, c) & \text{if } p_1 > p_2 \end{cases}$$

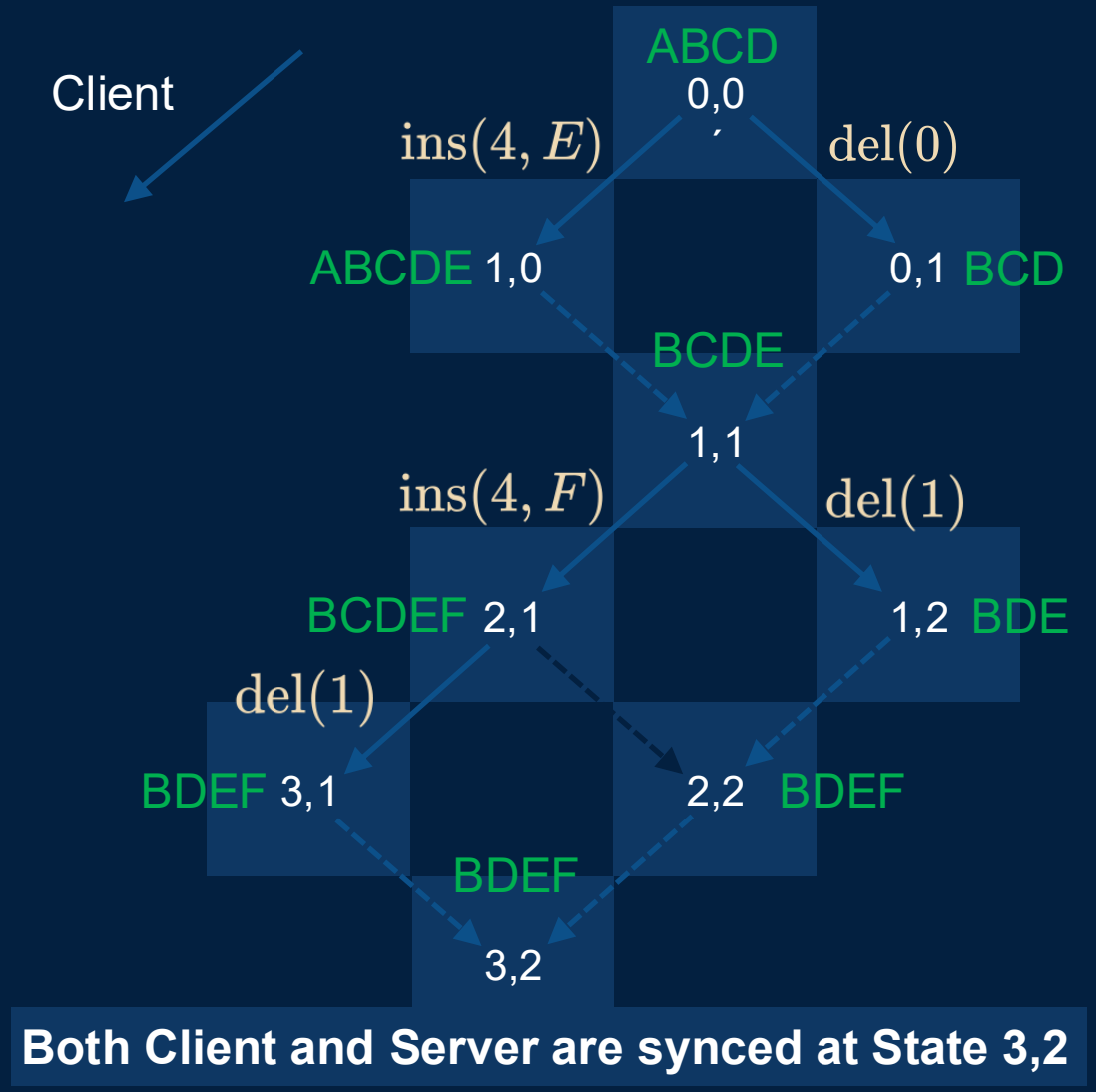
Jupiter OT



$$T(\mathrm{del}(p_1), \mathrm{ins}(p_2, c)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 + 1) & \text{if } p_1 \geq p_2 \end{cases}$$

$$T(\mathrm{del}(p_1), \mathrm{del}(p_2)) = \begin{cases} \mathrm{del}(p_1) & \text{if } p_1 < p_2 \\ \mathrm{del}(p_1 - 1) & \text{if } p_1 > p_2 \\ \mathrm{nop}() & \text{if } p_1 = p_2 \end{cases}$$

Jupiter OT



Both Client and Server are synced at State 3,2

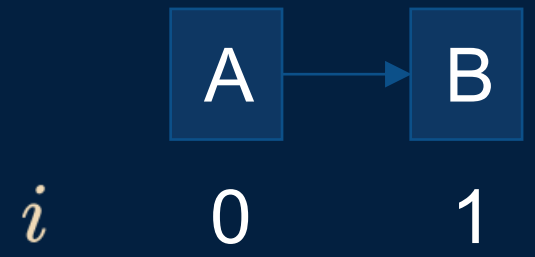
The problem with indices

$$i \in \{0, 1, 2, 3, \ldots, n\}$$



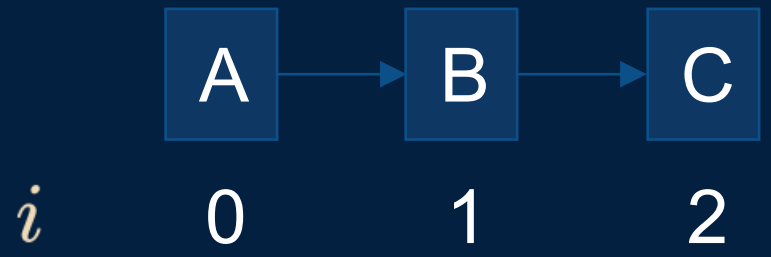
The problem with indices

$i \in \{0, 1, 2, 3, \ldots, n\}$



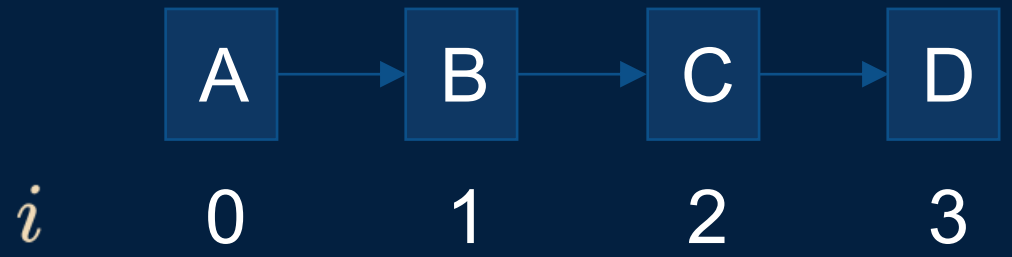
The problem with indices

$i \in \{0, 1, 2, 3, \ldots, n\}$



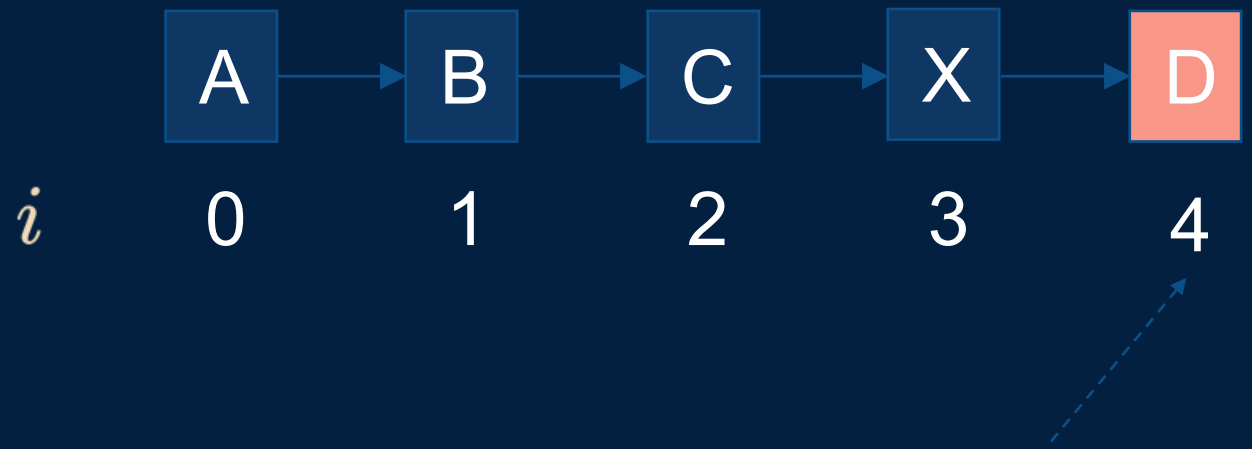
The problem with indices

$i \in \{0, 1, 2, 3, \ldots, n\}$

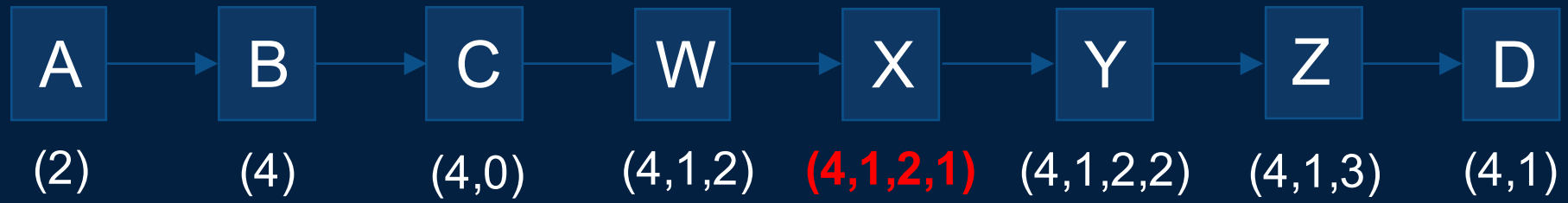


The problem with indices

$$i \in \{0, 1, 2, 3, \ldots, n\}$$

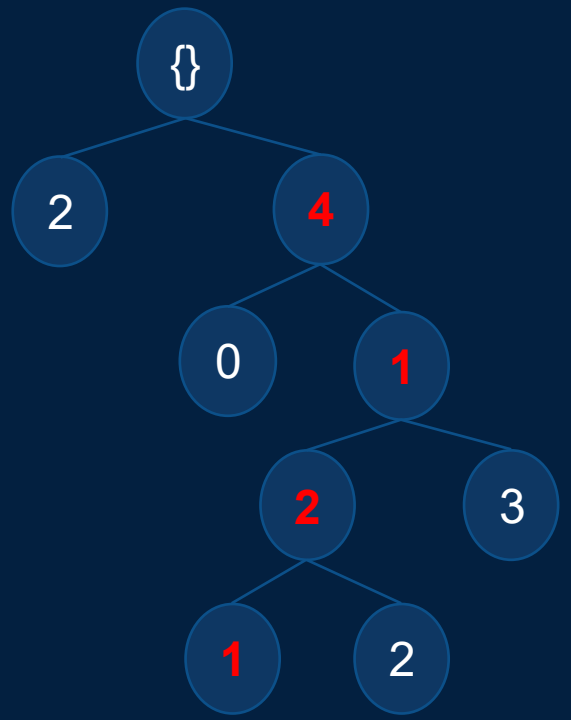


Logoot / LSEQ: identifiers as tree paths



$$\text{id} = (p_1, p_2, \ldots, p_k) \text{ is a tree path where } p_i \in \{0, 1, \ldots, \text{base}_i - 1\}$$

Unlike traditional array-based structures, **LSEQ** assign each element a **unique, immutable identifier**, which serves both as a **position** and as a **replica-independent order key**



Each new element is placed **between two existing IDs**
If there's no room at the current level, just **create a new subtree**
No **need to transform or shift other elements' IDs**.

Weiss et al., 2009: https://doi.org/10.1109/ICDCS.2009.75 · Nédelec et al., 2013: https://doi.org/10.1145/2494266.2494278

Assume Base=5

LSEQ: allocating identifiers between neighbours

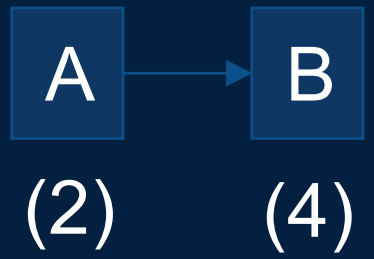
A

(2)

Randomly pick an ID between 0 and Base-1

Assume Base=5

LSEQ: allocating identifiers between neighbours



Randomly pick an ID between 3 and Base-1

Assume Base=5

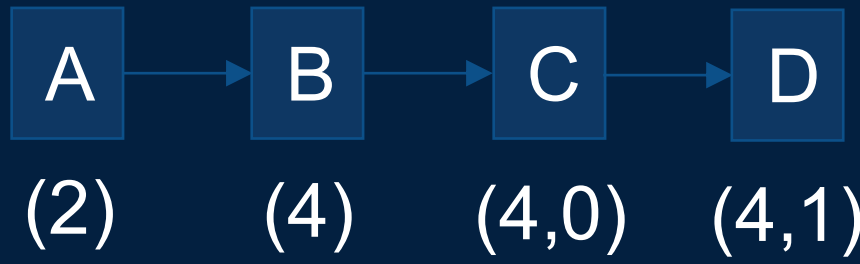
LSEQ: allocating identifiers between neighbours



We run out range, we need to split, randomly pick an id between 0 and Base-1

Assume Base=5

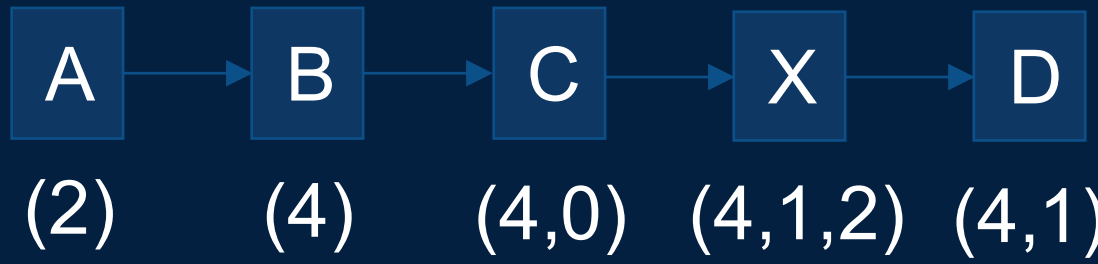
LSEQ: allocating identifiers between neighbours



Randomly pick an ID between 1 and Base-1

Assume Base=5

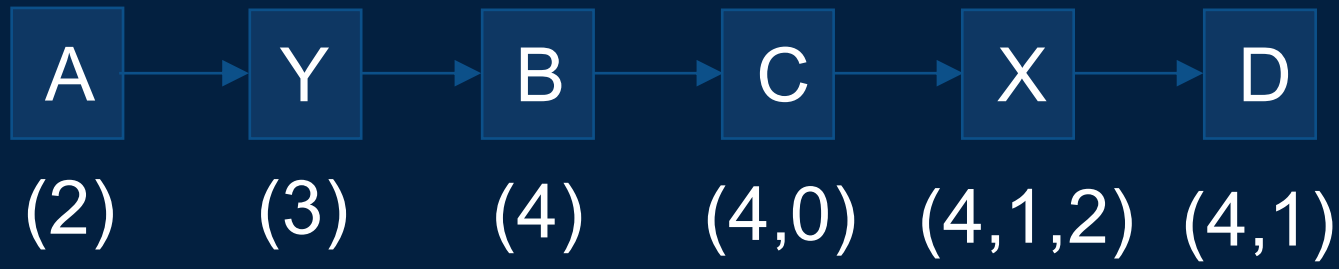
LSEQ: allocating identifiers between neighbours



We run out range, we need to split, randomly pick an id between 0 and Base-1

Assume Base=5

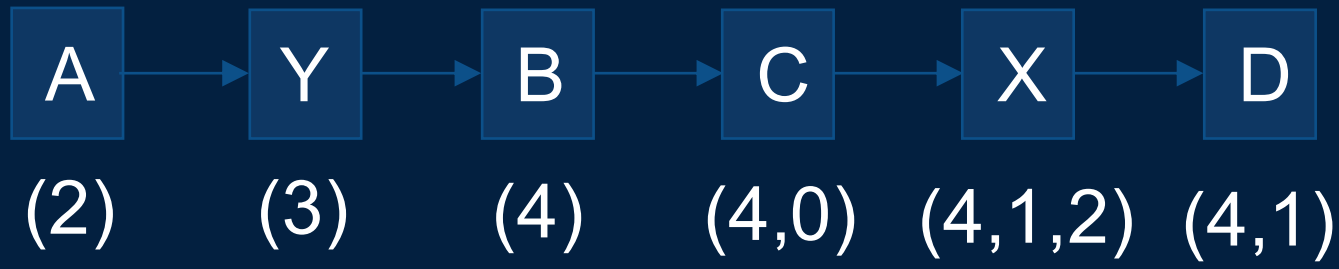
LSEQ: allocating identifiers between neighbours



Randomly pick an id between 3 and Base-1
Only 3 is possible

Assume Base=5

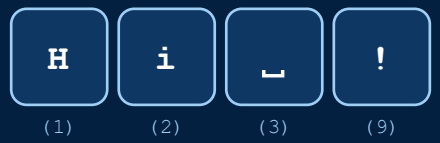
LSEQ: allocating identifiers between neighbours



Randomly pick an id between 3 and Base-1
Only 3 is possible

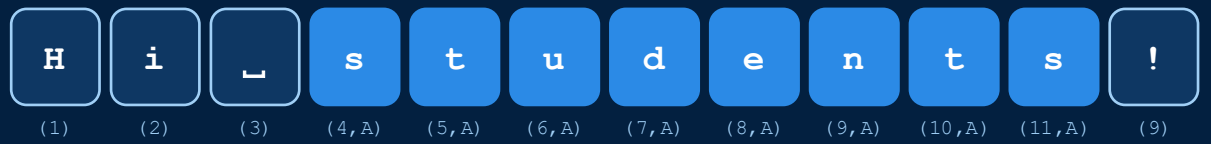
Interleaving: two users type into the same gap

Initial document, shared by both replicas

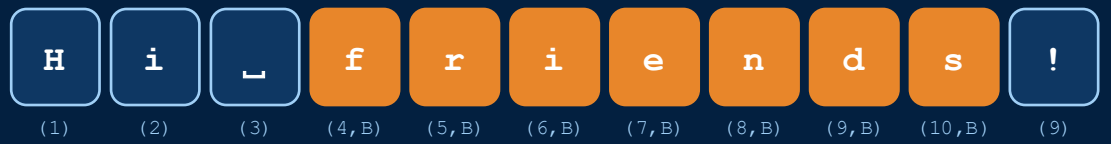


identifiers are immutable; the document is the set of characters sorted by identifier

User A (offline) types "students" between (3) and (9)



User B (offline) types "friends" between (3) and (9)



Locally, both are perfect

A sees "Hi students !"

B sees "Hi friends !"

Both allocated identifiers from 4 upwards in the same gap, tie-broken by site id (A < B).

Every allocation was legal and every replica is internally consistent. Now let them synchronise.

After synchronisation: converged — and unreadable

Merge = set union, then sort by identifier



Hi sftruideenndtss !

What held

Both replicas have exactly this document (strong eventual consistency).

No character was lost; each word's own order is preserved.

What broke

Neither user's intention survived: two words became one garbled word.

Identifiers were allocated independently in the same gap and sort interleaves them.

Not a bug in the code

Provably present in Logoot, LSEQ, Treedoc and relatives; RGA/Yjs avoid this case but not all (Kleppmann et al. 2019).

Fugue (2023) is the first with a proven minimal-interleaving property.

Kleppmann et al. 2019: https://doi.org/10.1145/3301419.3323972 · Fugue (2023): https://arxiv.org/abs/2305.00583